

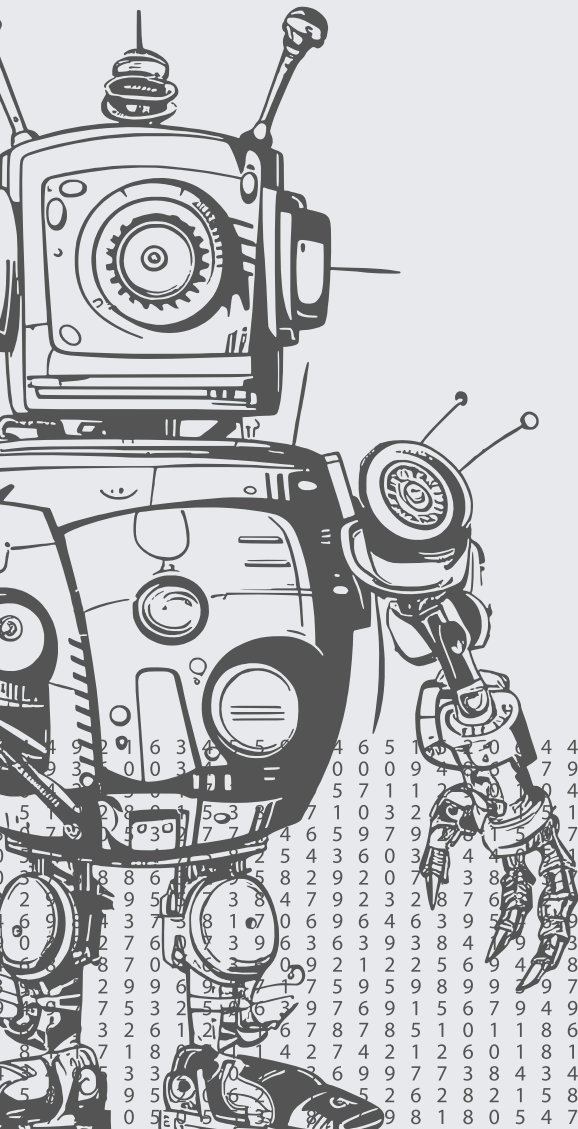
# Tutoriales Webots

Hernández Cárdenas Perla

15-sep-2026

7E

Programación de  
microprocesadores



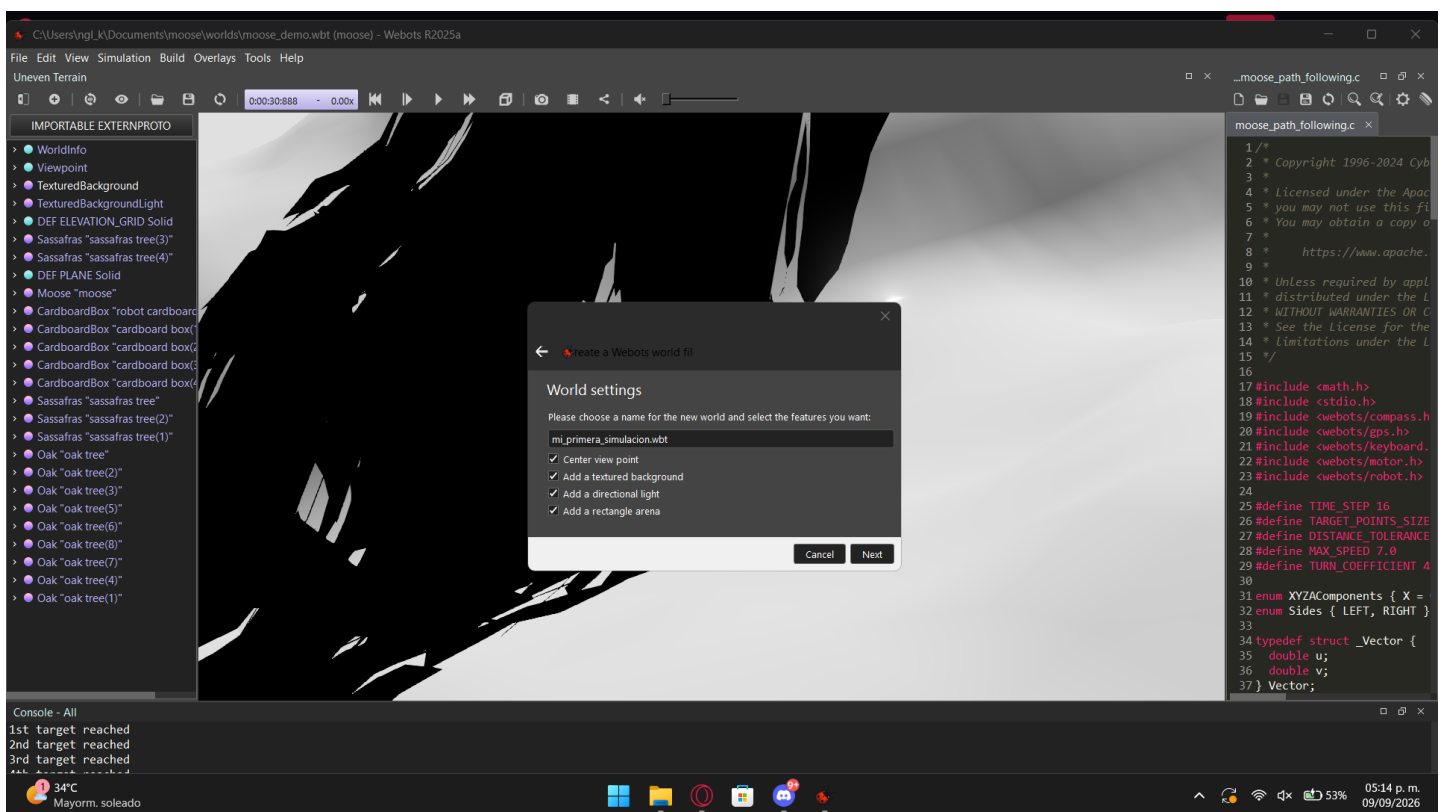
# Tutorial #1

En este primer tutorial fuimos armando nuestra simulación en Webots. Primero iniciamos el programa y creamos un nuevo mundo con una arena rectangular, suelo y paredes. Después añadimos varias cajas y las distribuimos dentro de la arena para darle forma al entorno.

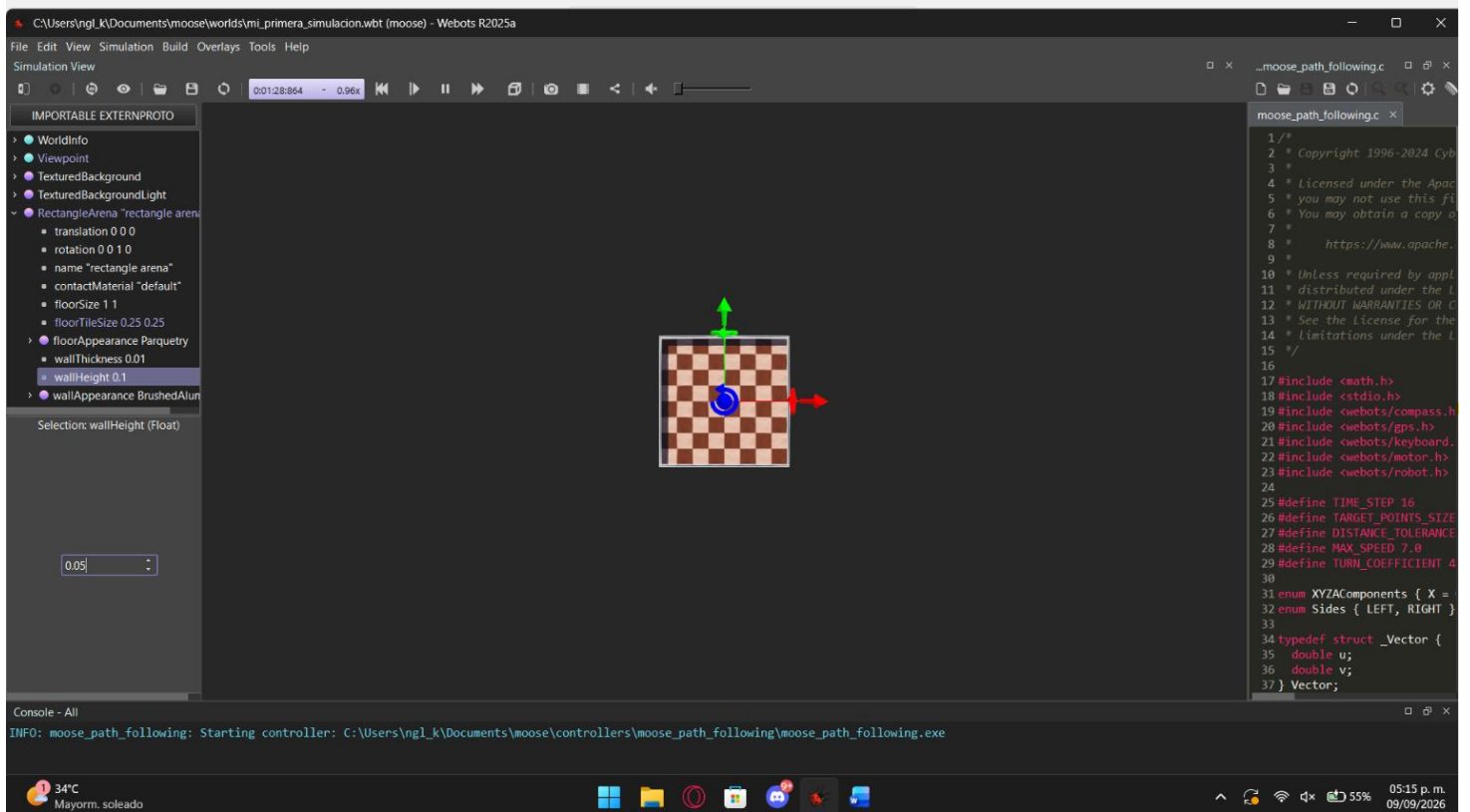
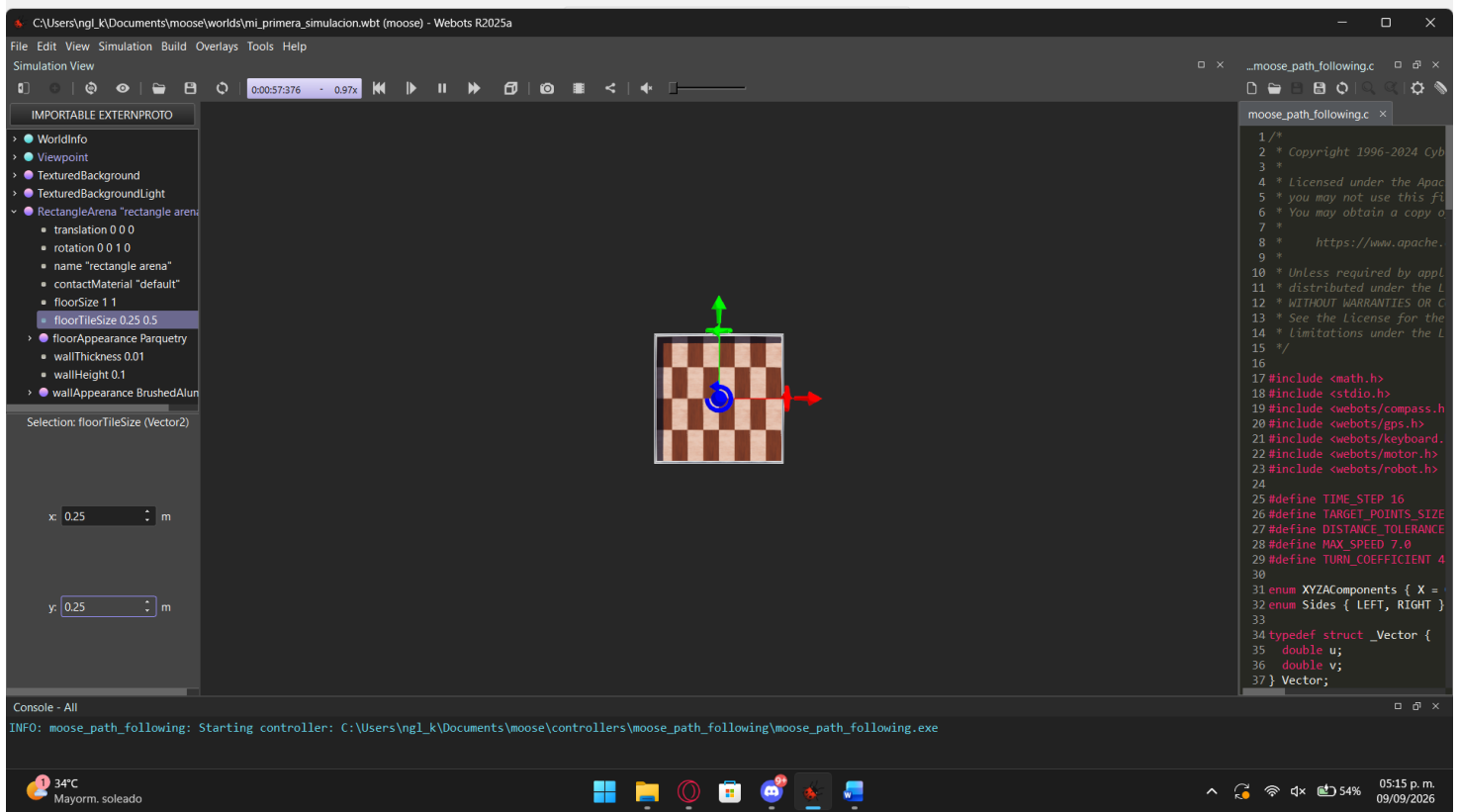
Luego importamos un robot e-puck, lo colocamos en la escena y lo guardamos. Más adelante programamos juntos un controlador sencillo en C: al inicio hicimos que las ruedas giraran una cantidad fija de radianes, y después extendimos el código para que el robot se moviera controlando la velocidad de manera continua.

Al final, nuestro robot avanzó dentro de la arena.

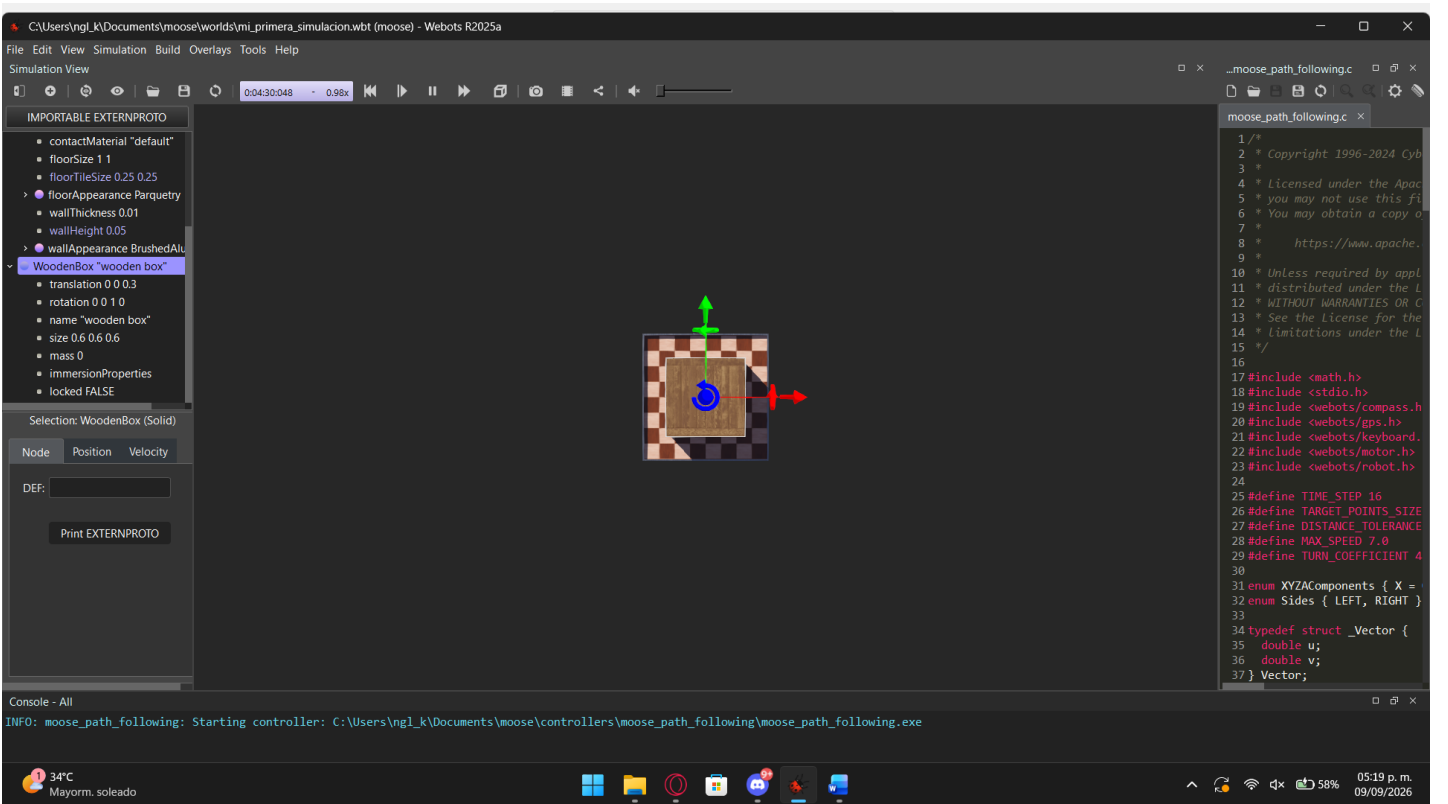
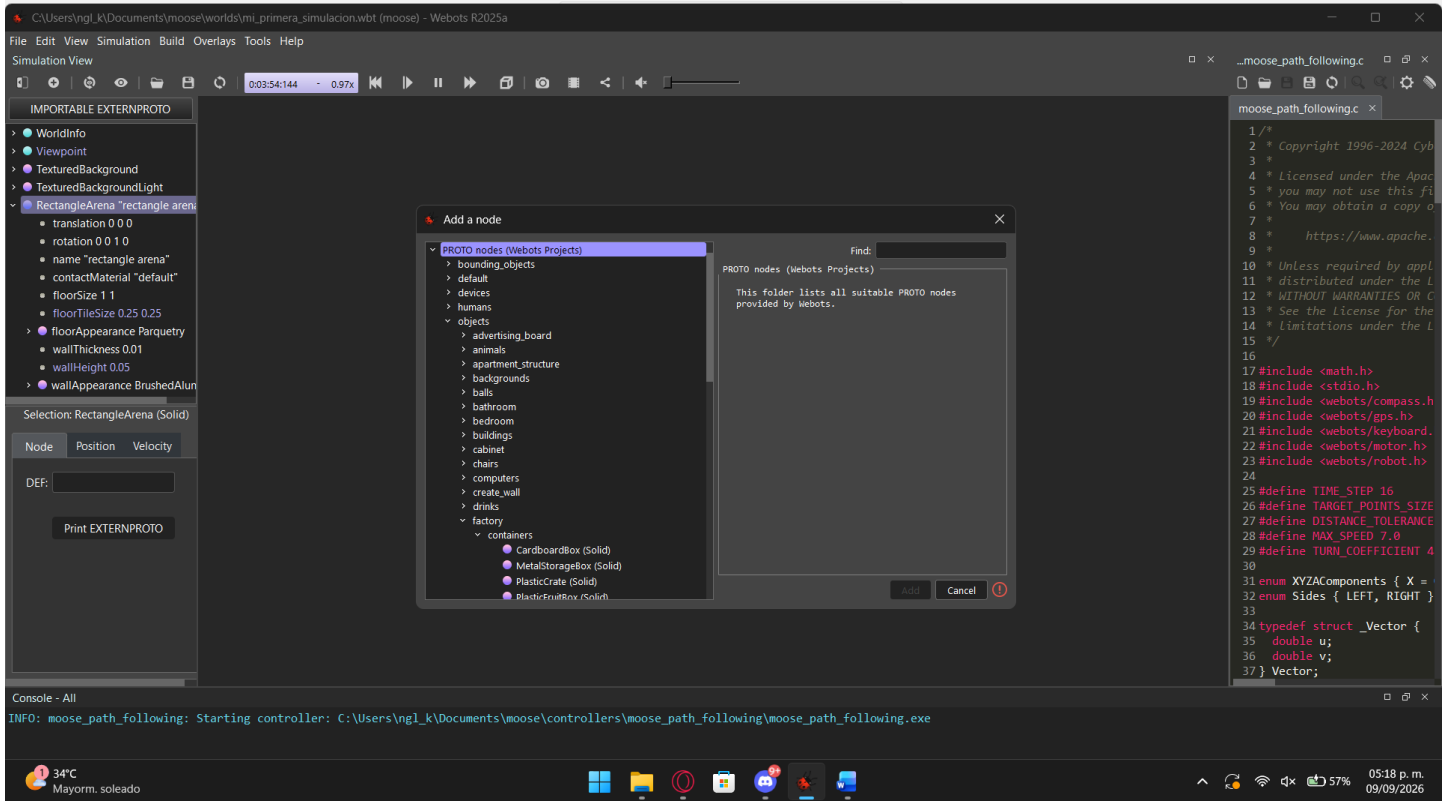
## 1. Creación y configuración del mundo.



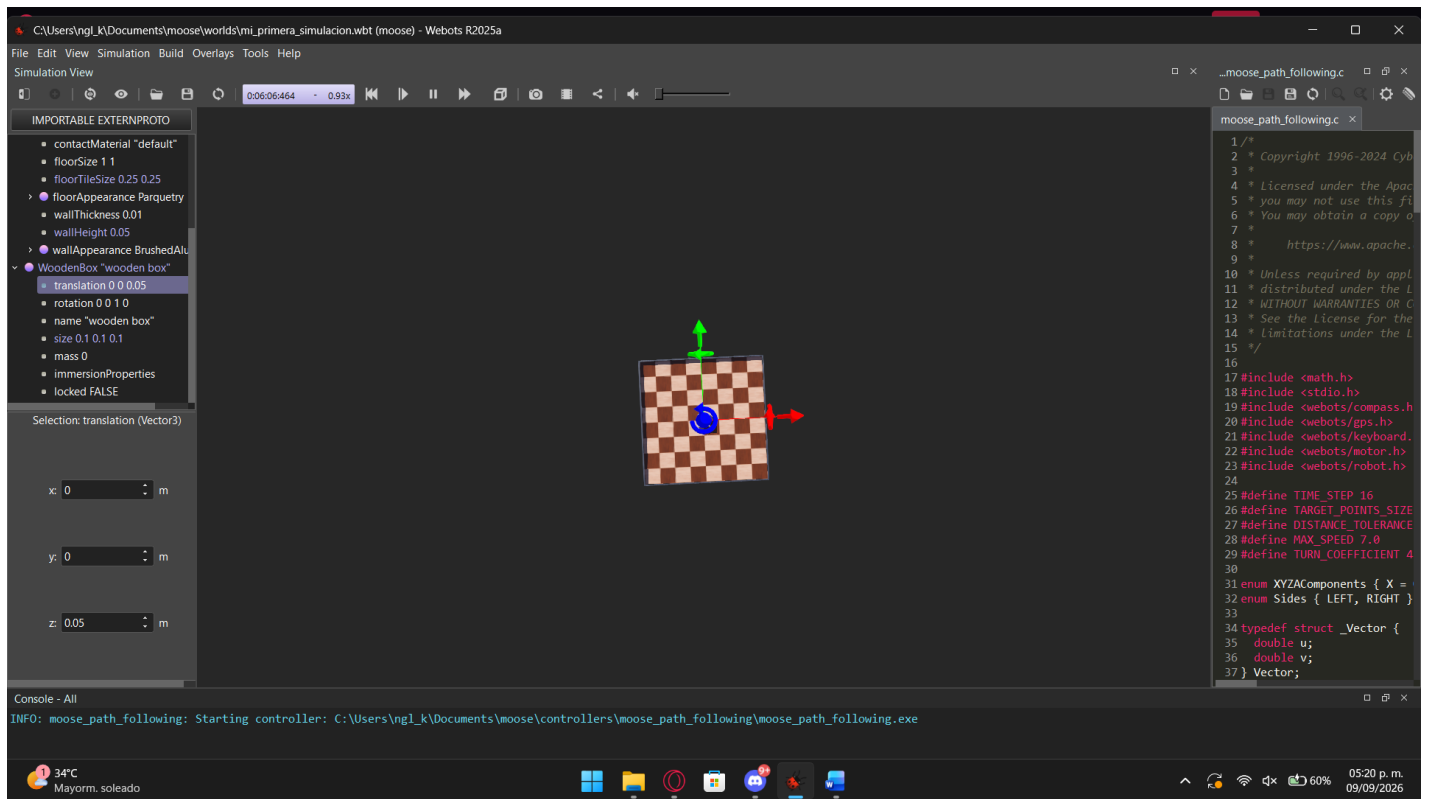
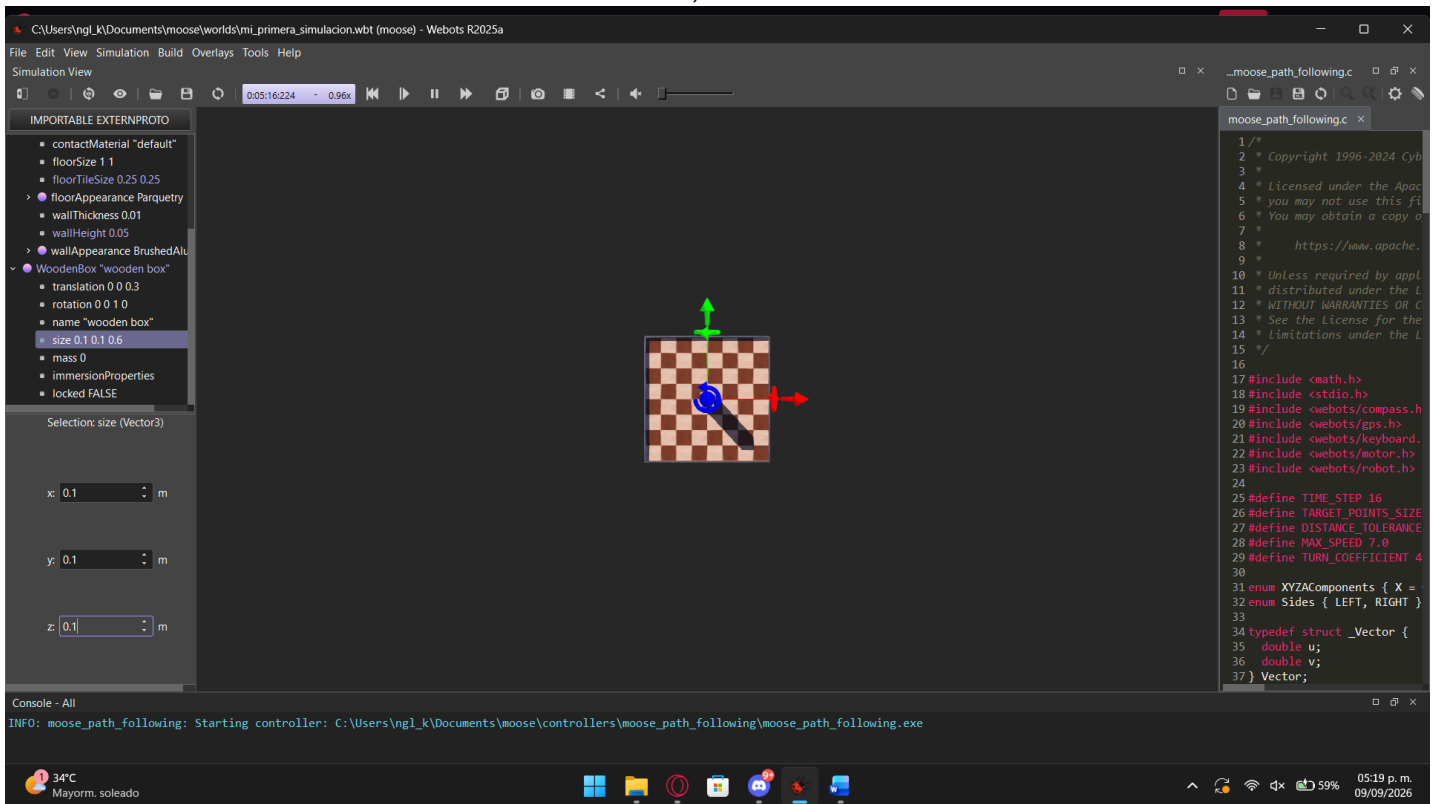
## 2. Modificación de tamaño del suelo del mundo

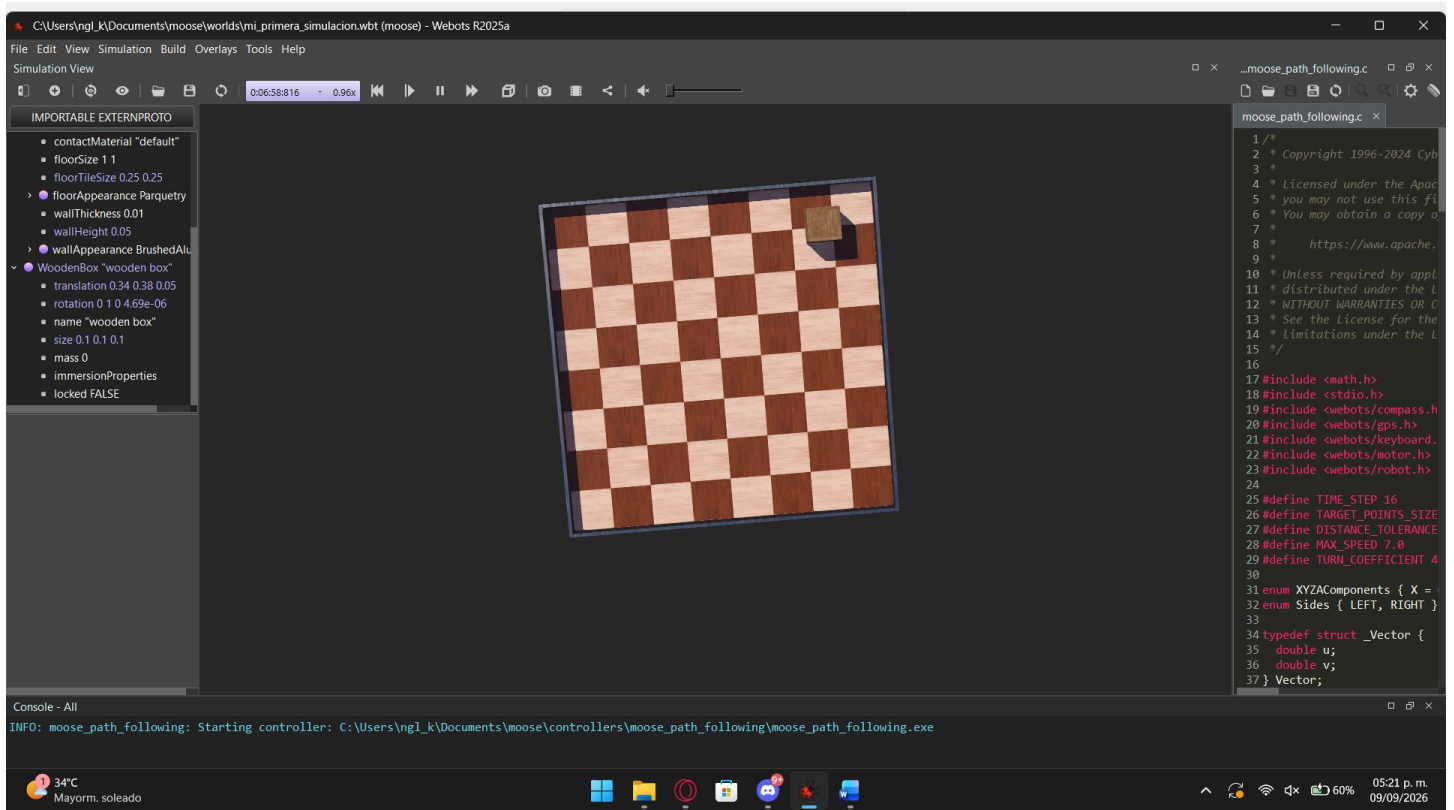


### 3. Integración de obstáculos o cajas en el mundo

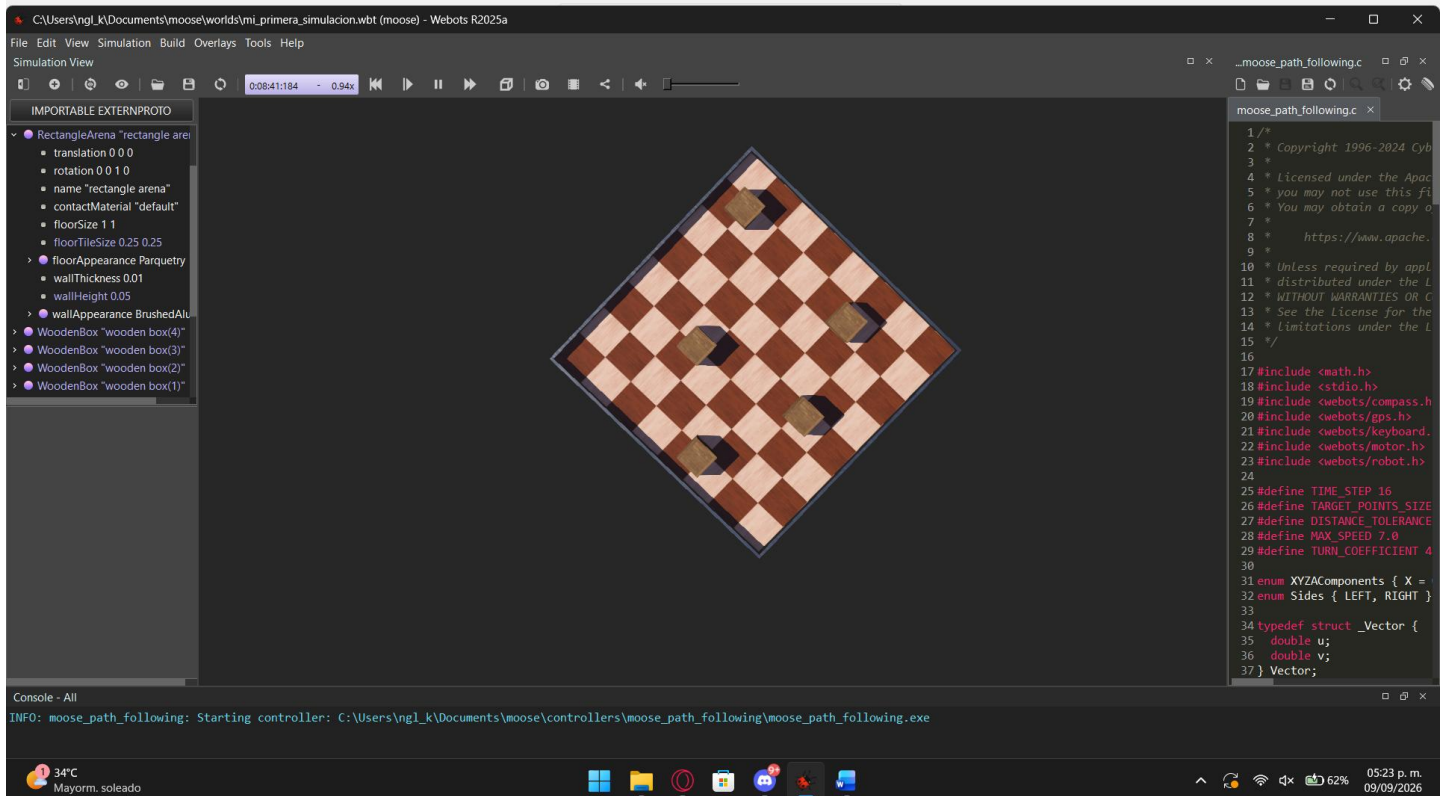


#### 4. Modificación del tamaño de los obstáculos, así como también su ubicación





## 5. Duplicar el obstáculo 4 veces mas



## 6. Añadir robot E-puck al mundo

C:\Users\ngrl\_k\Documents\moose\worlds\mi\_primer\_simulacion.wbt (moose) - Webots R2025a

File Edit View Simulation Build Overlays Tools Help

Simulation View

0:11:05:04 - 0.97x

IMPORTABLE EXTERNPROTO

- translation 0 0 0
- rotation 0 0 1 0
- name "rectangle arena"
- contactMaterial "default"
- floorSize 1 1
- floorTileSize 0.25 0.25
- floorAppearance Parquetry
- wallThickness 0.01
- wallHeight 0.05
- wallAppearance BrushedAlu
- WoodenBox "wooden box(4)"
- WoodenBox "wooden box(2)"
- WoodenBox "wooden box(1)"
- WoodenBox "wooden box"
- translation 0.33 0.26 0.05

Selection: WoodenBox (Solid)

Node Position Velocity

DEF:

Print EXTERNPROTO

Add a node

- bounding\_objects
- default
- devices
- humans
- objects
- robots
  - 3d
  - abb
  - adept
  - bitcraze
  - bluebotics
  - boston\_dynamics
  - clearpath
  - dji
  - epfl
  - epson
  - festo
  - fp\_robotics
  - franka\_emika
  - fujitsu
  - gctronic
    - E-puck (Robot)**
    - elisa
    - husarion

E-puck (Robot)

Find:

EPFL educational and research mini mobile robot. The e-puck is powered by a dsPIC processor and features a very large number of sensors in its basic configuration. More info here: <http://www.e-puck.org>.

Documentation: <https://webots.cloud/cum?url=https://github.com/cyberbotics/webots/blob/develop/projects/robots/gctronic/e-puck/protos/E-puck.proto>

License: Copyright Cyberbotics Ltd. Licensed for use only with Webots. [More information](#).

Add Cancel

\_moose\_path\_following.c

```
1 /*
2  * Copyright 1996-2024 Cyb
3  *
4  * Licensed under the Apac
5  * you may not use this fi
6  * You may obtain a copy o
7  *
8  * https://www.apache.
9  *
10 * Unless required by appl
11 * distributed under the L
12 * WITHOUT WARRANTIES OR C
13 * See the License for the
14 * Limitations under the L
15 */
16
17 #include <math.h>
18 #include <stdio.h>
19 #include <webots/compass.h>
20 #include <webots/gps.h>
21 #include <webots/keyboard.h>
22 #include <webots/motor.h>
23 #include <webots/robot.h>
24
25 #define TIME_STEP 16
26 #define TARGET_POINTS_SIZE
27 #define DISTANCE_TOLERANCE
28 #define MAX_SPEED 7.0
29 #define TURN_COEFFICIENT 4
30
31 enum XYZComponents { X =
32 enum Sides { LEFT, RIGHT }
33
34 typedef struct _Vector {
35     double u;
36     double v;
37 } Vector;
```

Console - All

INFO: moose\_path\_following: Starting controller: C:\Users\ngrl\_k\Documents\moose\controllers\moose\_path\_following\moose\_path\_following.exe

34°C  
Mayorm. soleado

05:25 p. m.  
09/09/2026

C:\Users\ngrl\_k\Documents\moose\worlds\mi\_primer\_simulacion.wbt (moose) - Webots R2025a

File Edit View Simulation Build Overlays Tools Help

Simulation View

0:13:20:384 - 0.97x

IMPORTABLE EXTERNPROTO

- translation 0 0 0
- rotation 0 0 1 0
- name "rectangle arena"
- contactMaterial "default"
- floorSize 1 1
- floorTileSize 0.25 0.25
- floorAppearance Parquetry
- wallThickness 0.01
- wallHeight 0.05
- wallAppearance BrushedAlu
- WoodenBox "wooden box(4)"
- E-puck "e-puck"**
- WoodenBox "wooden box(2)"
- WoodenBox "wooden box(1)"
- WoodenBox "wooden box"

Console - All

INFO: moose\_path\_following: Starting controller: C:\Users\ngrl\_k\Documents\moose\controllers\moose\_path\_following\moose\_path\_following.exe

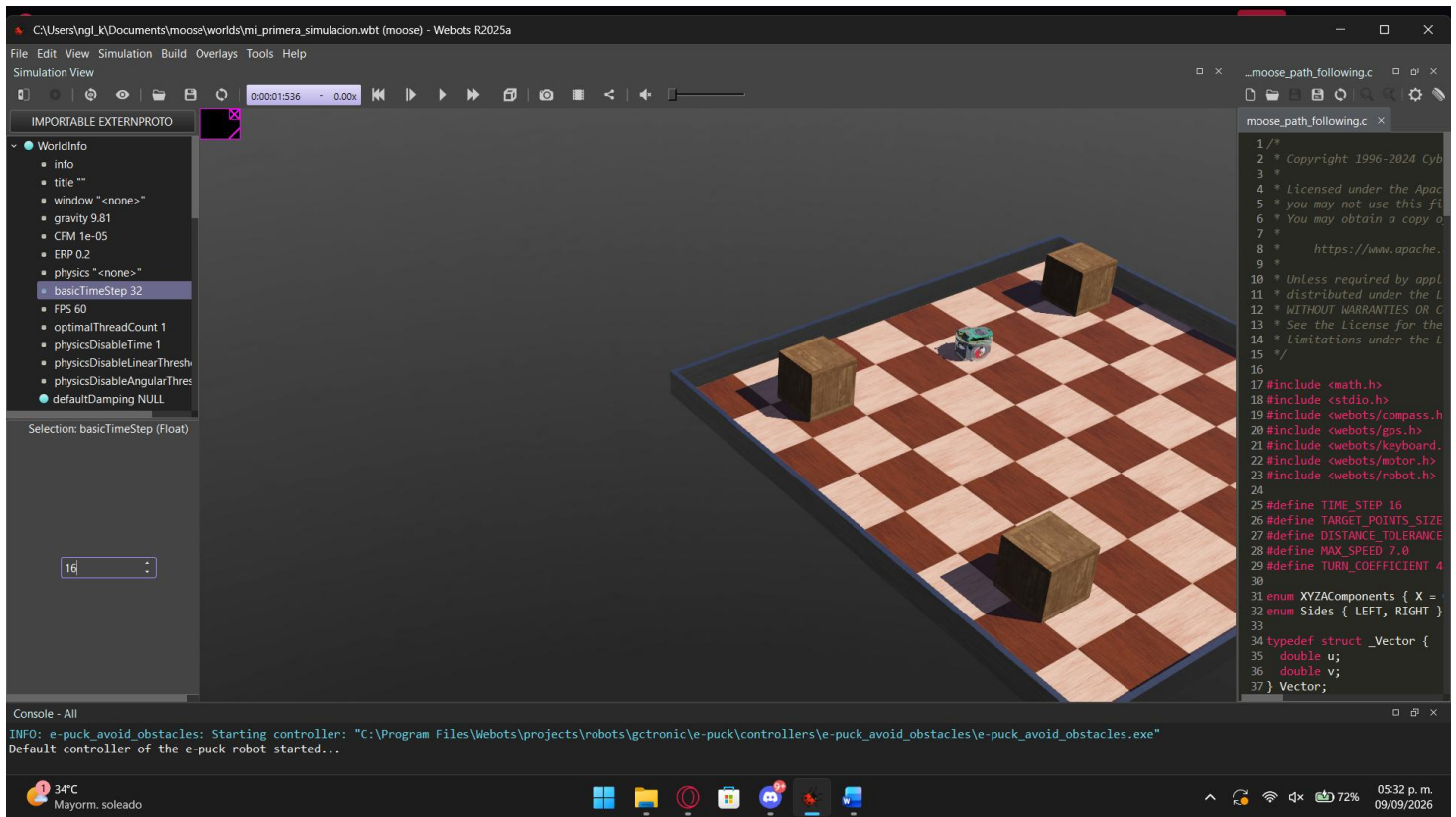
INFO: e-puck\_avoid\_obstacles: Starting controller: "C:\Program Files\Webots\projects\robots\gctronic\e-puck\controllers\e-puck\_avoid\_obstacles\e-puck\_avoid\_obstacles.exe"

Default controller of the e-puck robot started...

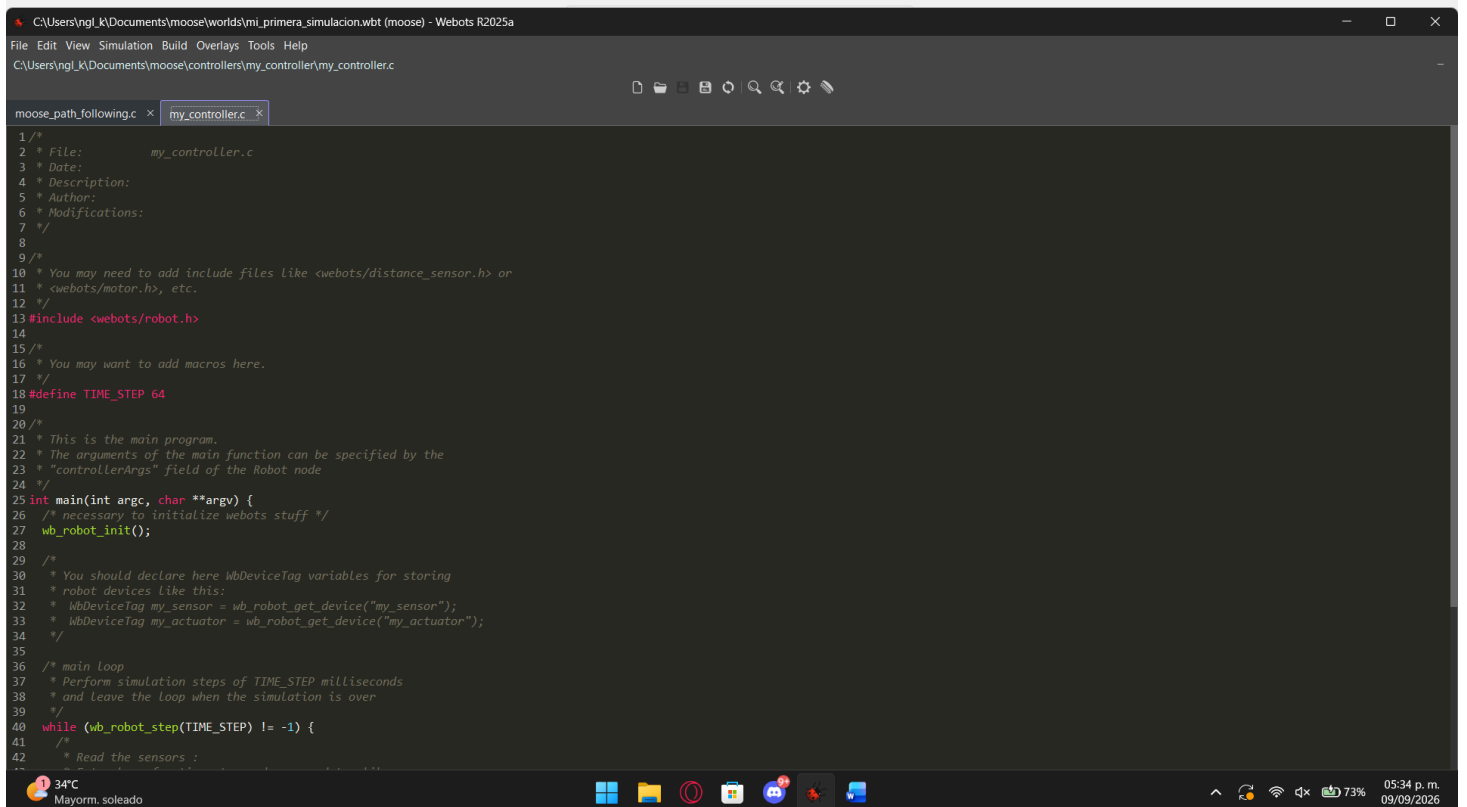
34°C  
Mayorm. soleado

05:28 p. m.  
09/09/2026

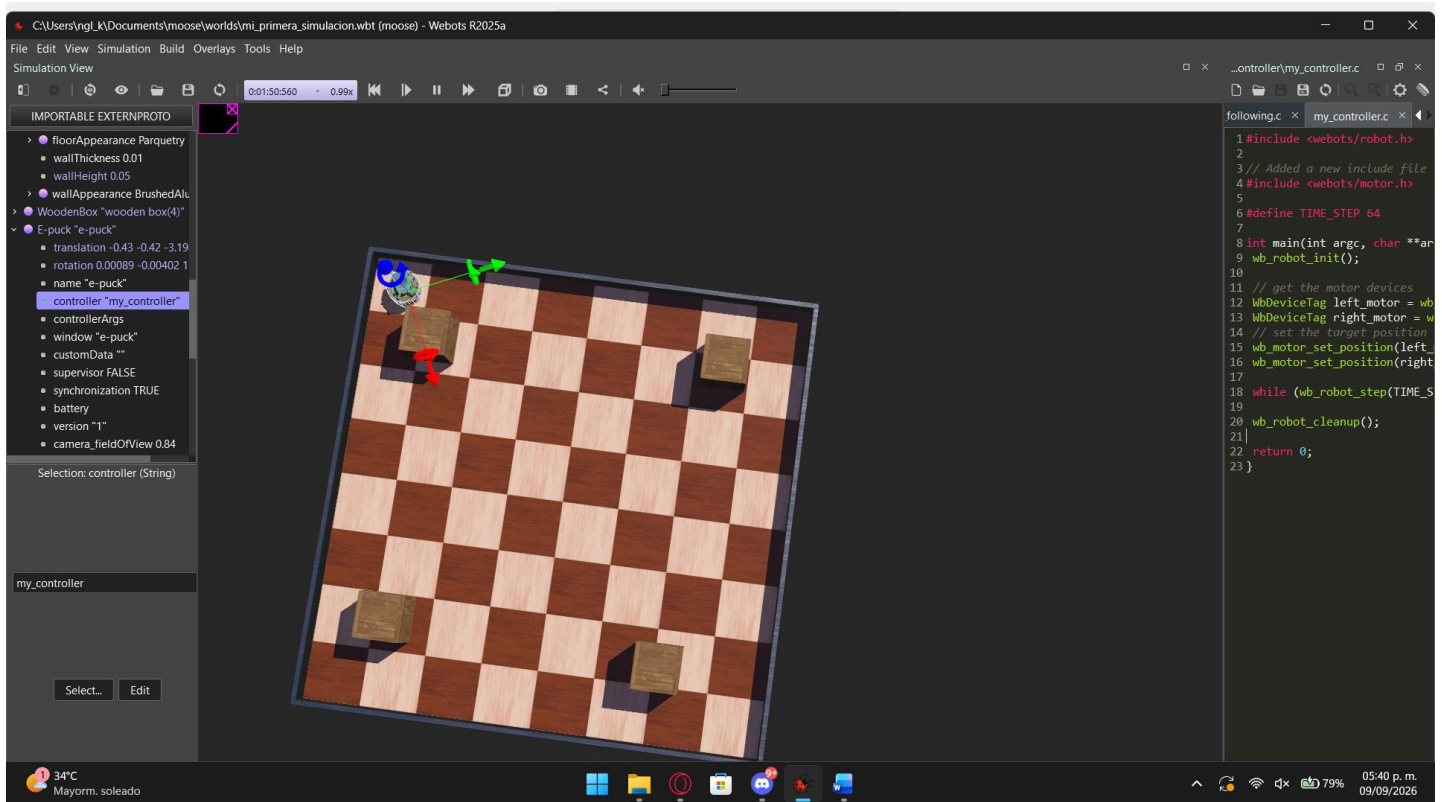
## 7. Modificar basicTimeStep del robot a 16



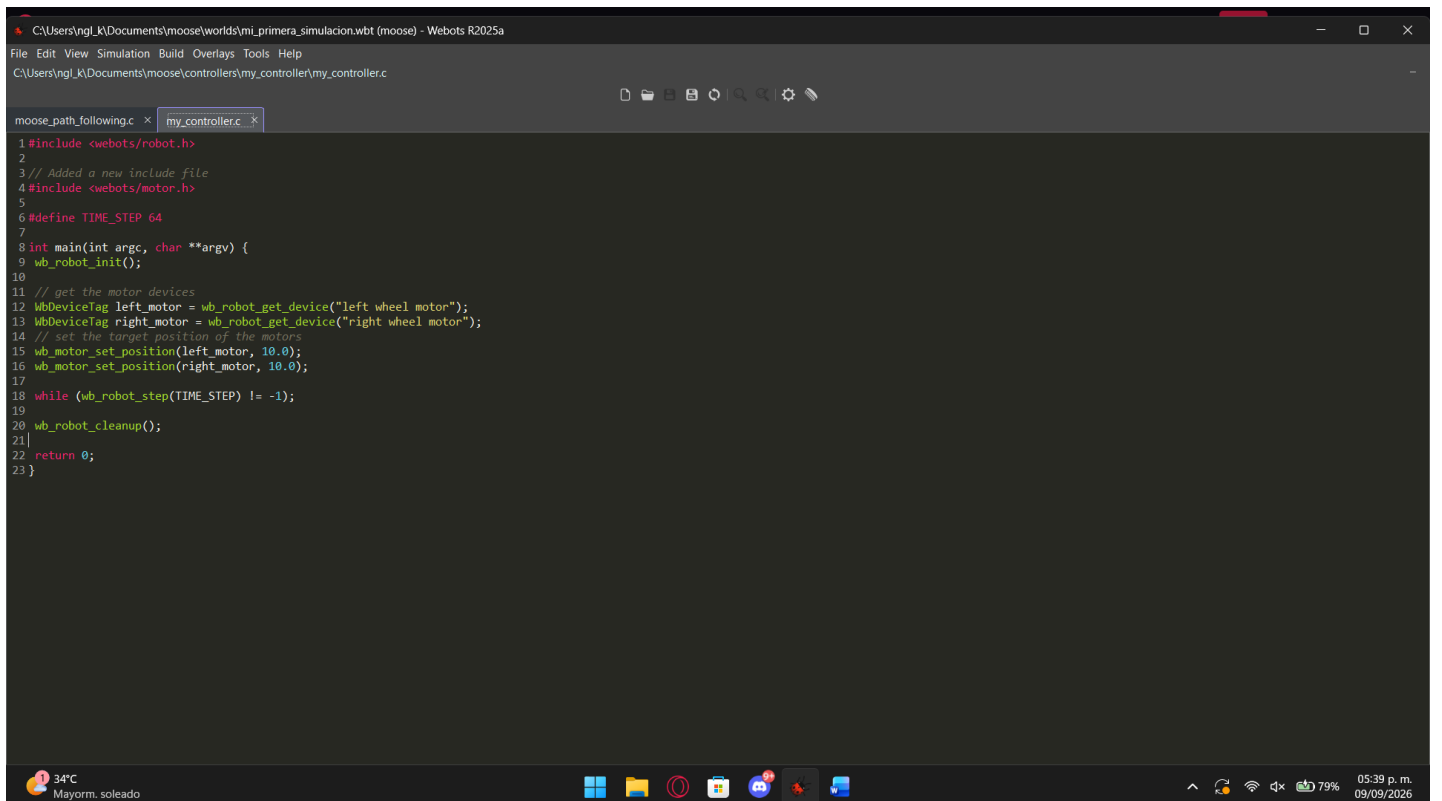
## 8.- Creador archivo controlador de código tipo C para el robot E-puck



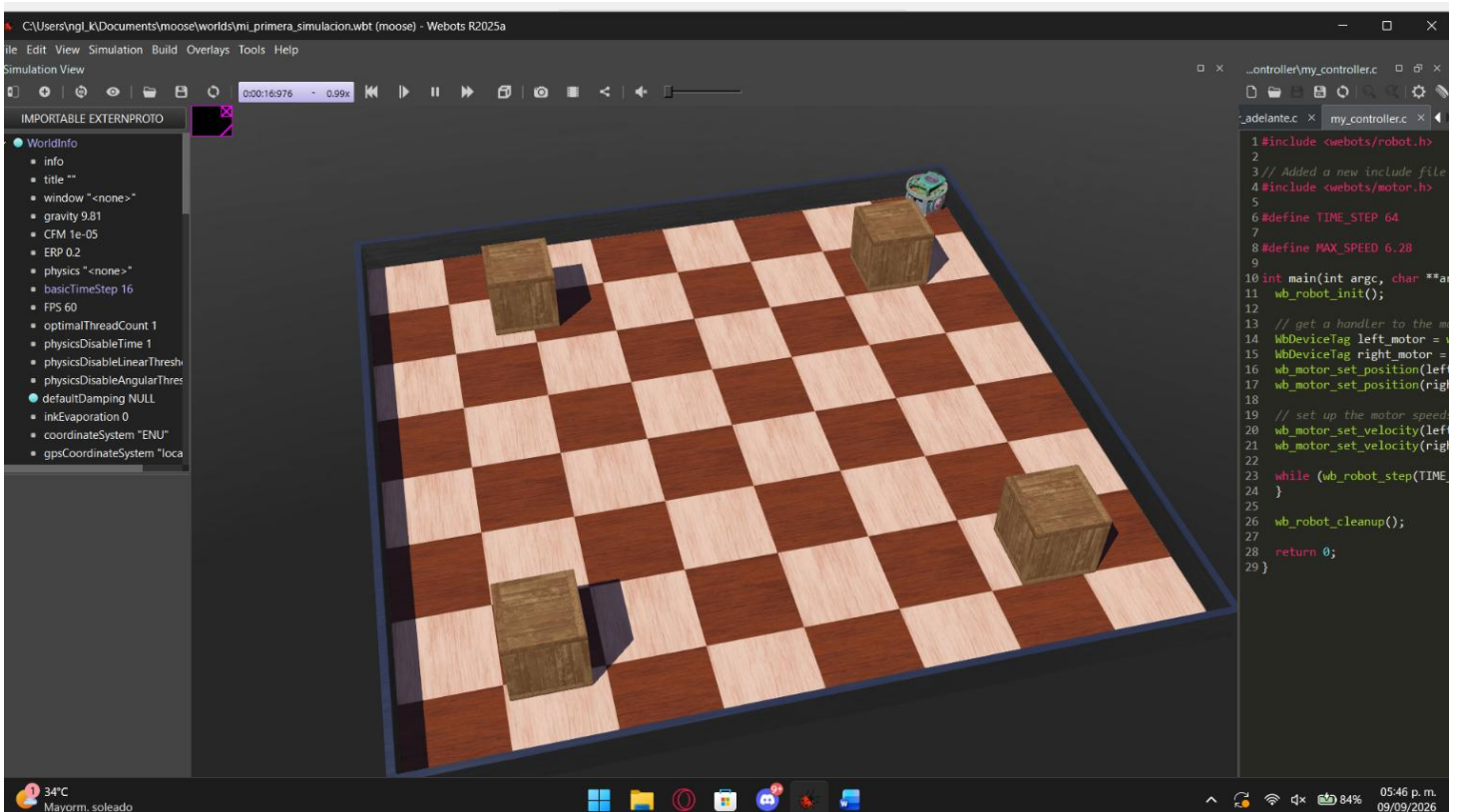
## 9.- Asignar el controlador anteriormente creador a nuestro robot E-puck



## 10.- Modificar código del controlador del robot



## Ejecución del tutorial



## TUTORIAL #2

En este segundo tutorial trabajamos en modificar el entorno de la simulación. Primero guardamos el mundo anterior con un nuevo nombre y reemplazamos el suelo por uno más sencillo. Después aprendimos a usar el nodo **Sólido**, que nos permitió definir cuerpos rígidos con geometría, física y límites de colisión.

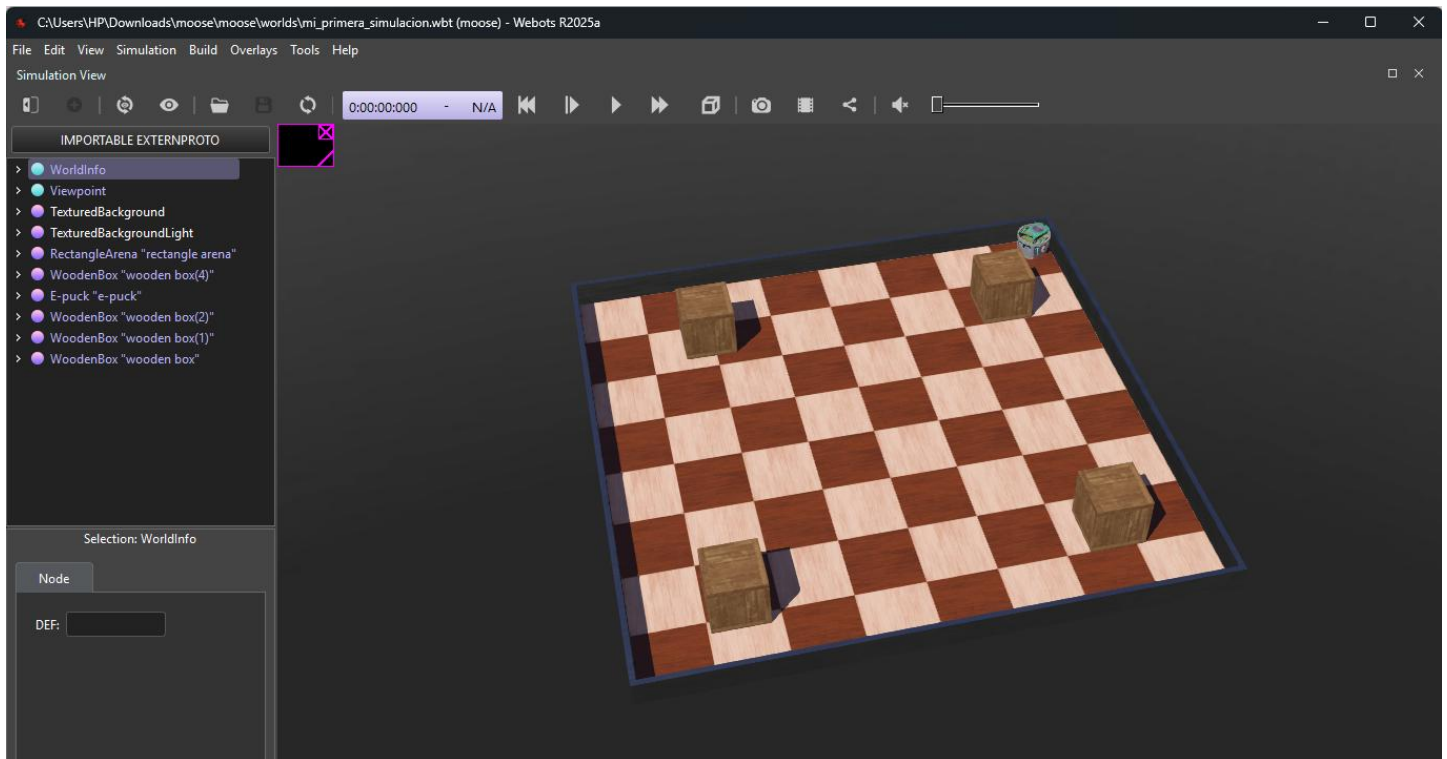
Creamos una pelota como cuerpo rígido usando nodos de esfera, le añadimos apariencia, boundingObject y física, y la colocamos frente al robot para que interactuara con el entorno. Luego aplicamos el mecanismo **DEF-USE** para reutilizar la misma definición de la esfera y evitar redundancia.

Más adelante añadimos cuatro muros estáticos alrededor del suelo, usando nodos de caja y también aprovechando el DEF-USE. Al final, nuestro entorno quedó con un piso, paredes y una pelota que podía moverse bajo la acción de la física.

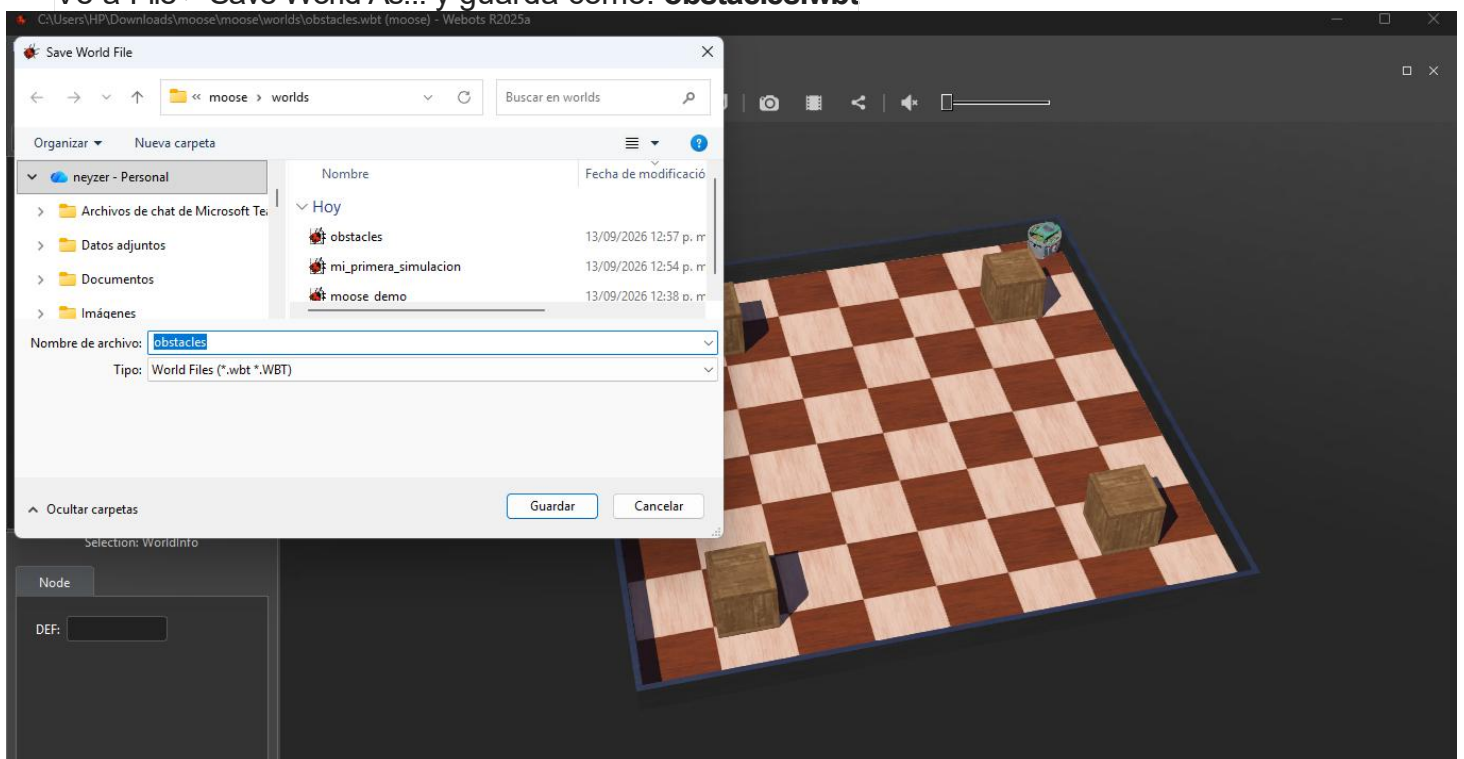
### PRACTICA1

Asegúrate de que my\_first\_simulation.wbt esté abierto

Verifica que la simulación esté pausada y en tiempo virtual 0:00:00:000 (usa el botón Reset si es necesario)



Ve a File > Save World As... y guarda como: **obstacles.wbt**

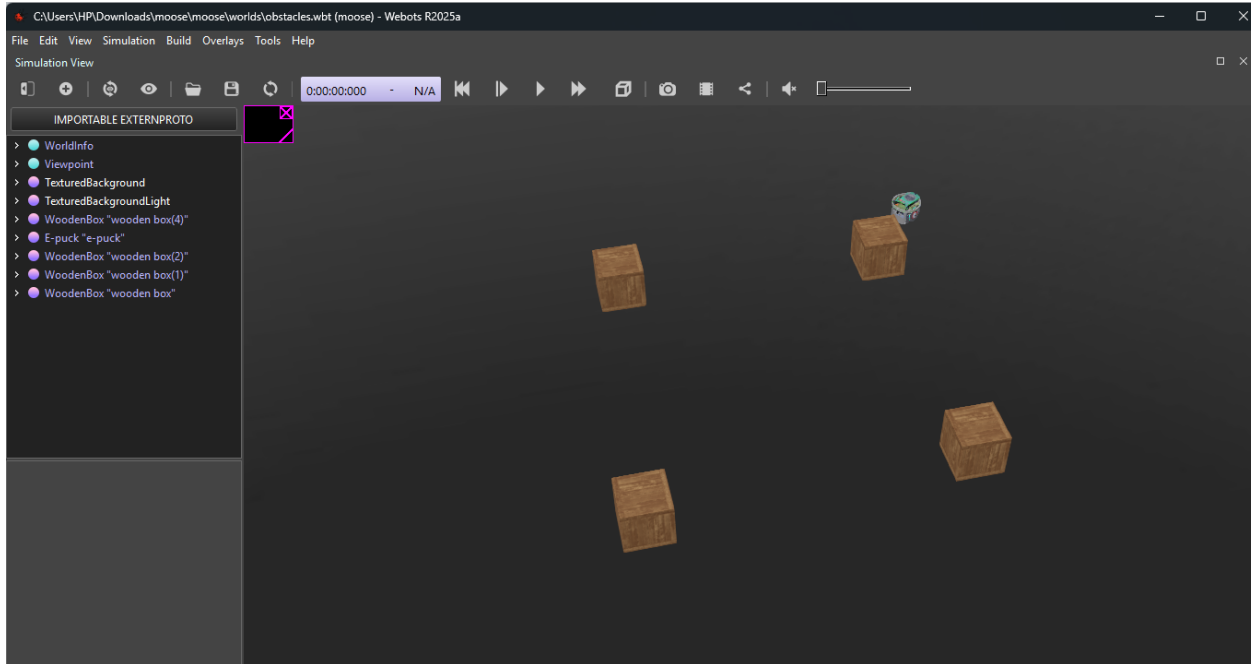


## PRACTICA2

### Modificar el Suelo

Selecciona el nodo RectangleArena en el Scene Tree (árbol de escena)

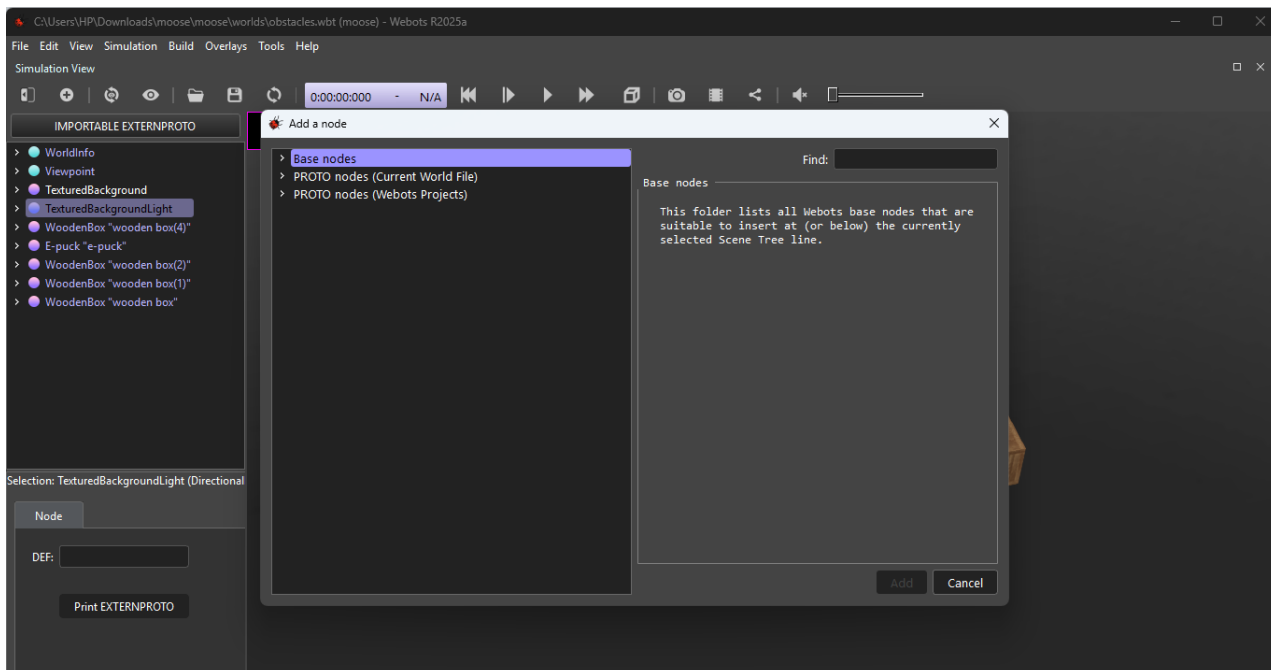
Presiona la tecla Delete en tu teclado



### Añadir un Floor nuevo:

Selecciona el nodo TexturedBackgroundLight

Haz clic en el botón Add (Añadir)

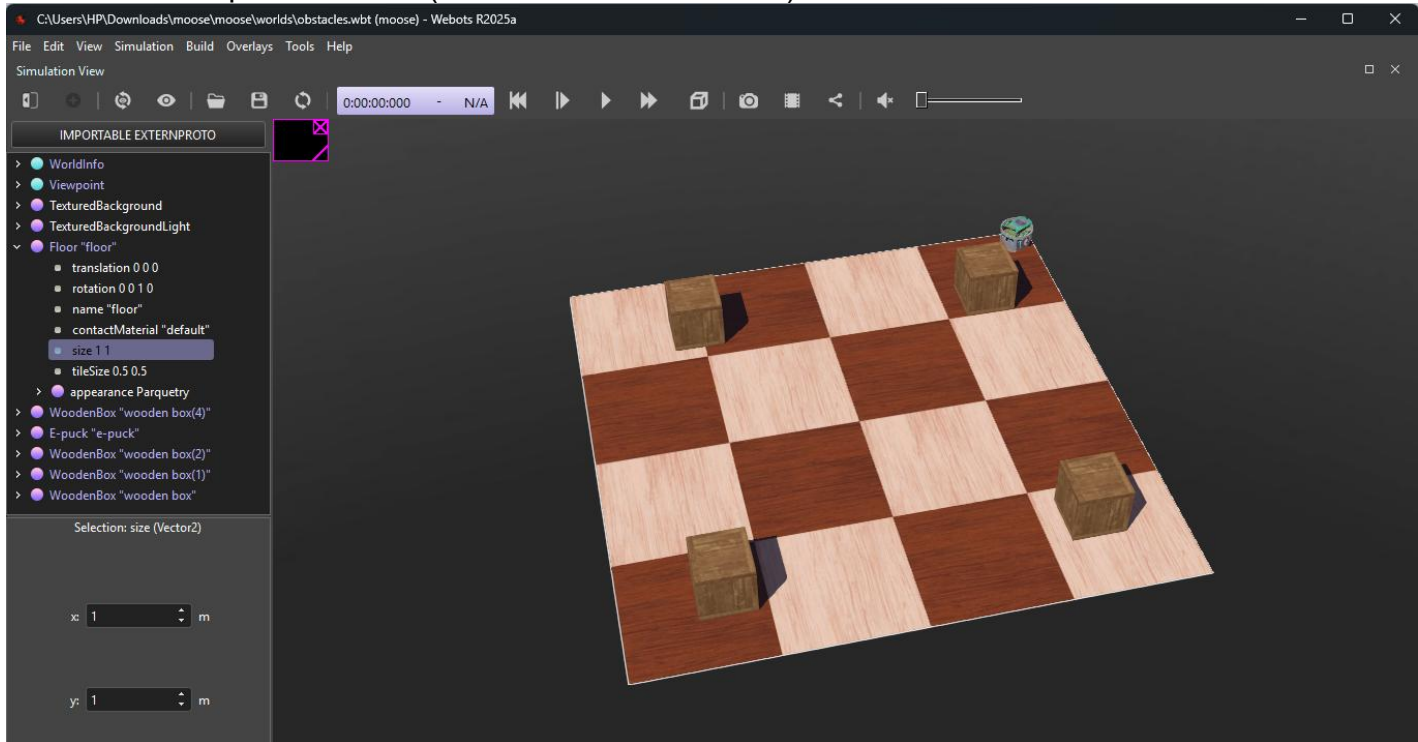


## PRACTICA3

Modificar tamaño del Floor

Expande el nodo Floor en el Scene Tree

Modifica el campo size a: 1 1 (esto lo hace de 1m x 1m)

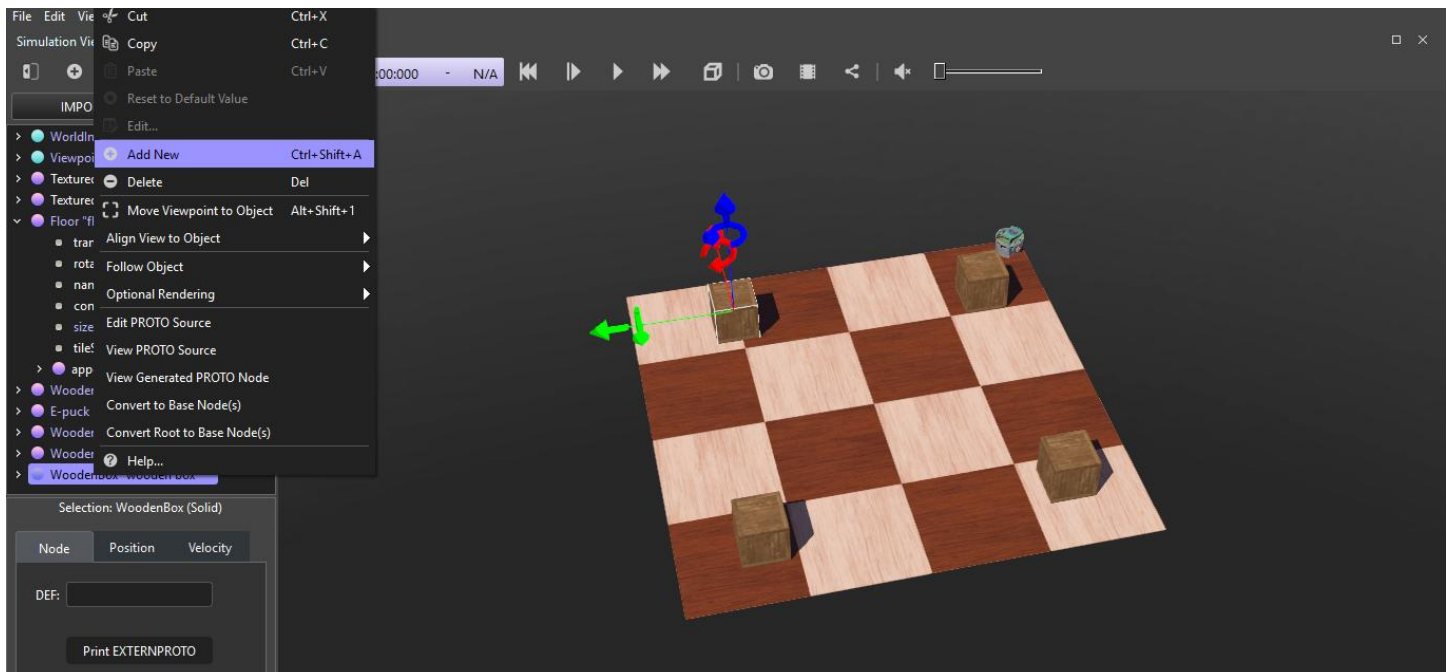


## PRACTICA4

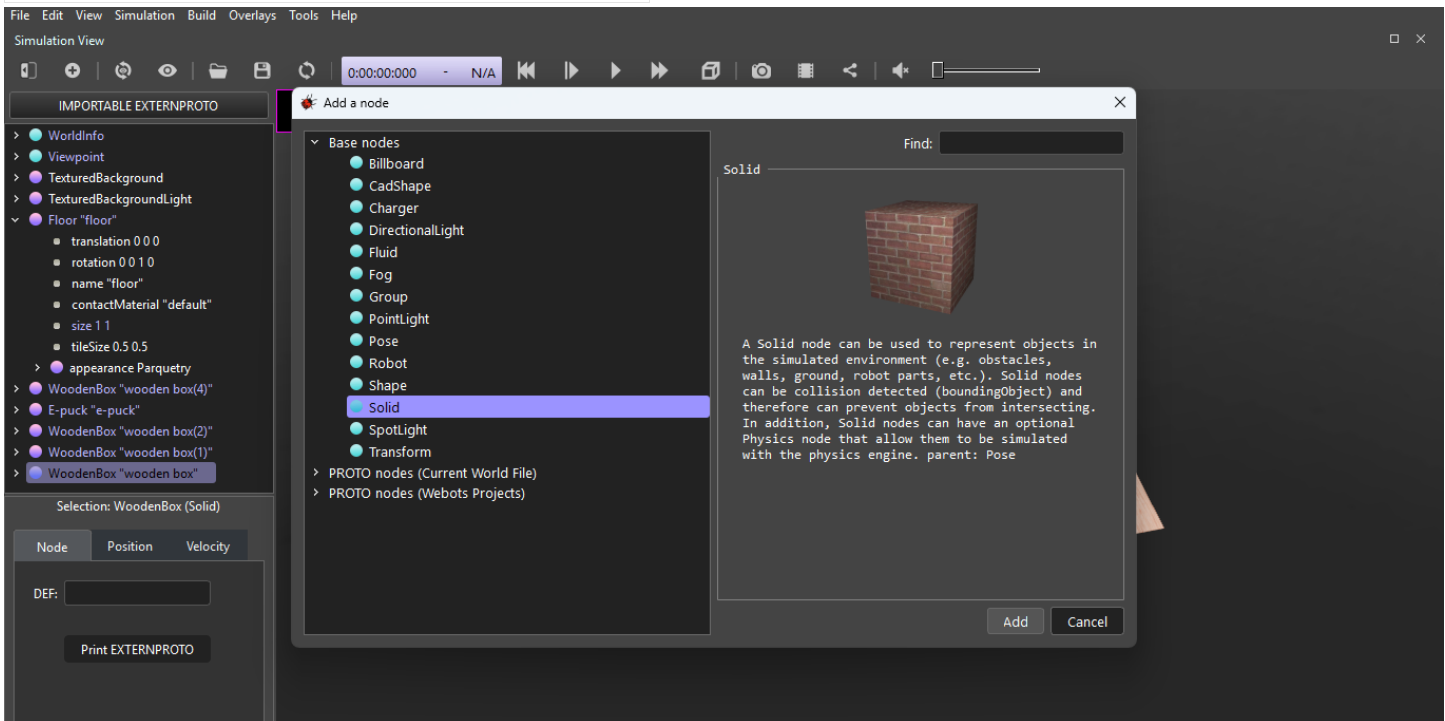
Crear una Bola (Rigid Body)

Ahora crearemos una bola que interactuará con la física. Este es el concepto más importante: el nodo Solid (cuerpo rígido).

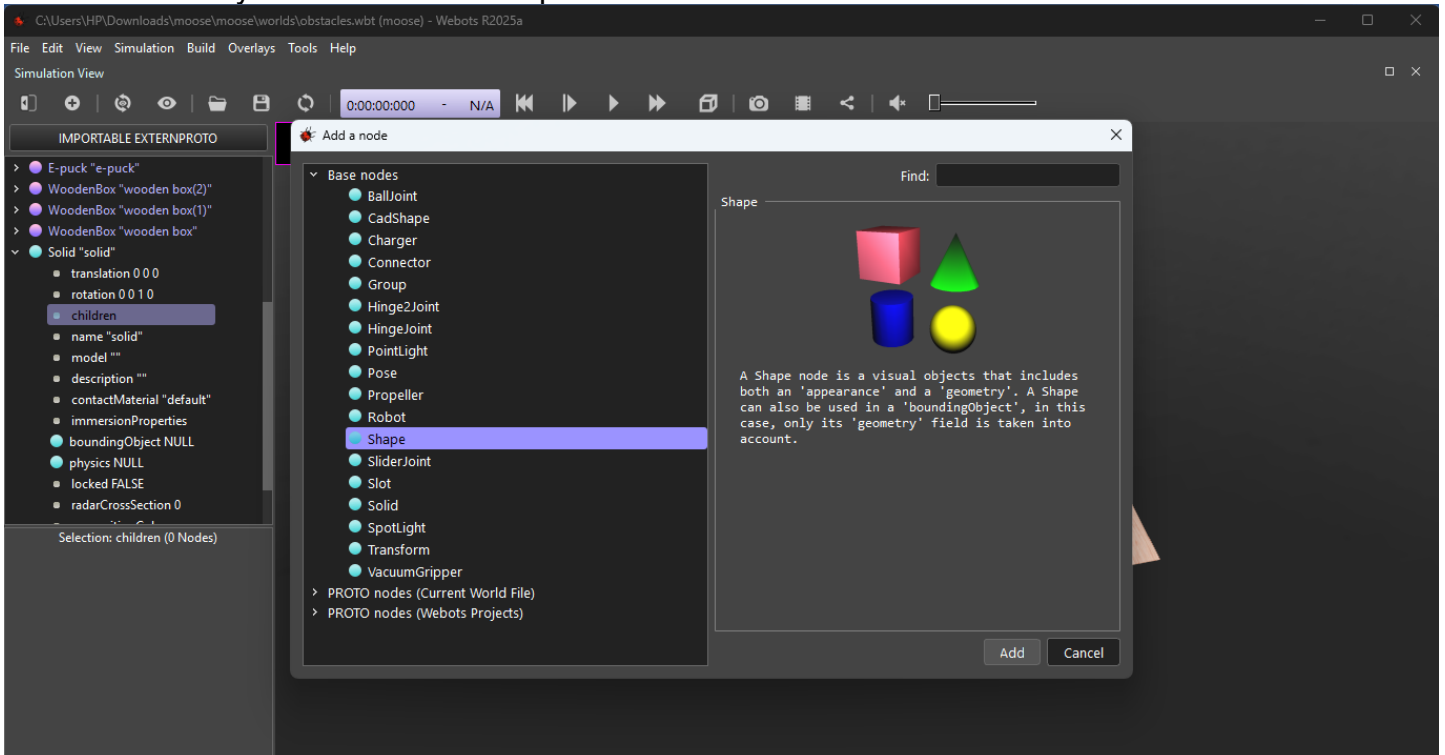
Selecciona el último nodo en el Scene Tree y presiona Add



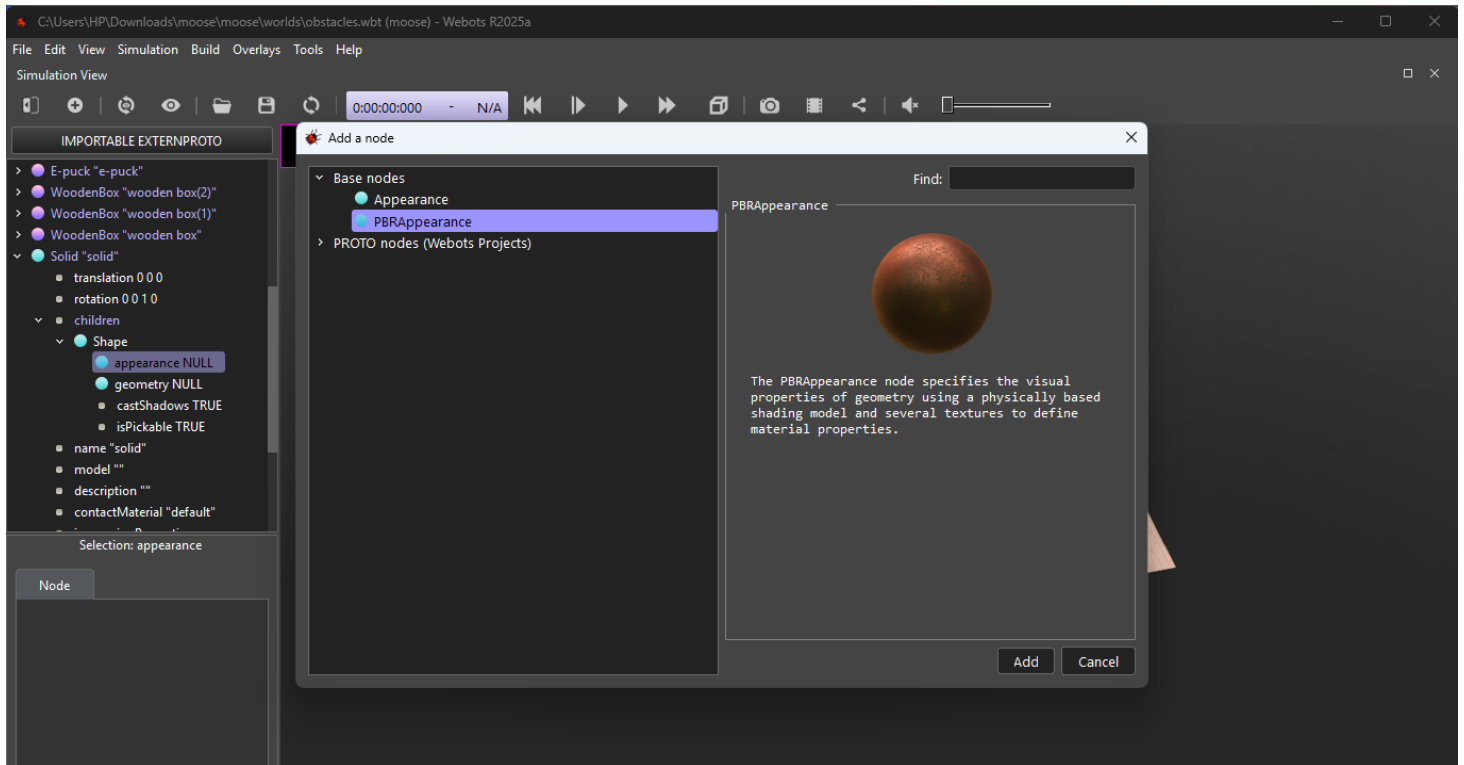
## En la sección **Base nodes**, selecciona **Solid**



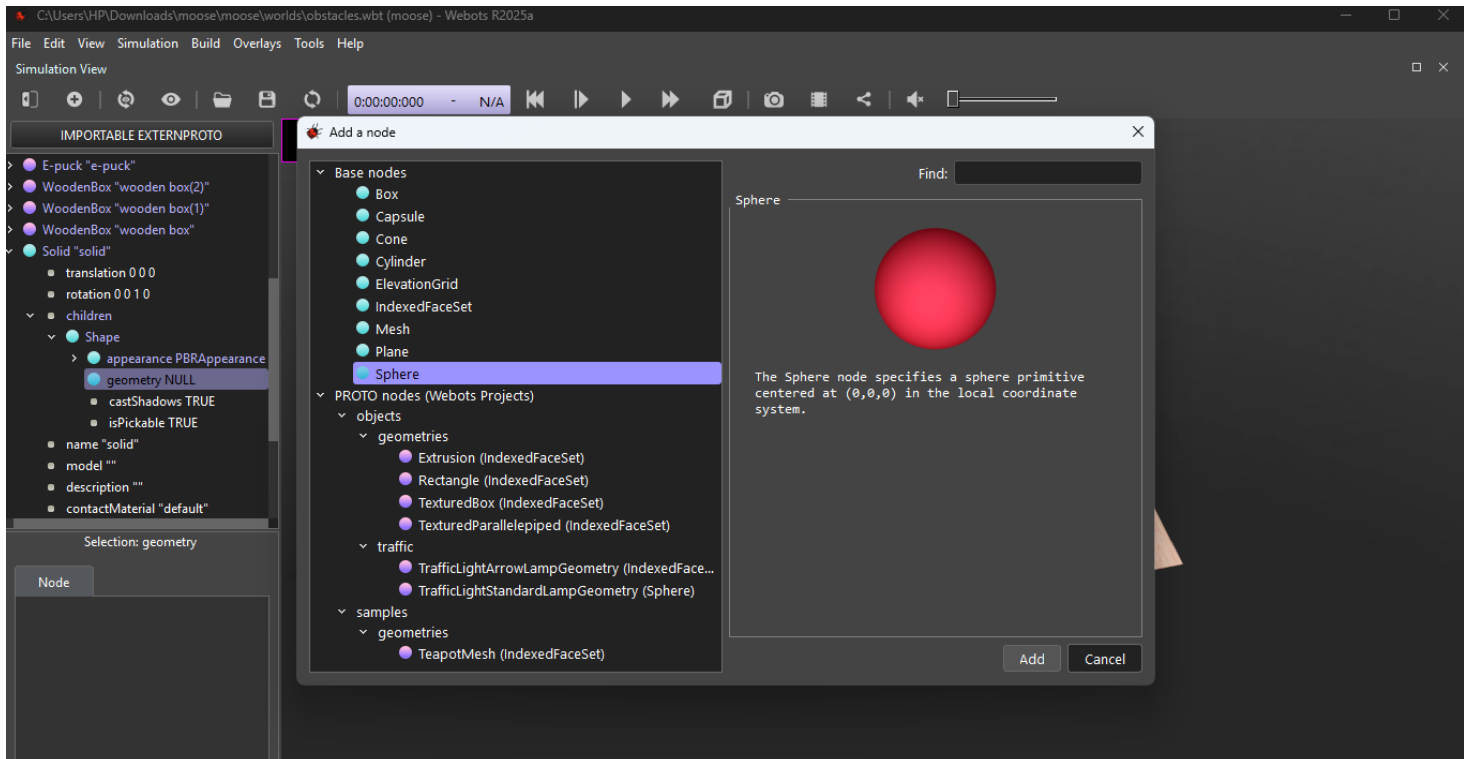
Expande el nodo Solid que acabas de crear y selecciona su campo children  
Haz clic en Add y añade un nodo Shape



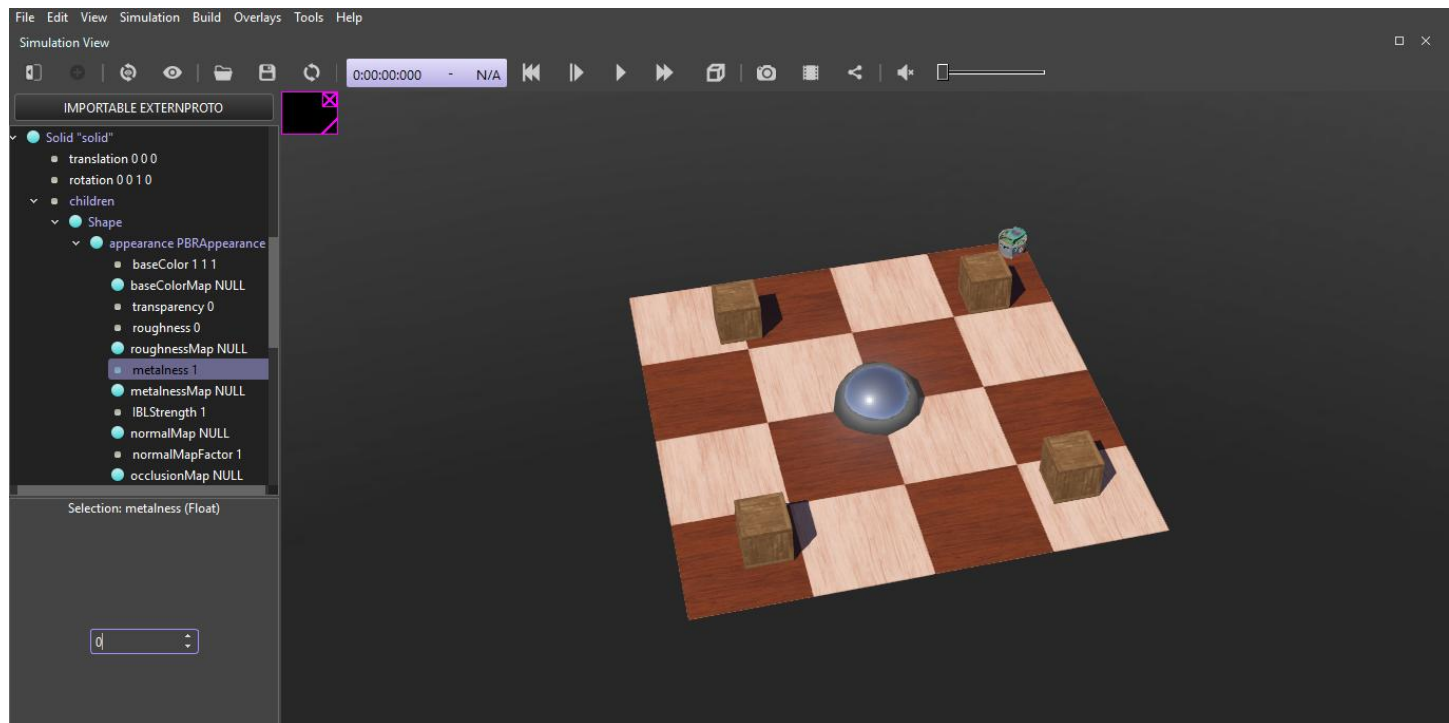
Selecciona el campo appearance del Shape y añade un nodo PBRAppearance



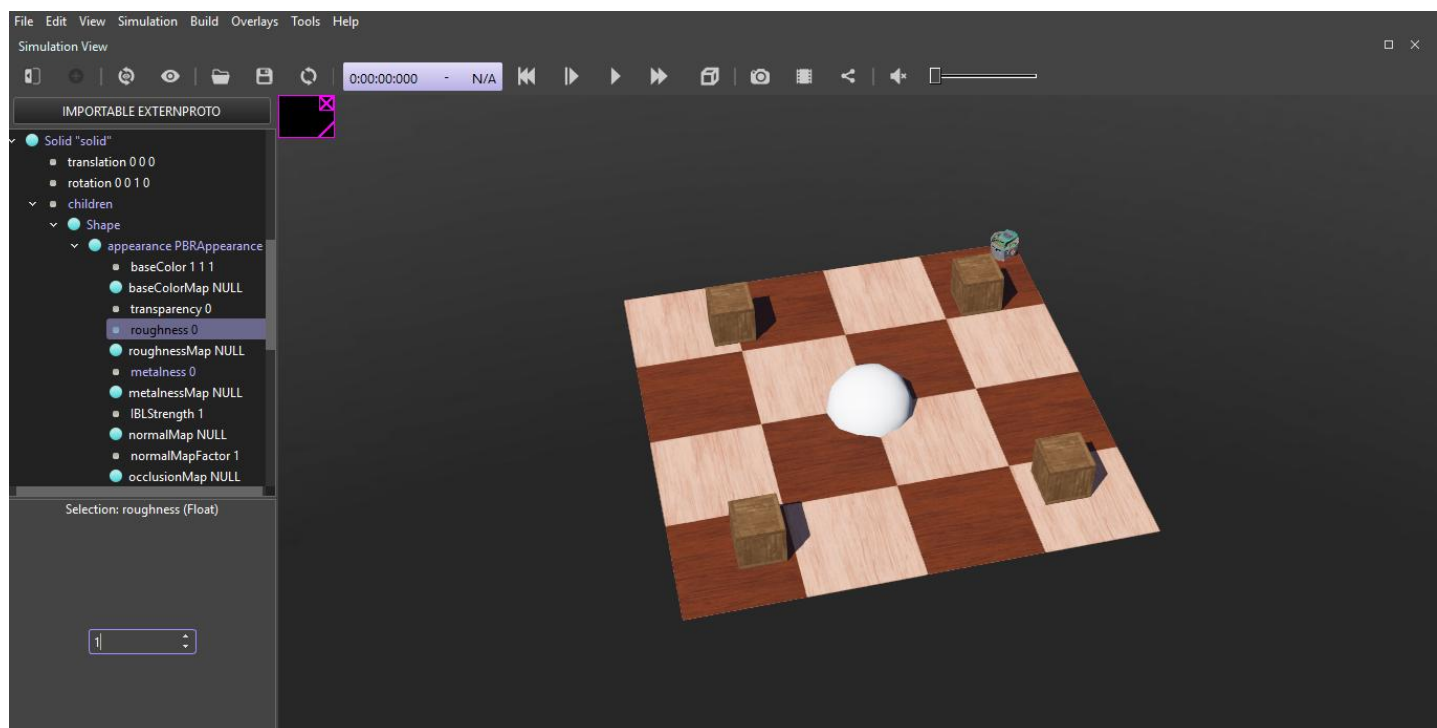
Ahora configura lo siguiente:  
Añade un nodo Sphere al campo geometry del Shape



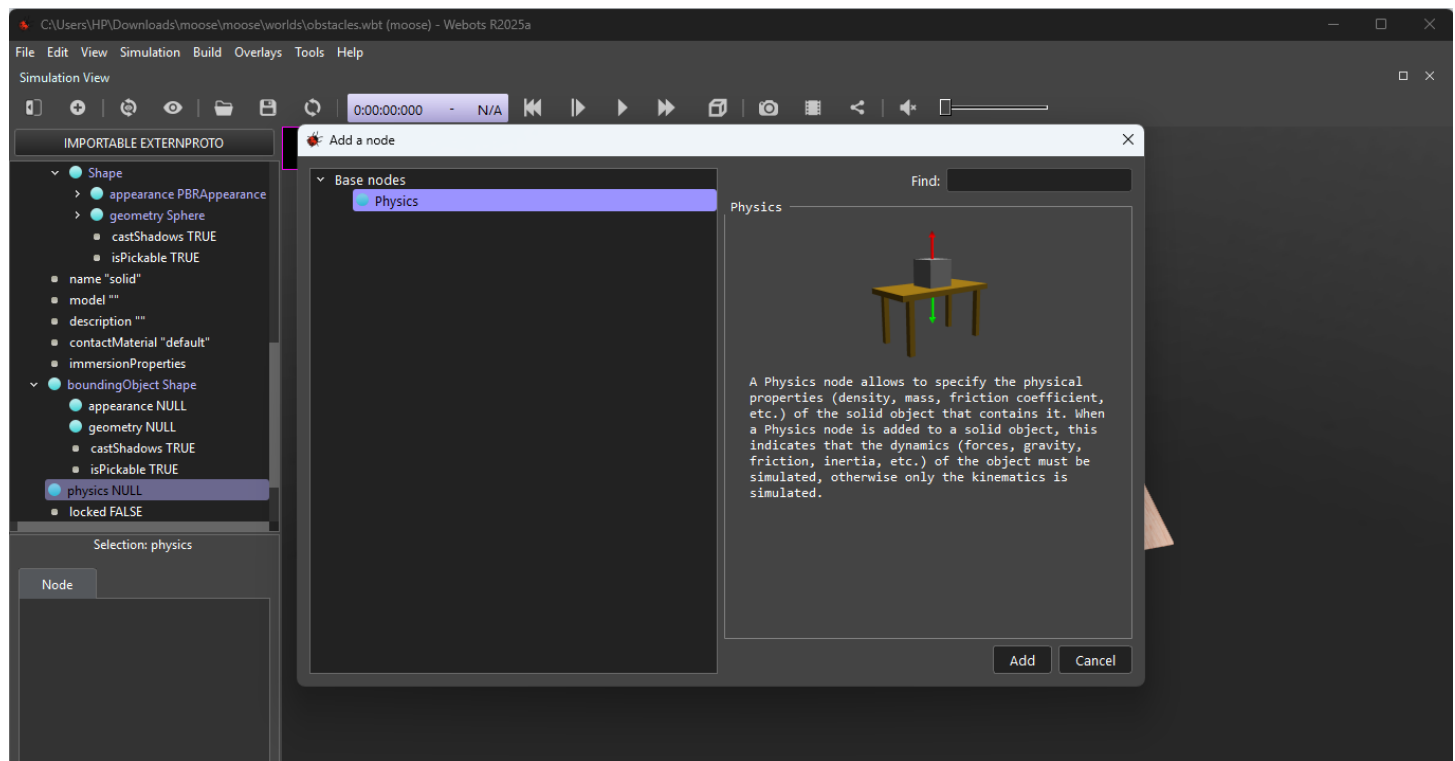
Expande el PBRAppearance y cambia:  
metalness = 0



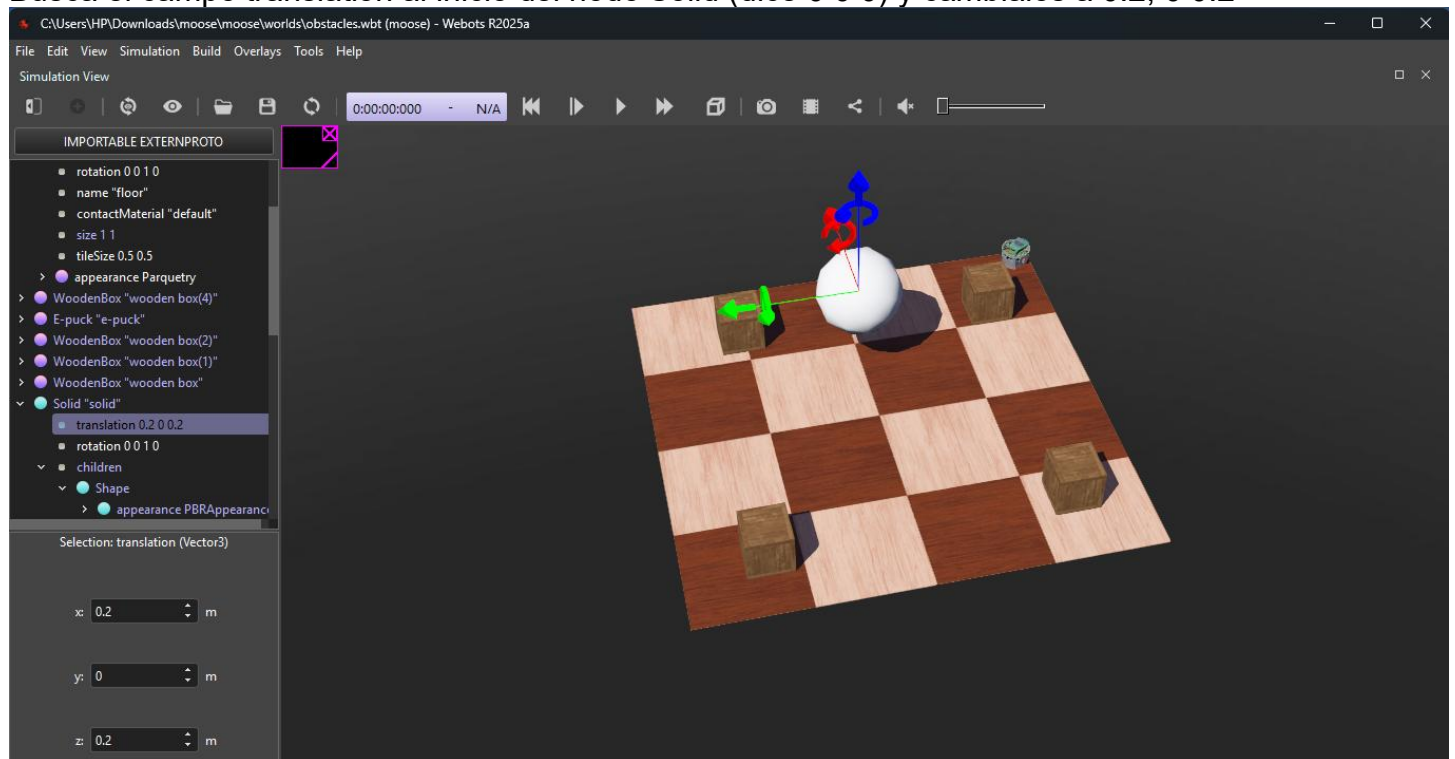
roughness =1



Añade otro nodo Sphere al campo boundingObject del Solid  
Añade un nodo Physics al campo physics del Solid



Cambiar la posición de la bola  
Busca el campo translation al inicio del nodo Solid (dice 0 0 0) y cambialos a 0.2, 0 0.2



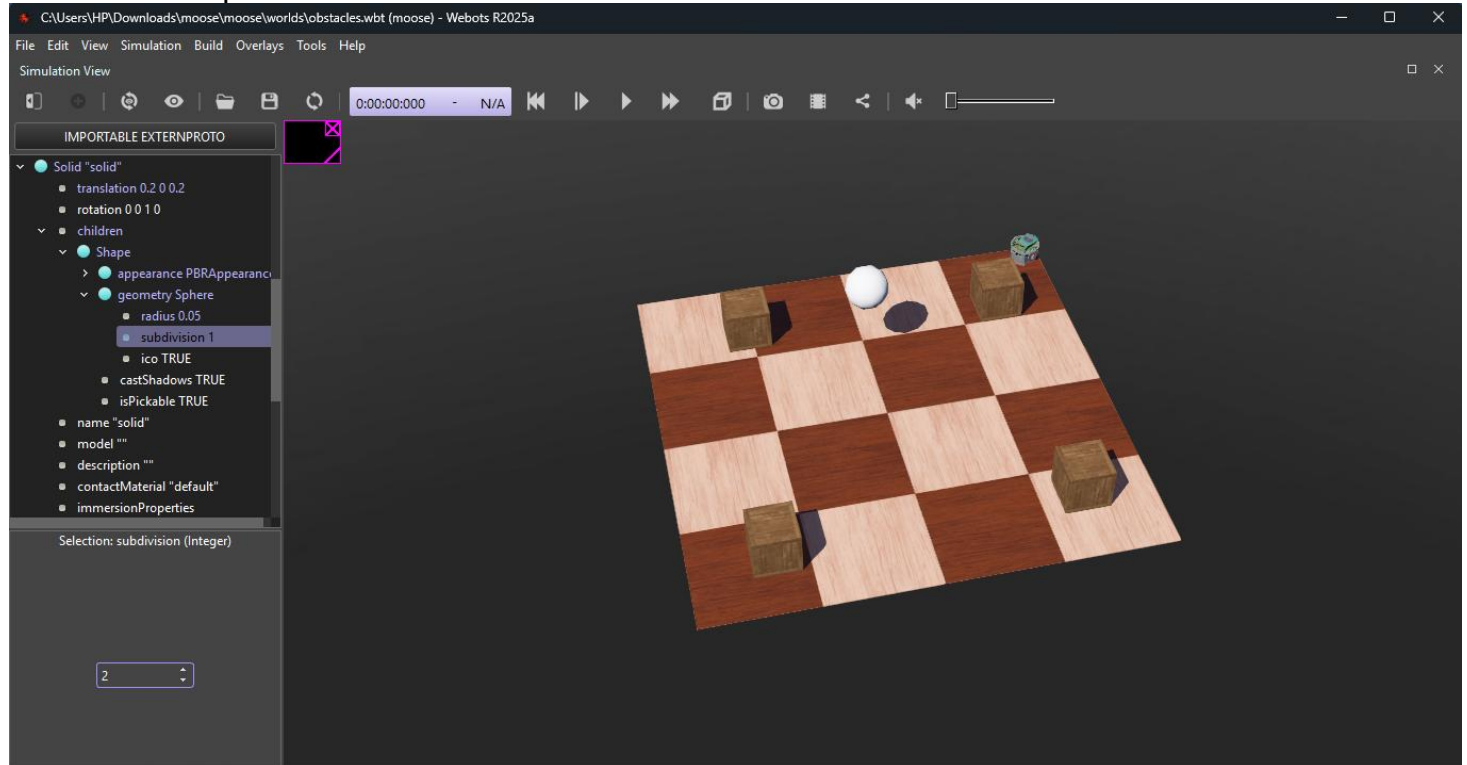
## PRACTICA5

Reducir el tamaño de la bola

Expande el nodo geometry Sphere (doble clic sobre él)

Cambia el campo radius a: 0.05

Cambia el campo subdivision a: 2

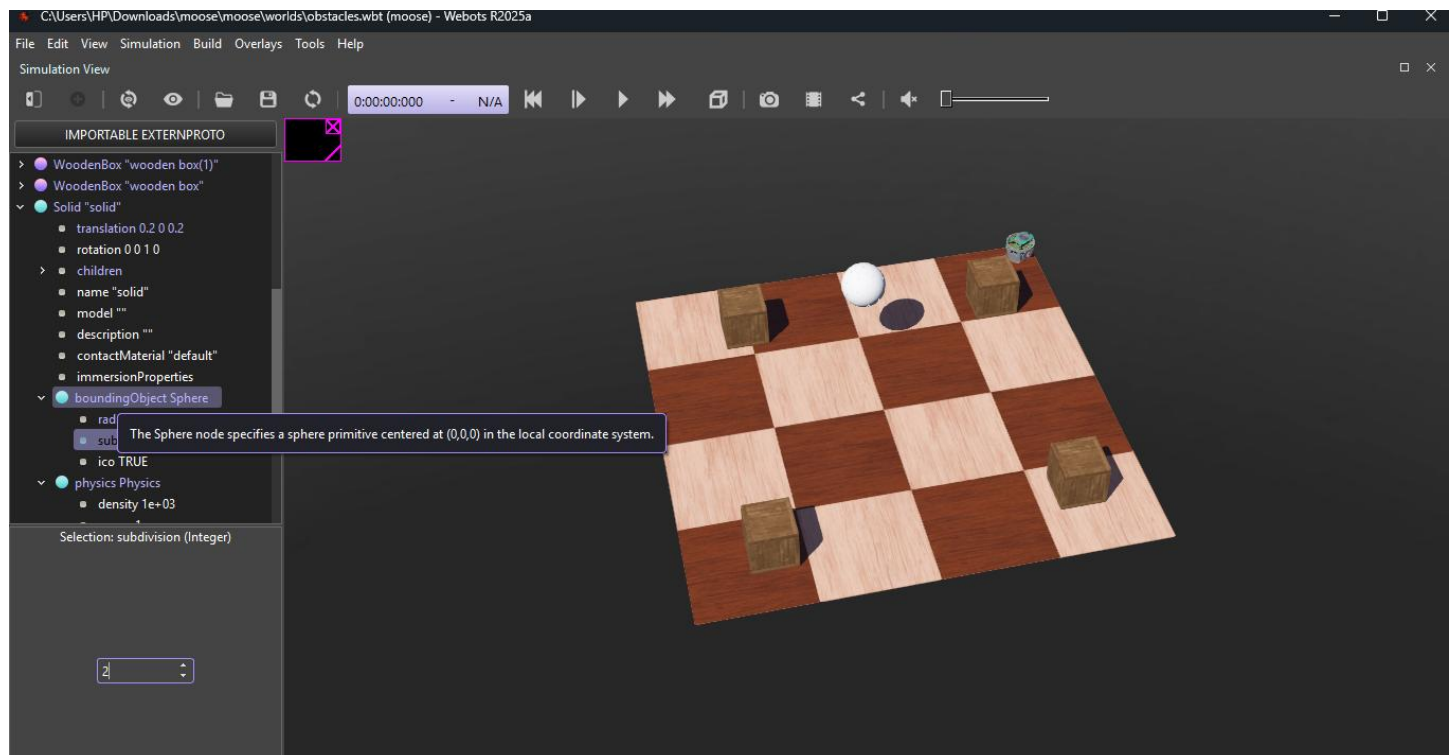


Lo mismo con el Sphere que está en boundingObject:

Expande el Sphere del boundingObject

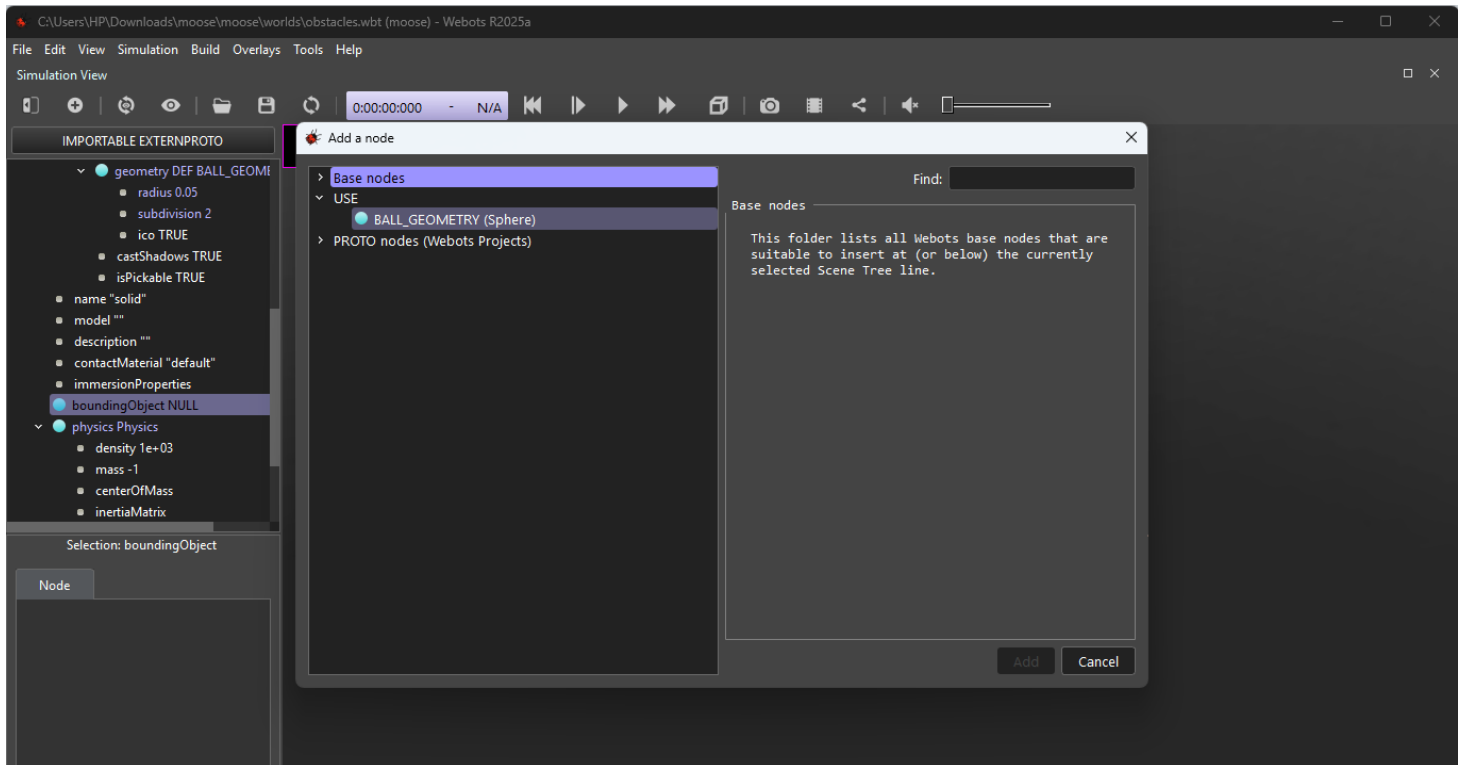
Cambia radius a: 0.05

Cambia subdivision a: 2

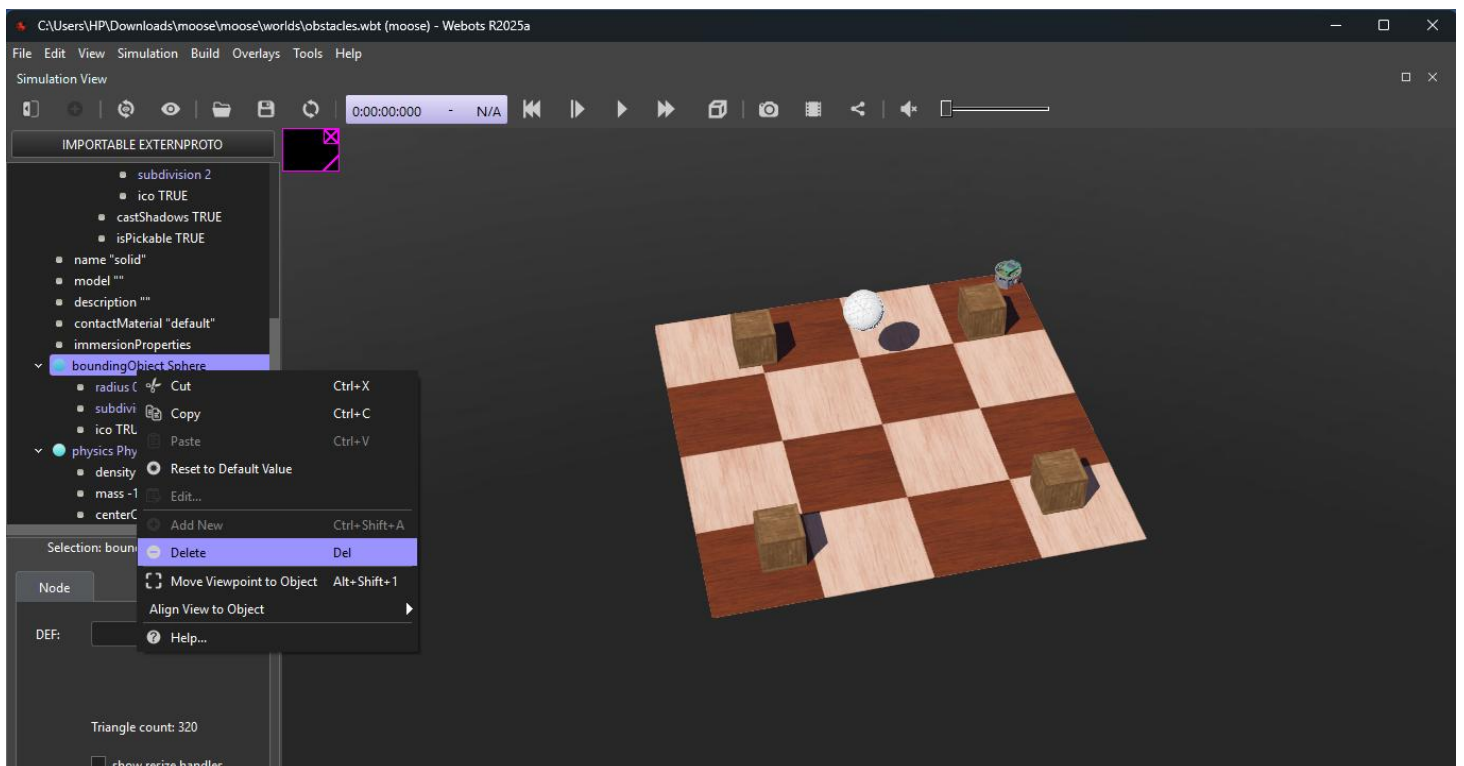


## PRACTICA6

Con boundingObject seleccionado, haz clic en Add  
Selecciona: USE > BALL\_GEOMETRY

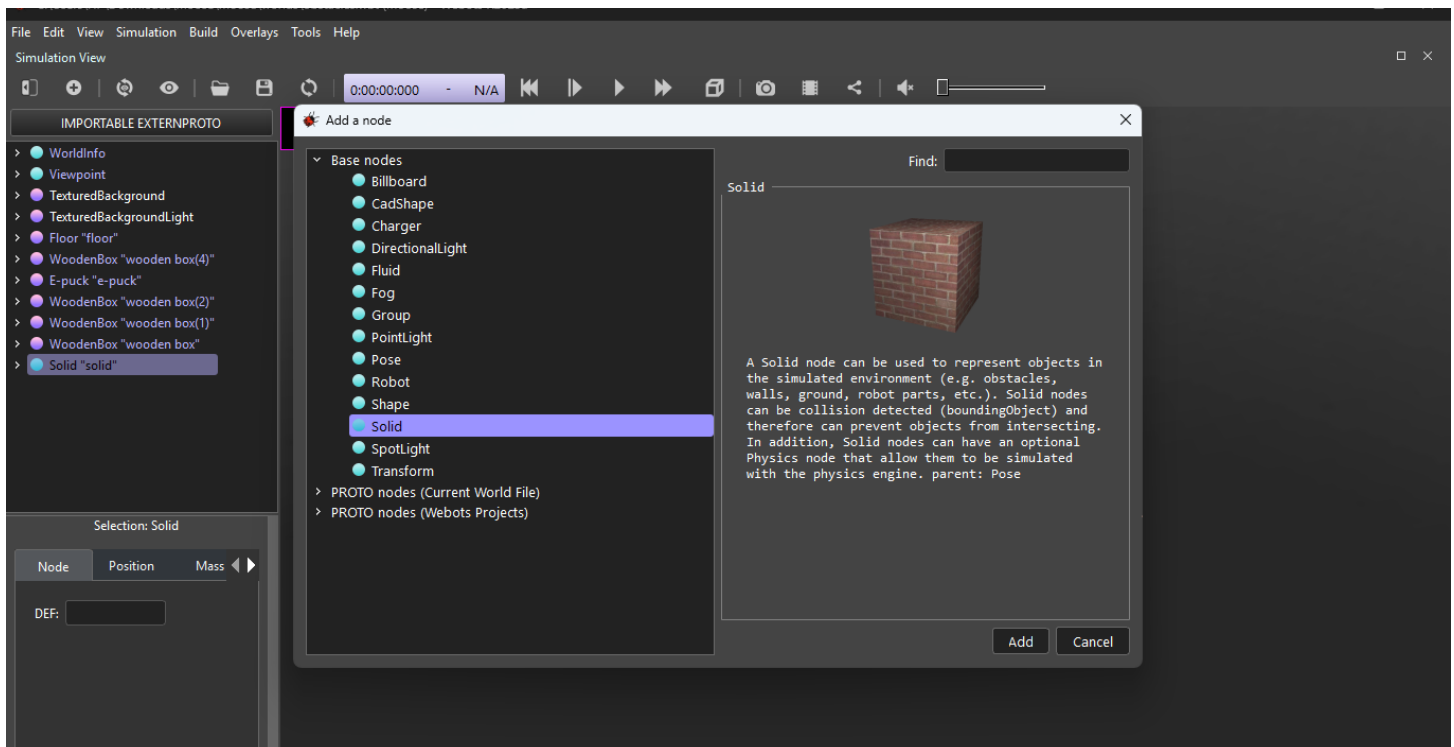
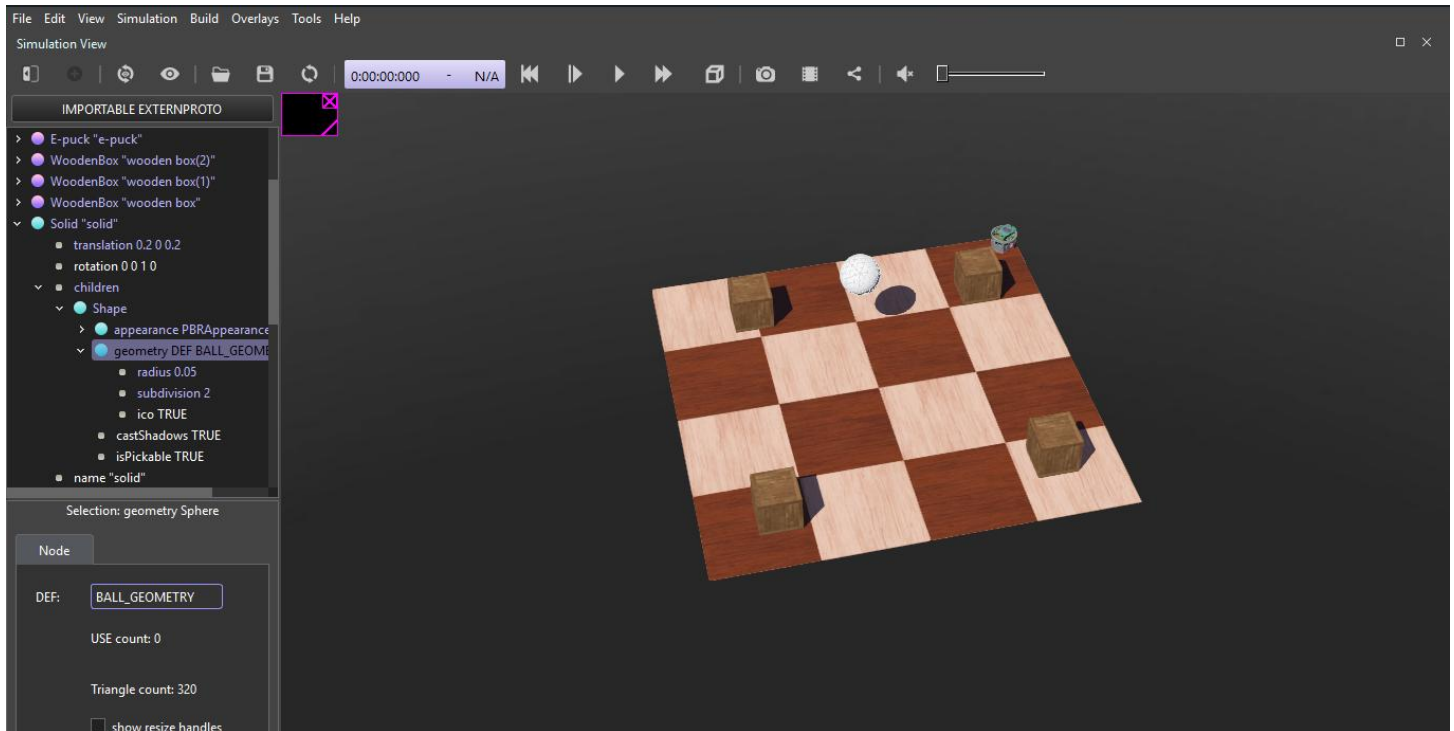


Eliminar el Sphere que está en boundingObject:  
Clic derecho sobre boundingObject Sphere > Delete

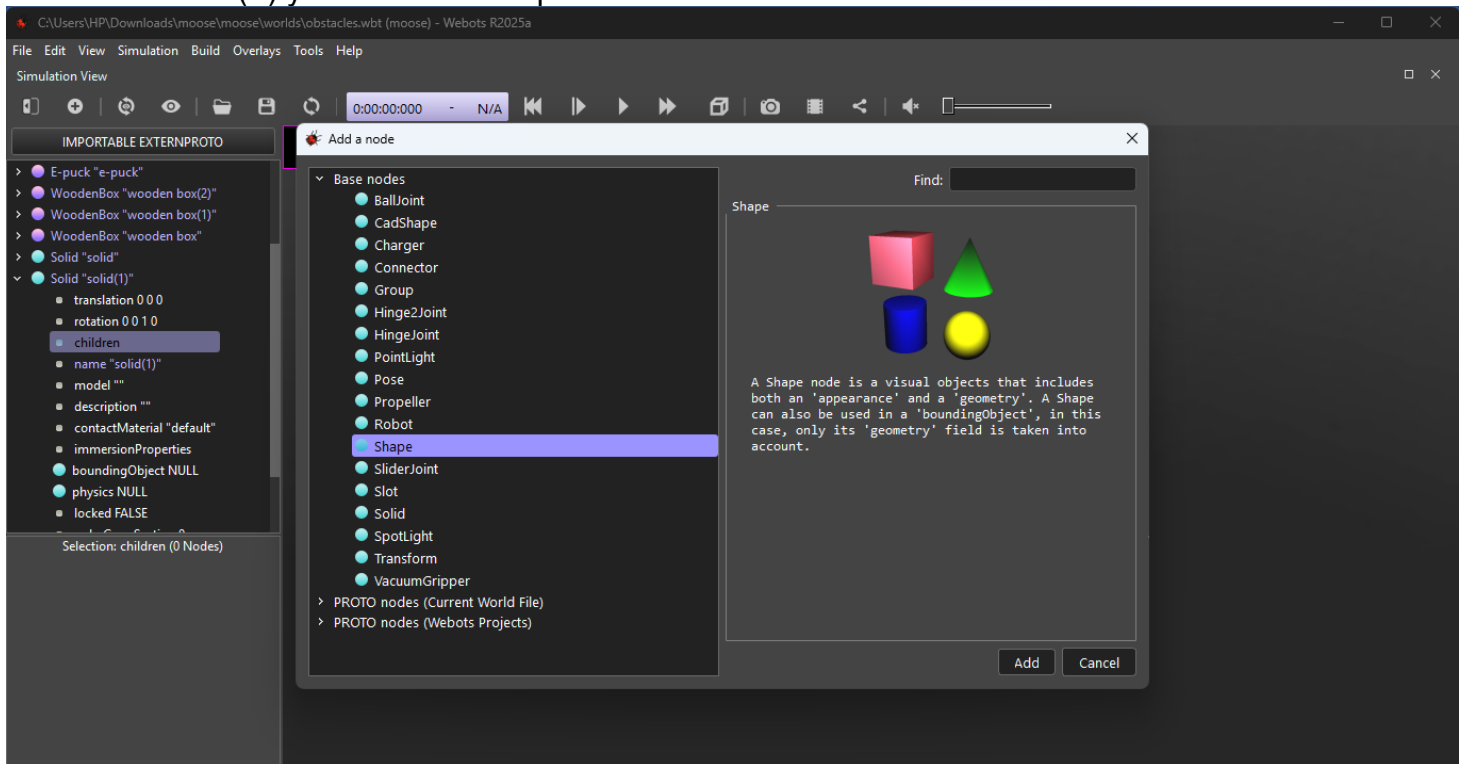


## PRACTICA7

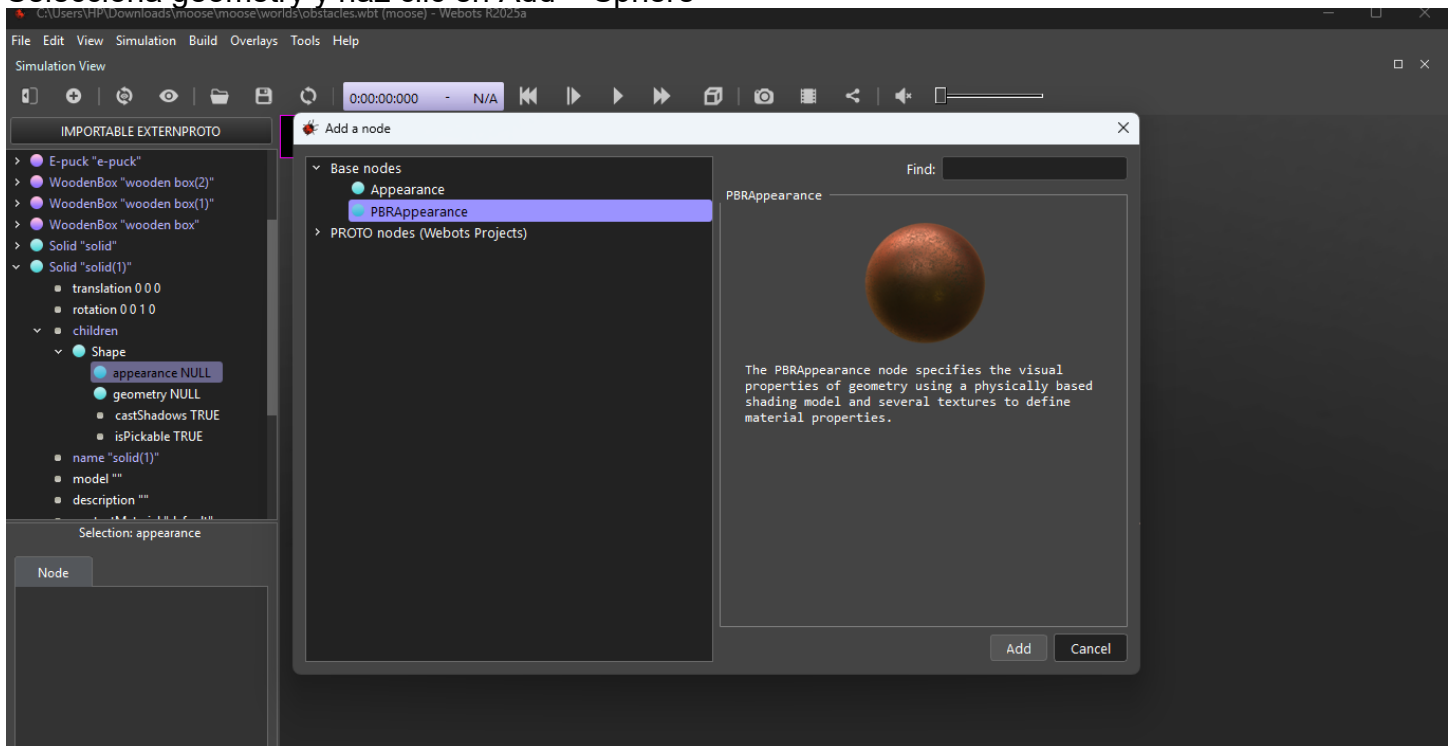
Crear una segunda bola con los mismos parámetros pero usando el nodo [Shape](#) (en lugar del nodo [Sphere](#) directamente) para el mecanismo DEF-USE.



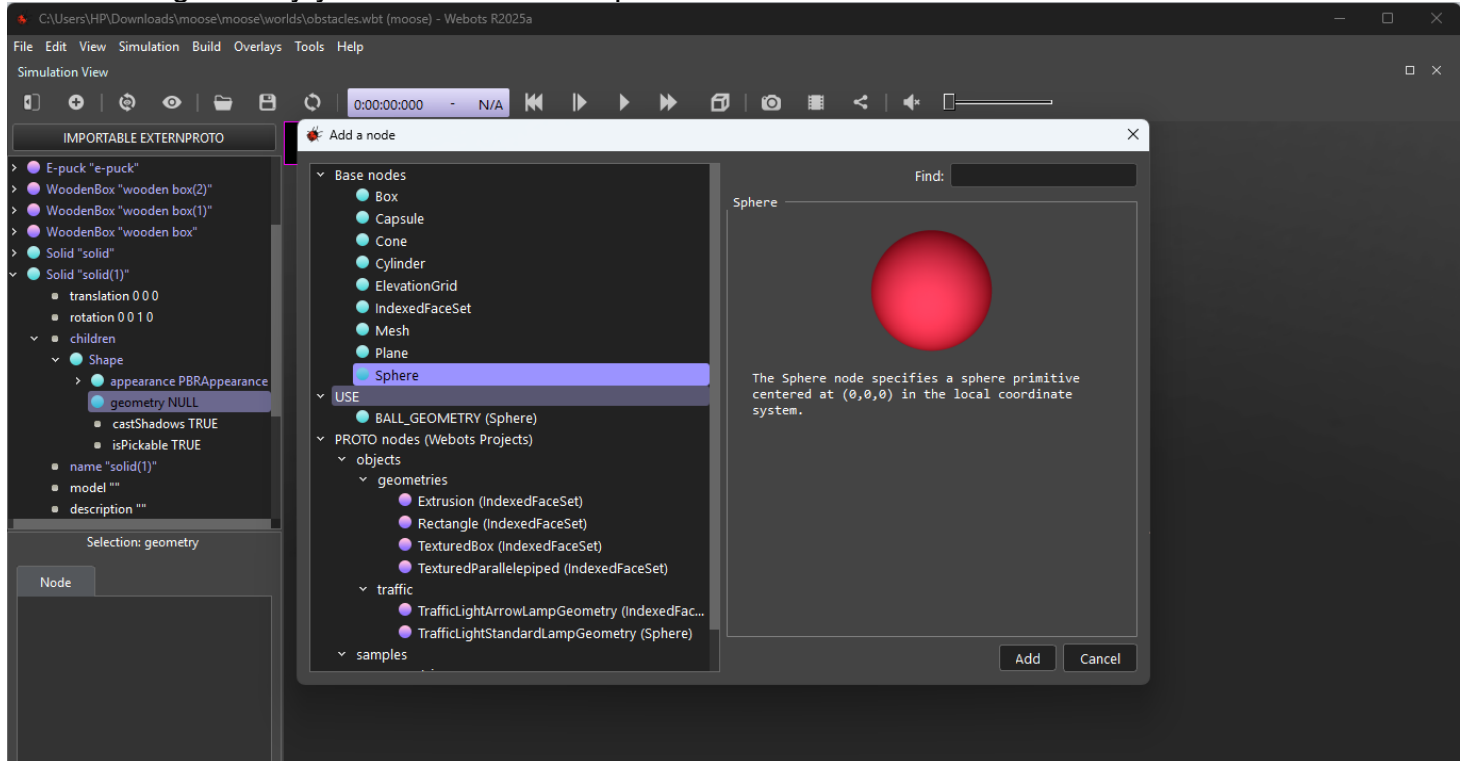
Selecciona el campo children  
Haz clic en Add (+) y selecciona: Shape



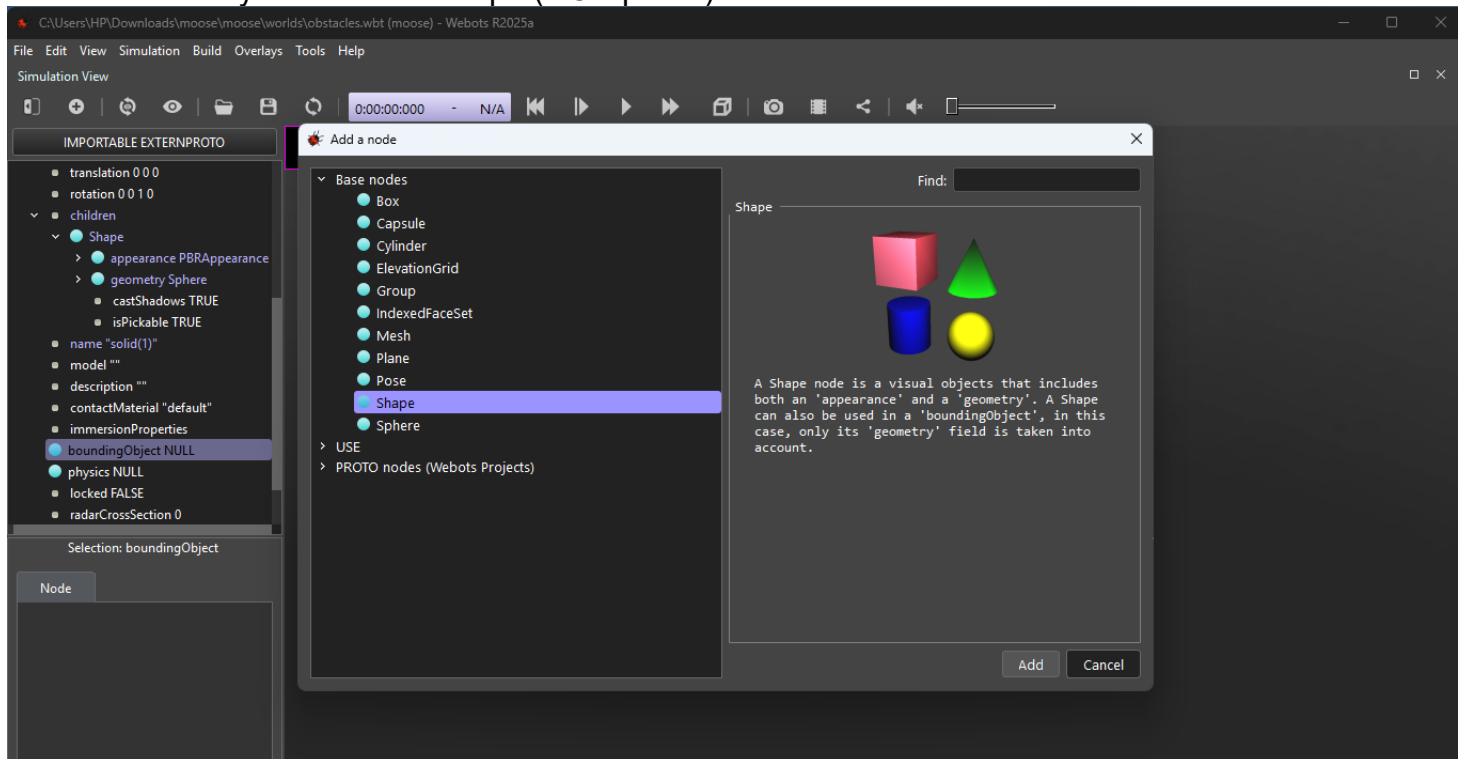
Expande ese Shape y selecciona appearance  
Haz clic en Add y selecciona: PBRAppearance  
Selecciona geometry y haz clic en Add > Sphere



Selecciona geometry y haz clic en Add > Sphere



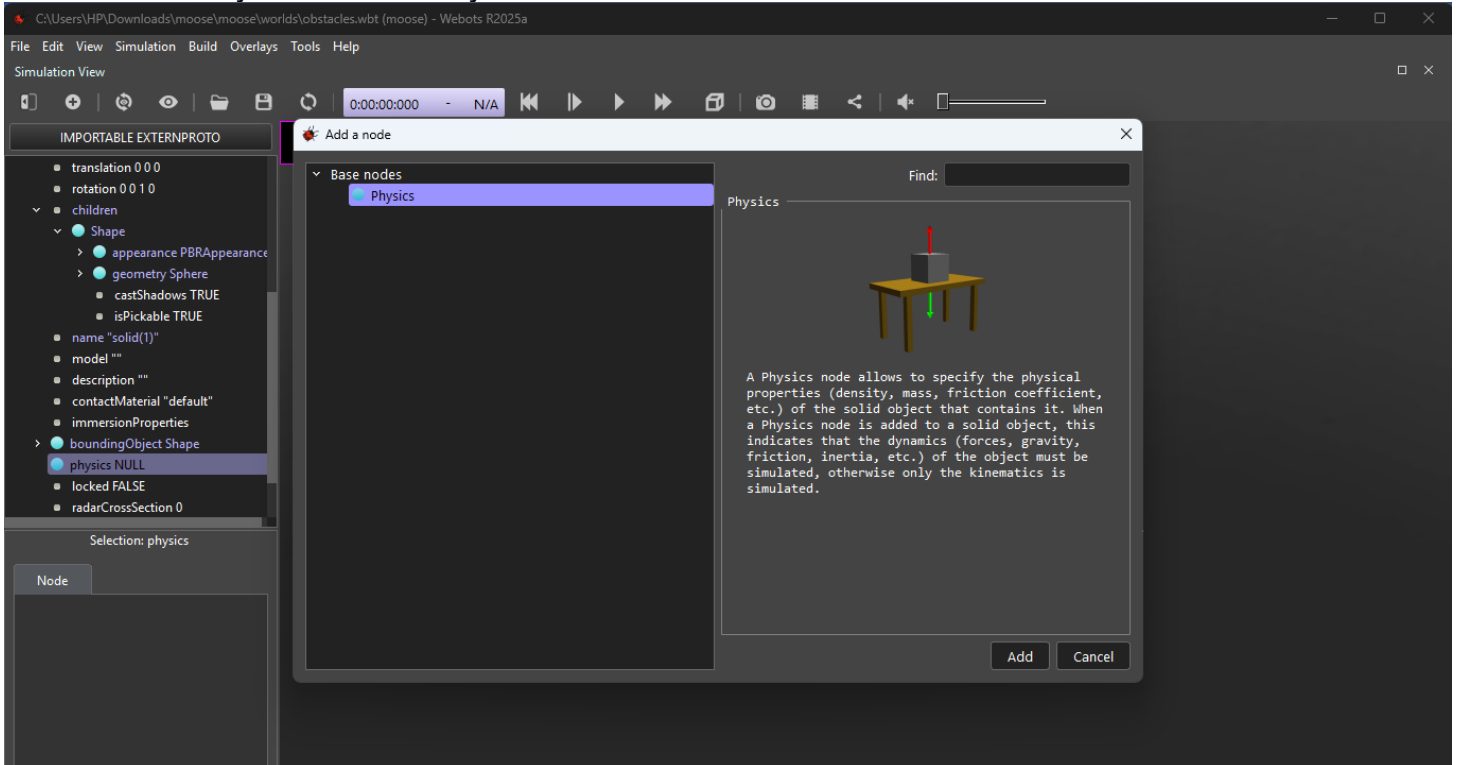
Busca el campo boundingObject (debería decir NULL)  
Haz clic en Add y selecciona: Shape (NO Sphere)



## Añadir Physics

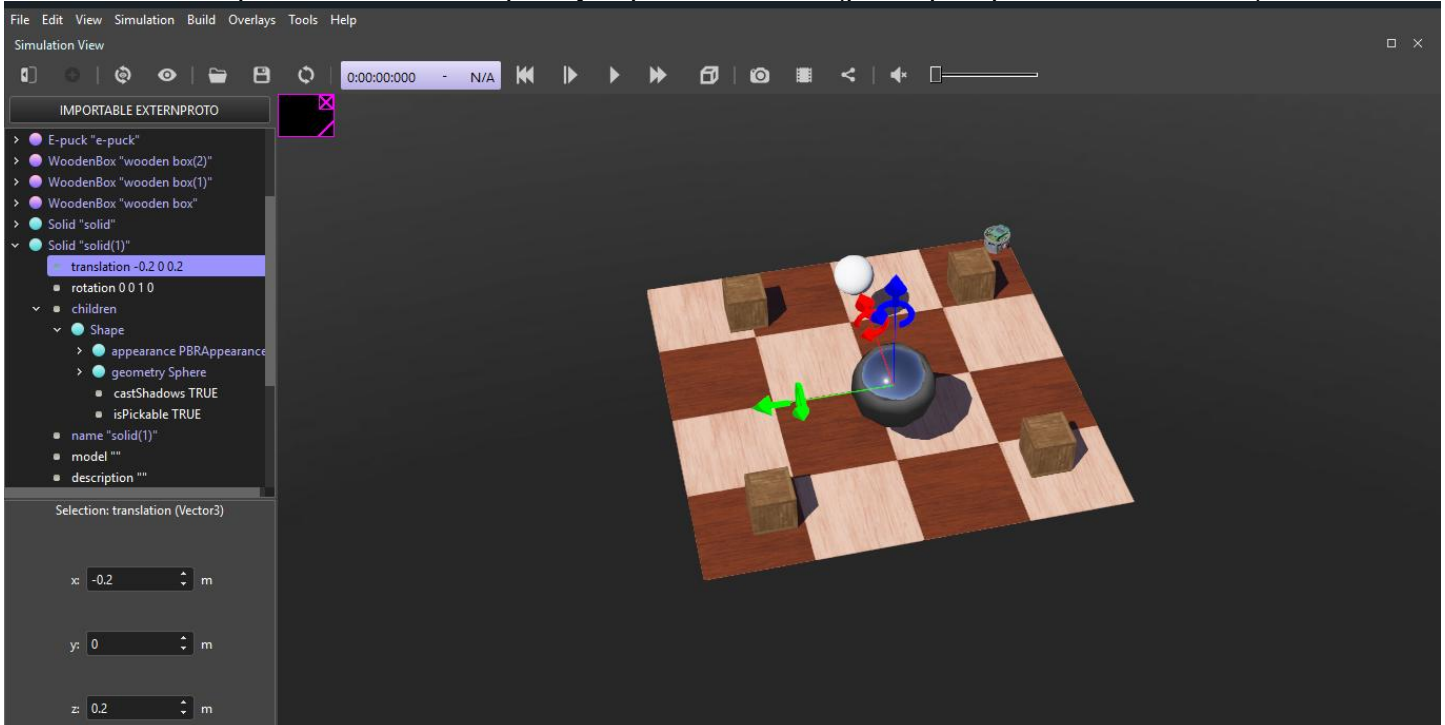
Busca el campo physics (debería decir NULL)

Haz clic en Add y selecciona: Physics



Busca el campo translation del segundo Solid

Cámbialo a una posición diferente, por ejemplo: -0.2 0 0.2 (para que quede del otro lado)

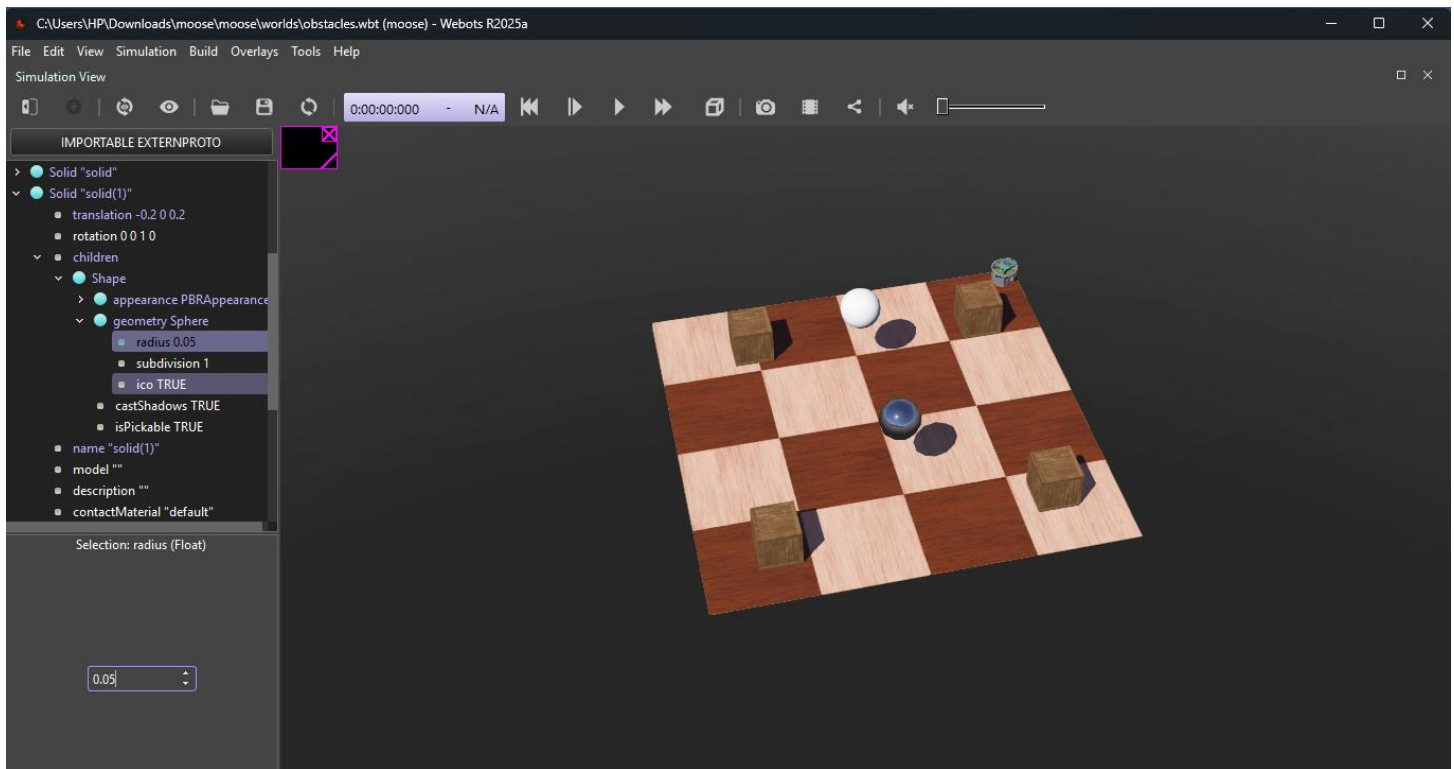


Configurar tamaños y apariencia

En ambos Sphere (el de geometry y el del boundingObject):

Cambia radius a: 0.05

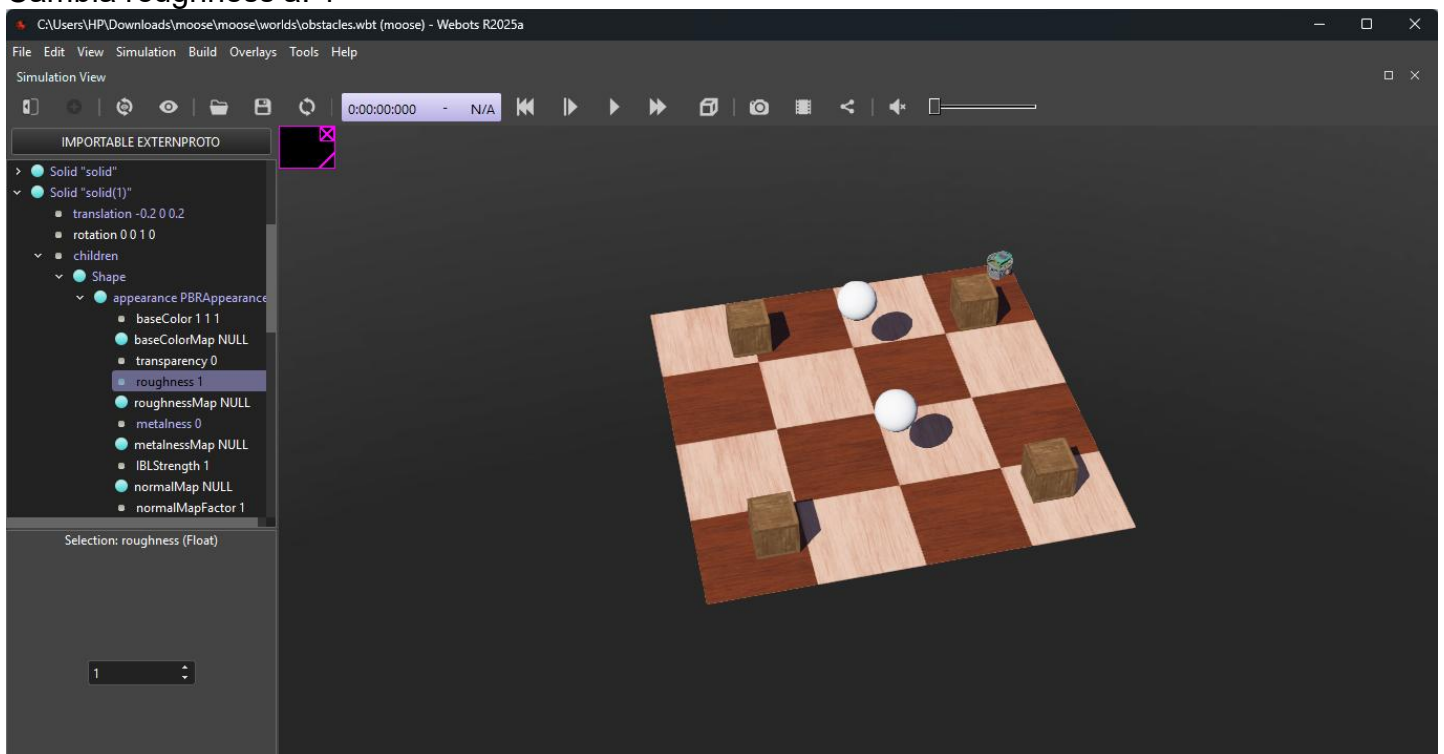
Cambia subdivision a: 2



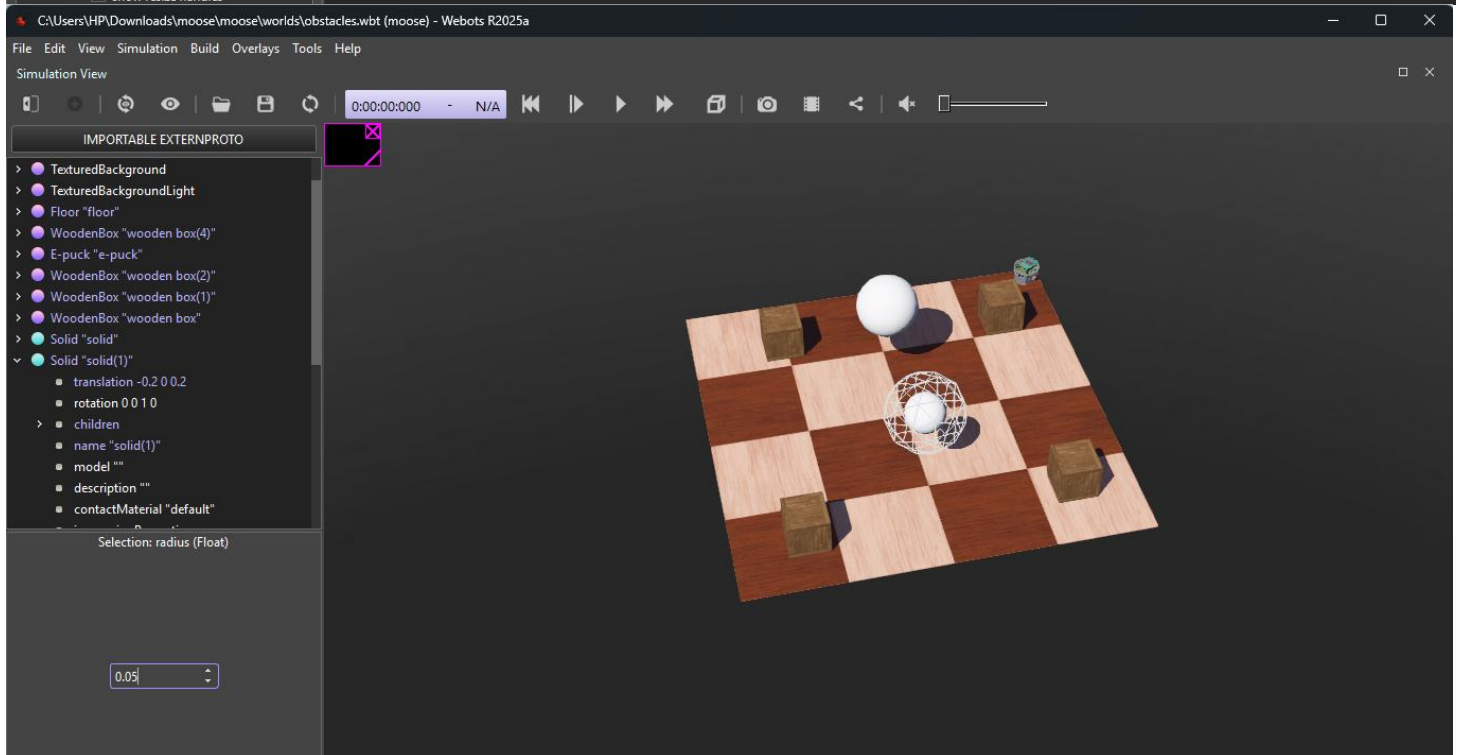
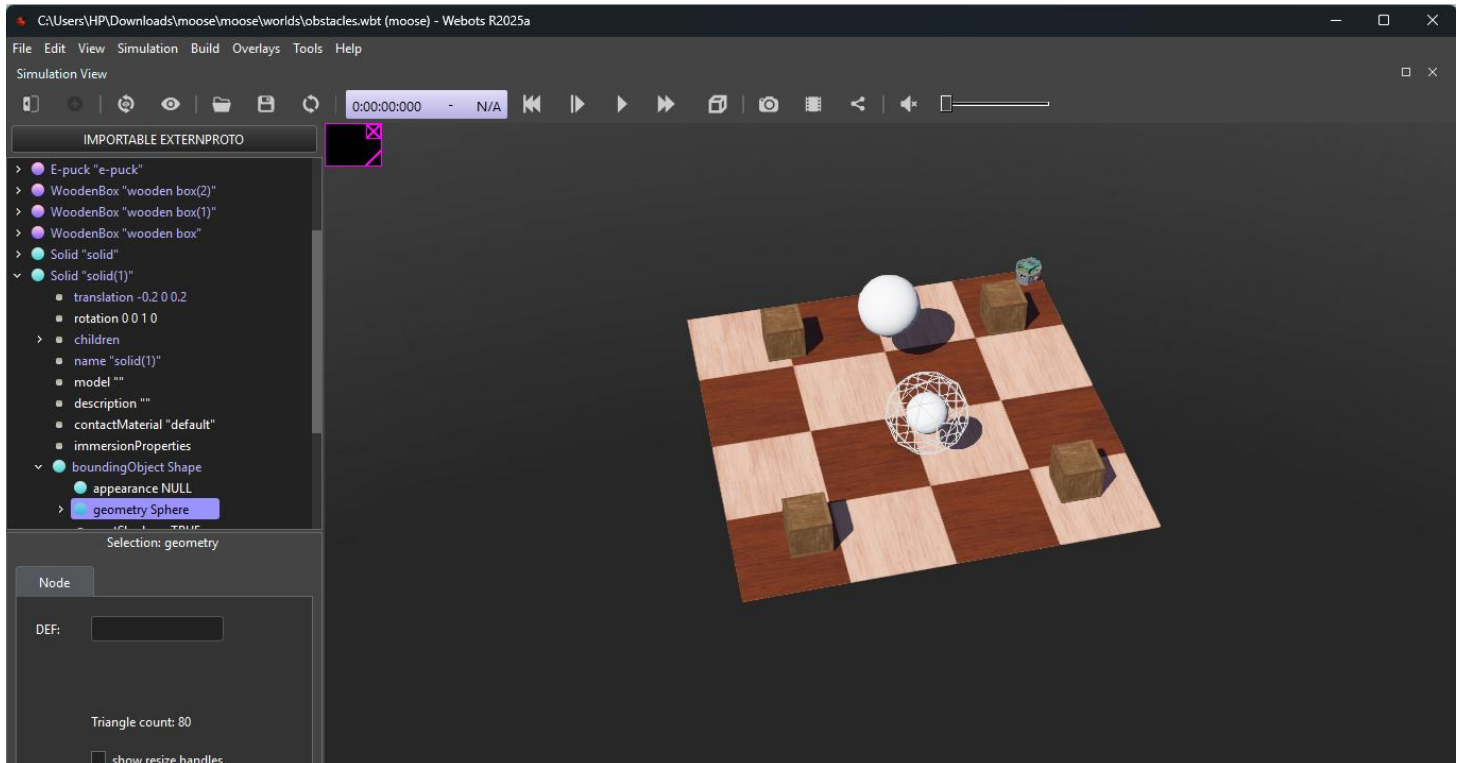
En el PBRAppearance de la segunda bola:

Cambia metalness a: 0

Cambia roughness a: 1



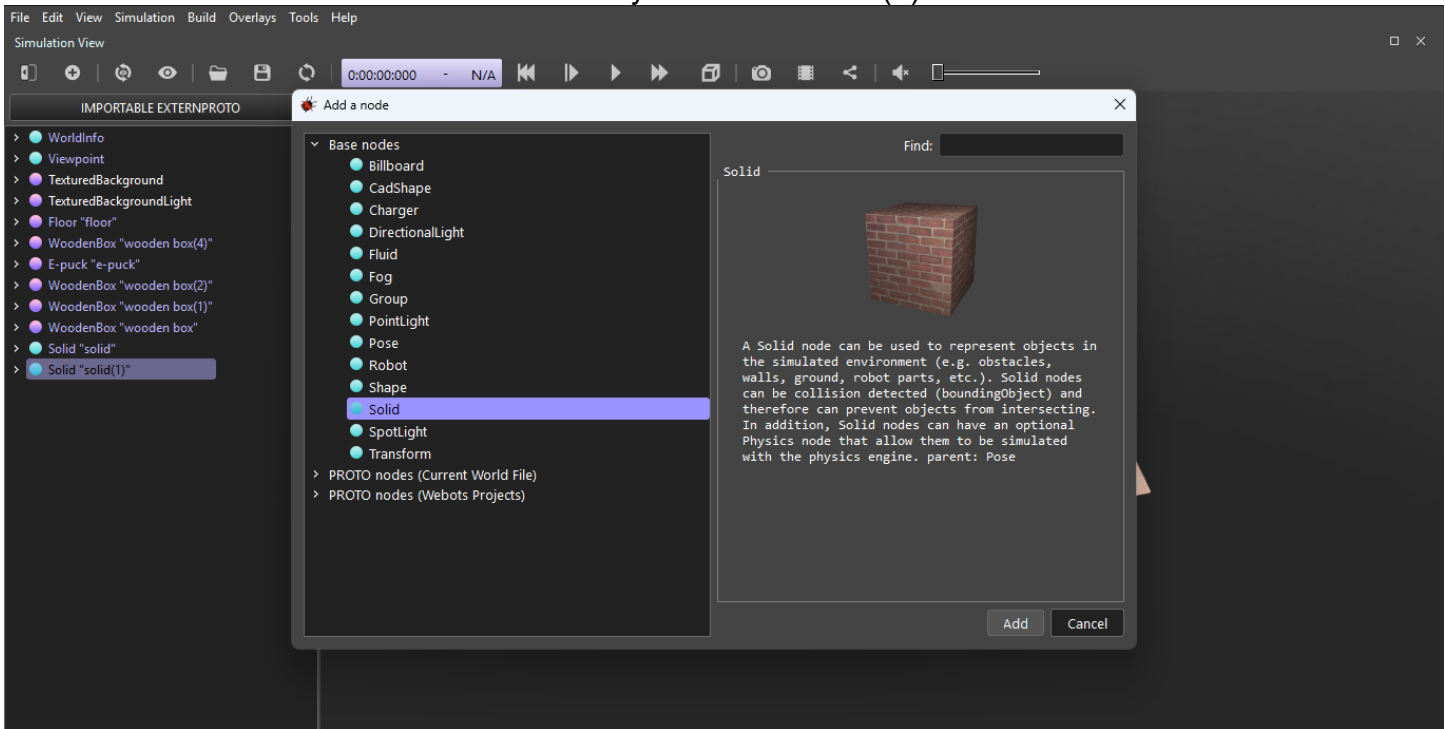
Expande ese Shape > selecciona geometry > Add > Box o Sphere  
Si usas Sphere: radius = 0.05



## PRACTICA8

Crear la primera pared (Modelo)

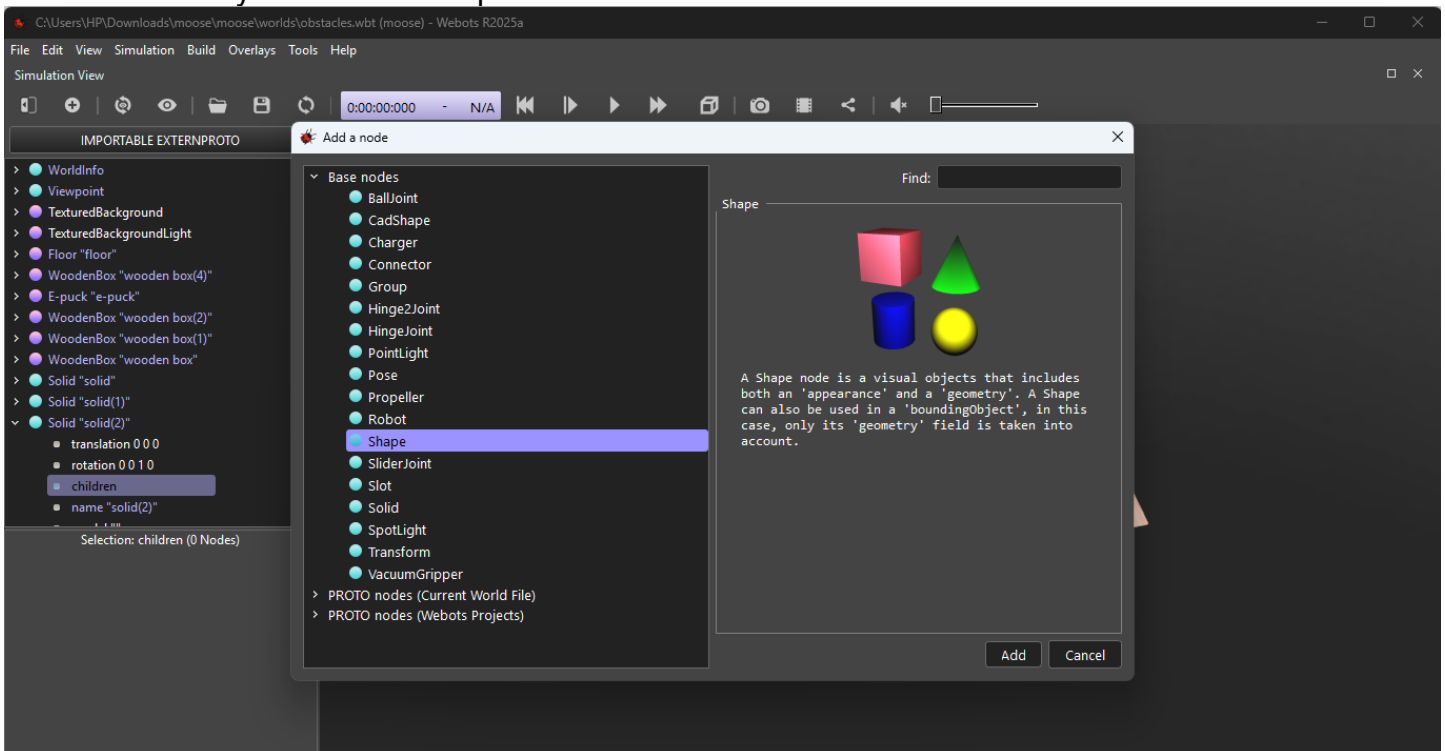
Selecciona el último nodo en el Scene Tree y haz clic en Add (+).



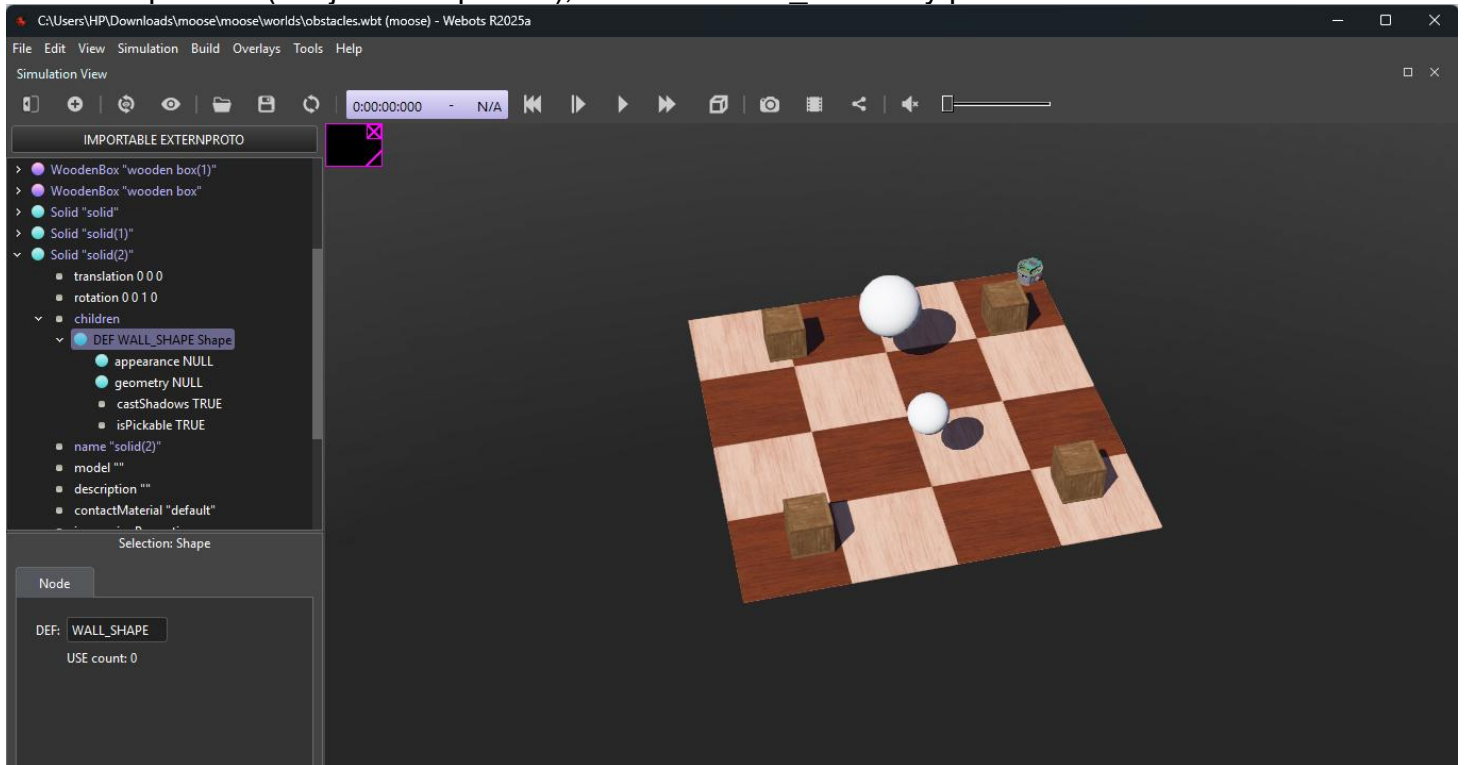
Expande ese nuevo Solid y selecciona el campo children.

Expande ese nuevo Solid y selecciona el campo children.

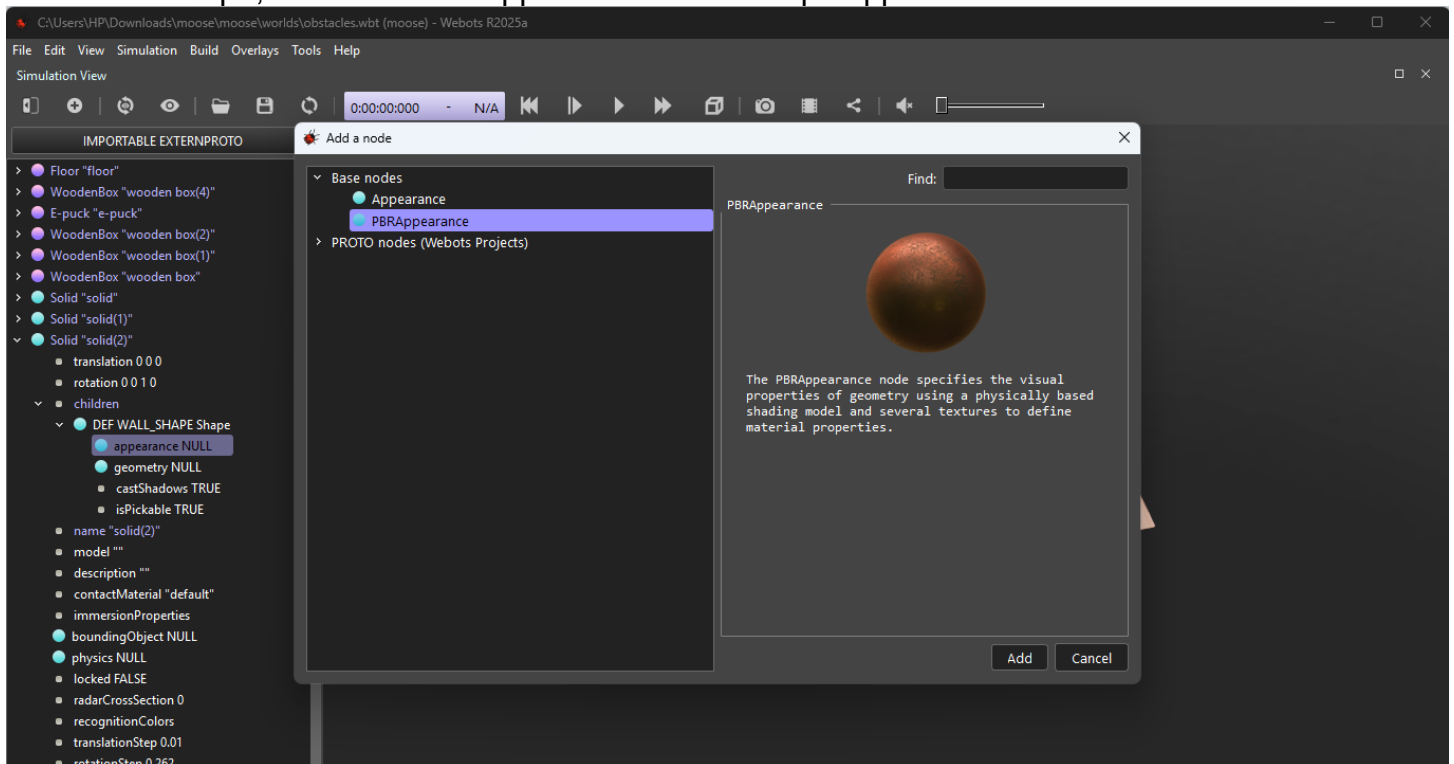
Haz clic en Add y selecciona Shape.



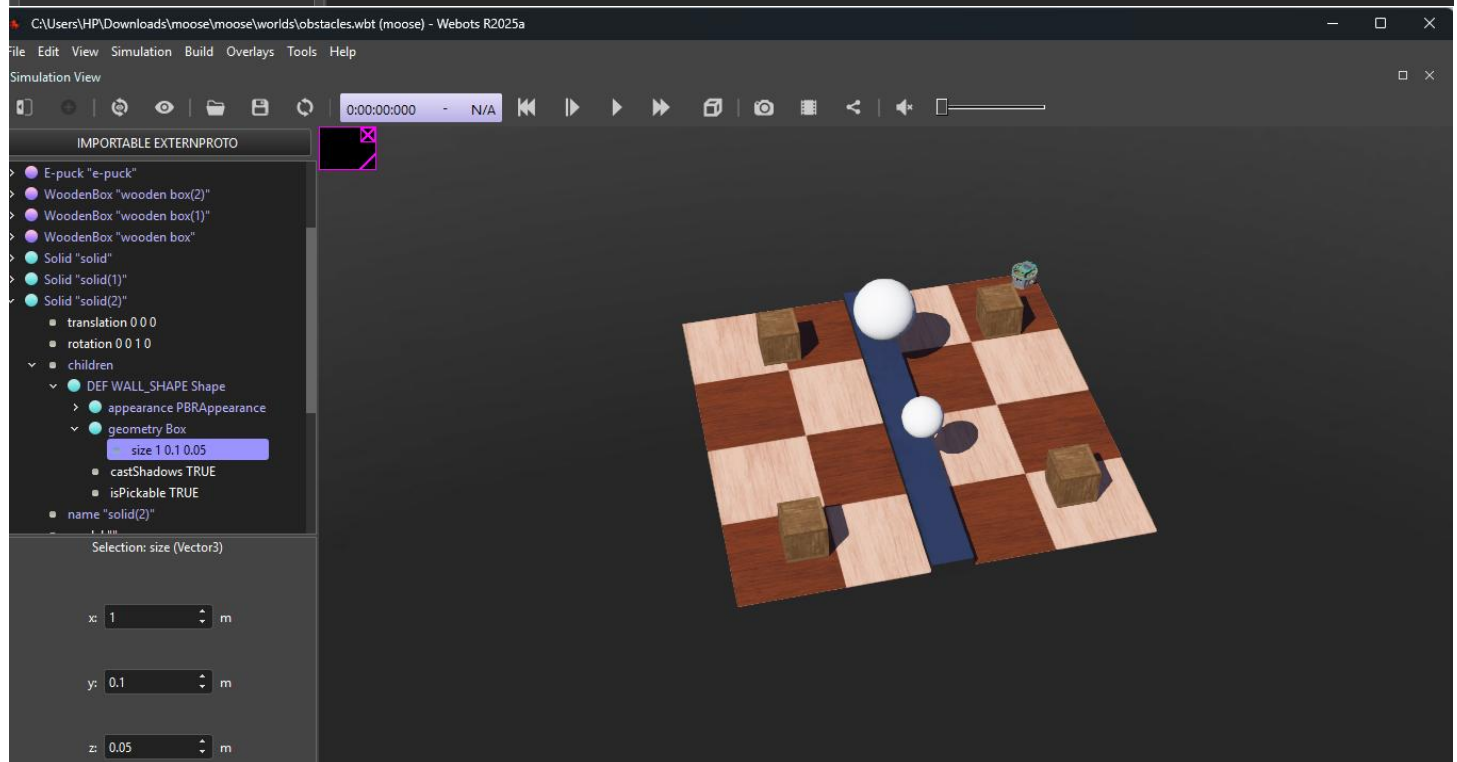
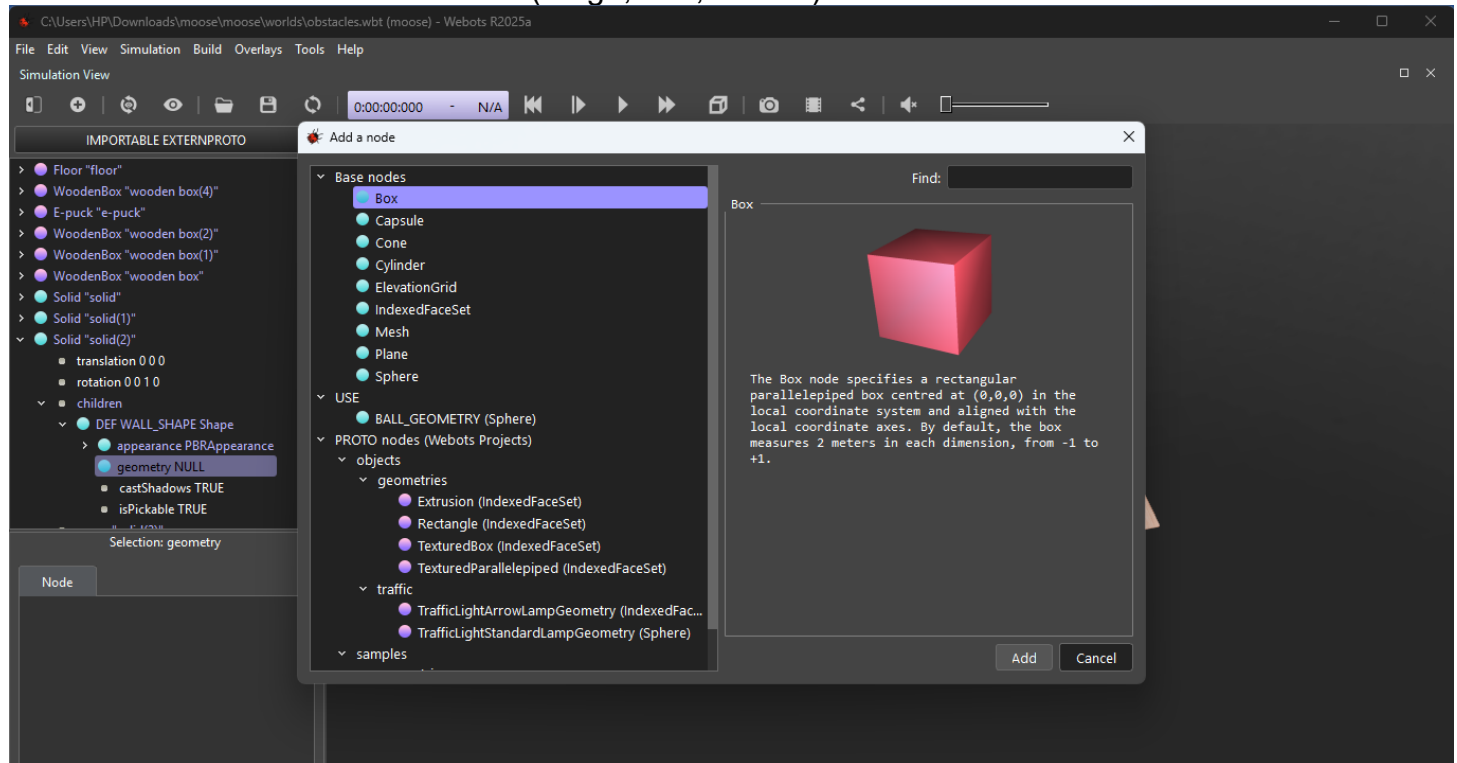
Configura el Shape (Aquí aplicamos DEF):  
Selecciona el nodo Shape que acabas de crear.  
En el campo DEF (abajo a la izquierda), escribe: WALL\_SHAPE y presiona Enter.



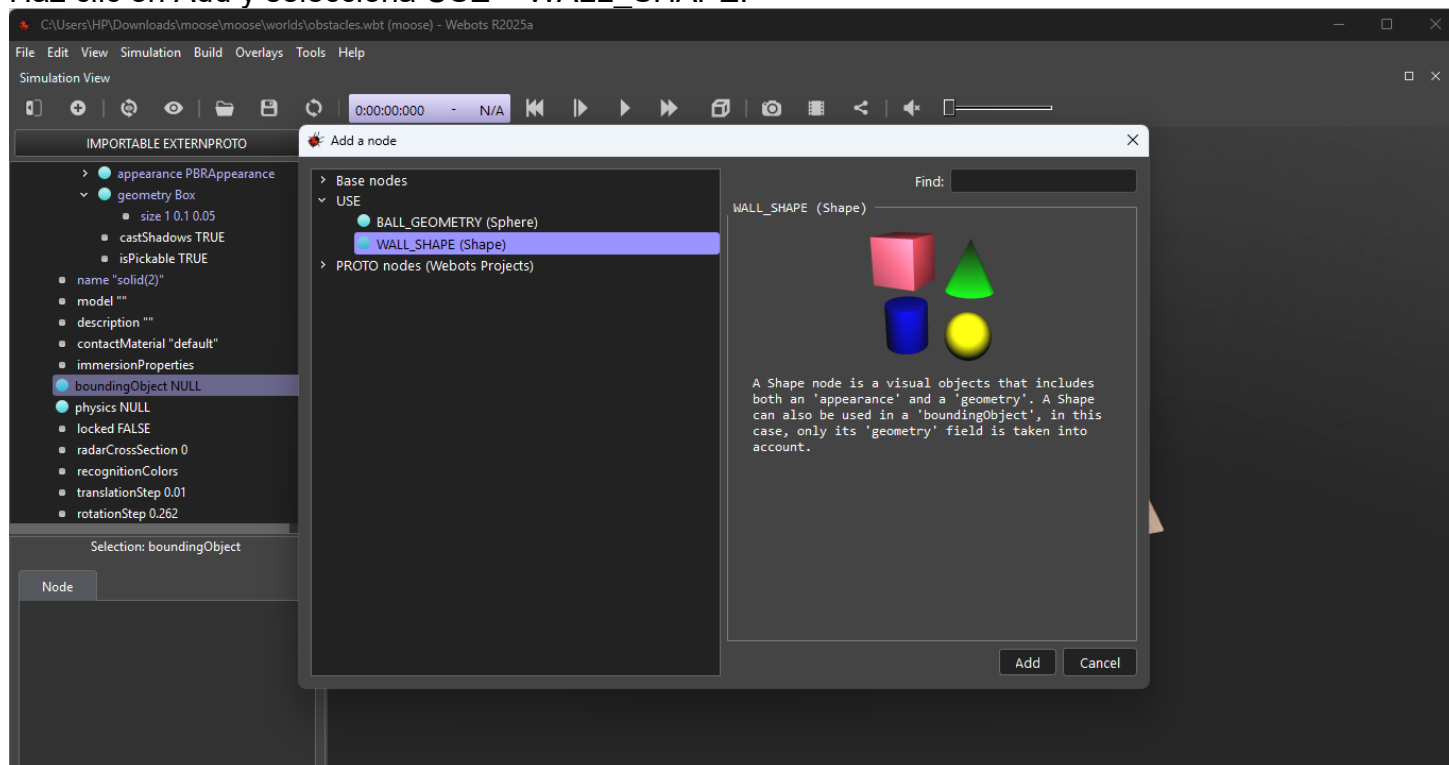
Dentro del Shape, añade un PBRAppearance en el campo appearance.



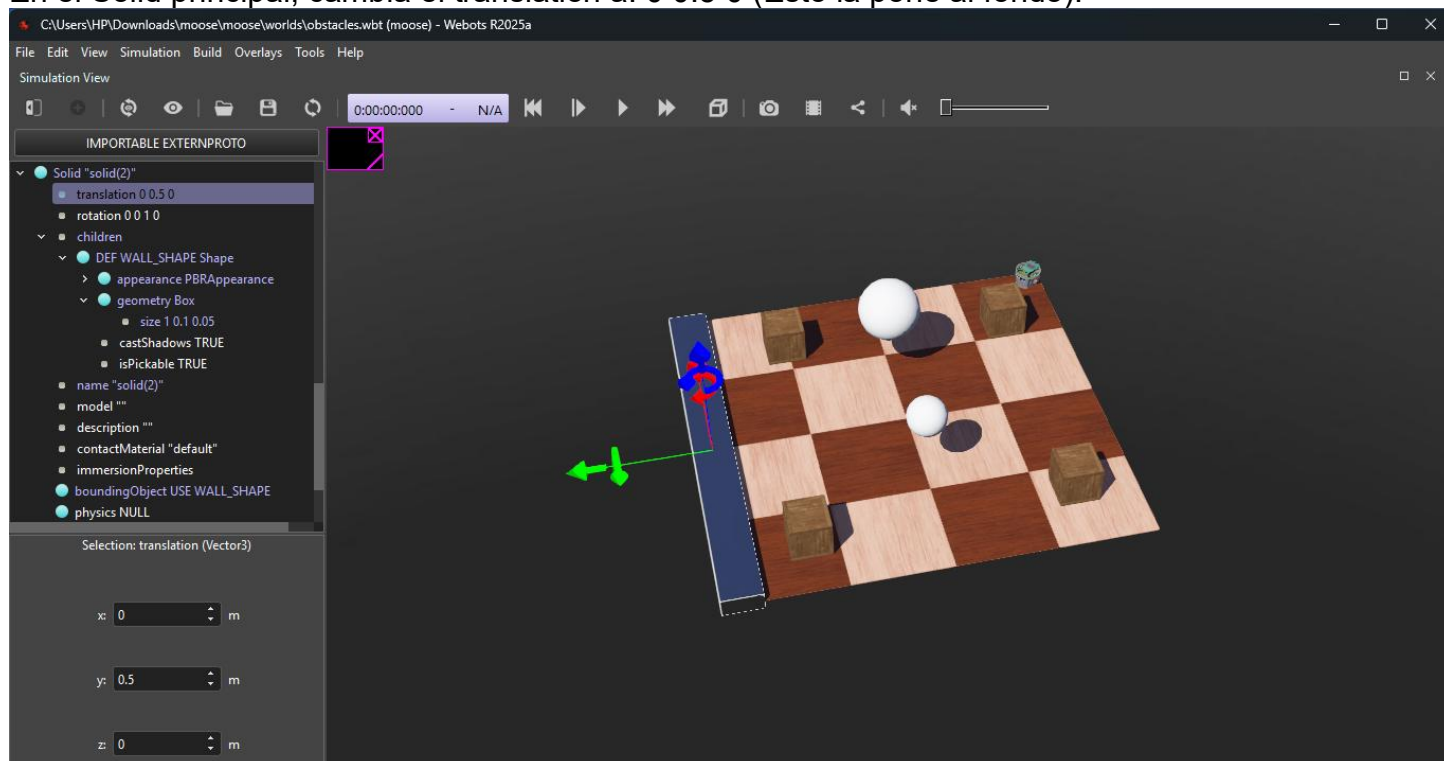
En el campo geometry, añade un Box.  
Cambia el size del Box a: 1 0.1 0.05 (Largo, Alto, Grosor).



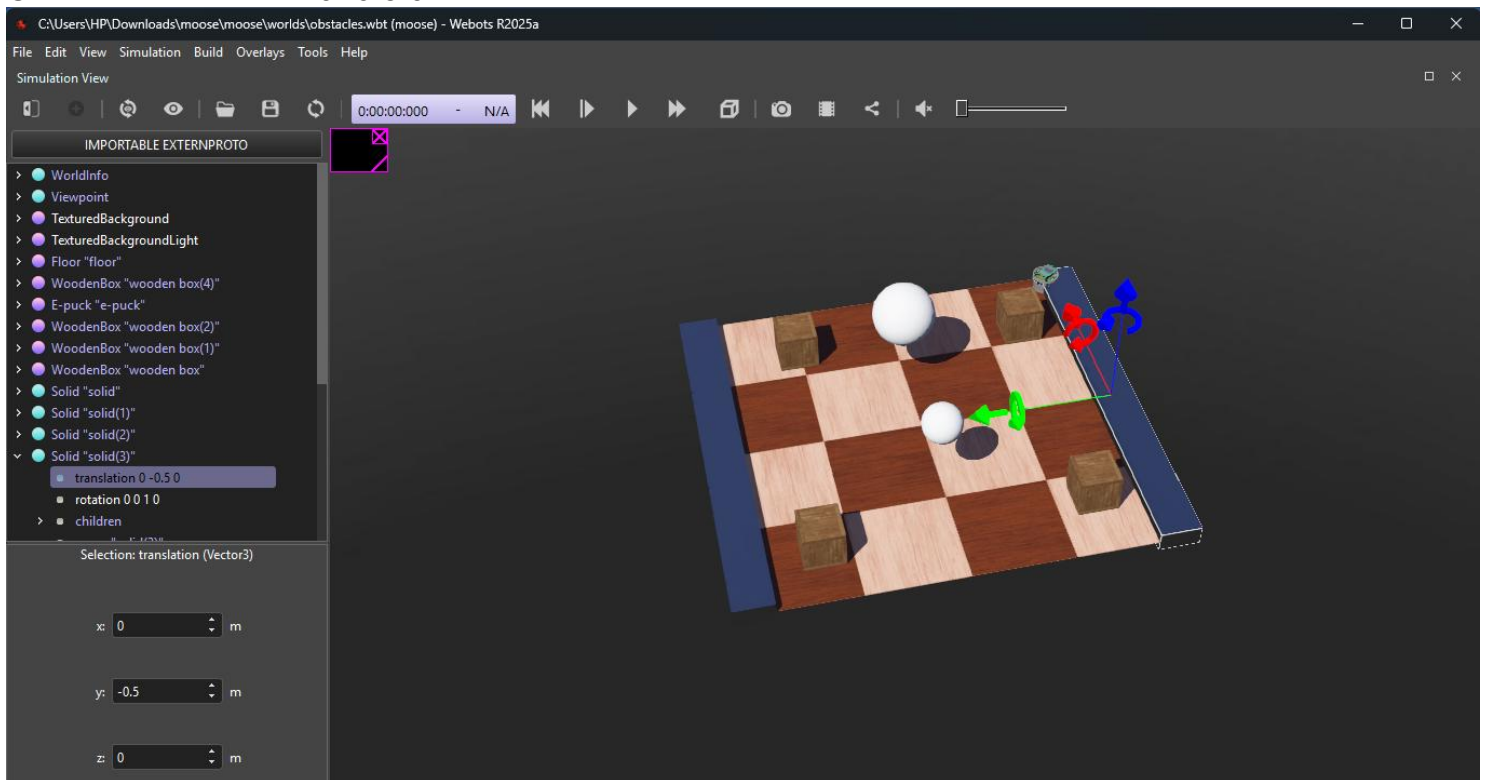
Configura el BoundingBox (Aquí aplicamos USE):  
En el Solid principal, selecciona el campo boundingObject.  
Haz clic en Add y selecciona USE > WALL\_SHAPE.



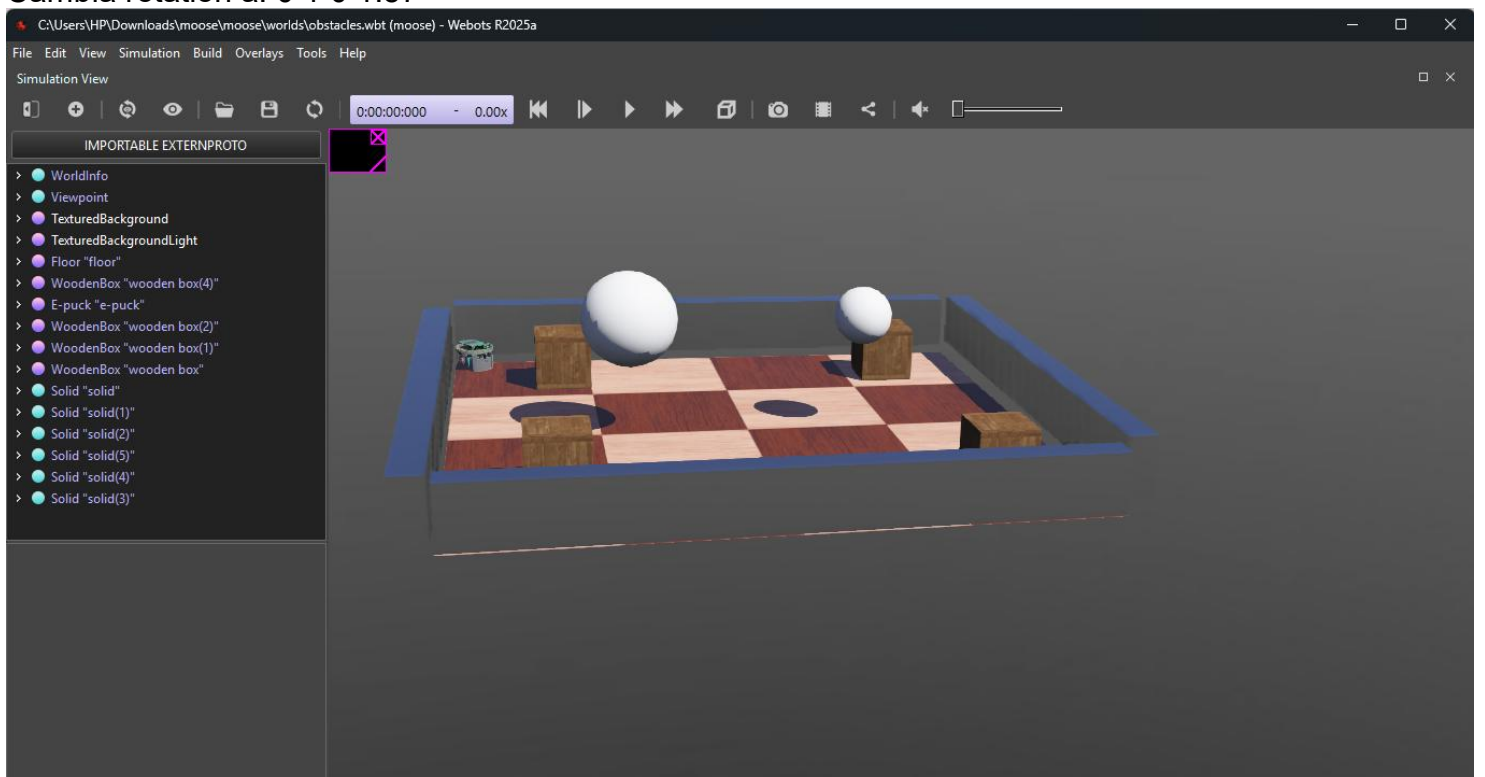
Posiciona la pared:  
En el Solid principal, cambia el translation a: 0 0.5 0 (Esto la pone al fondo).



Crear las otras 3 paredes (Copiar y Pegar)  
 Selecciona el Solid de la pared que acabas de crear.  
 Presiona Ctrl + C y luego Ctrl + V  
 Cambia translation a: 0 -0.5 0.



Pared Derecha (Este):  
 Copia y pega otra vez.  
 Cambia translation a: 0.5 0 0.  
 Cambia rotation a: 0 1 0 1.57



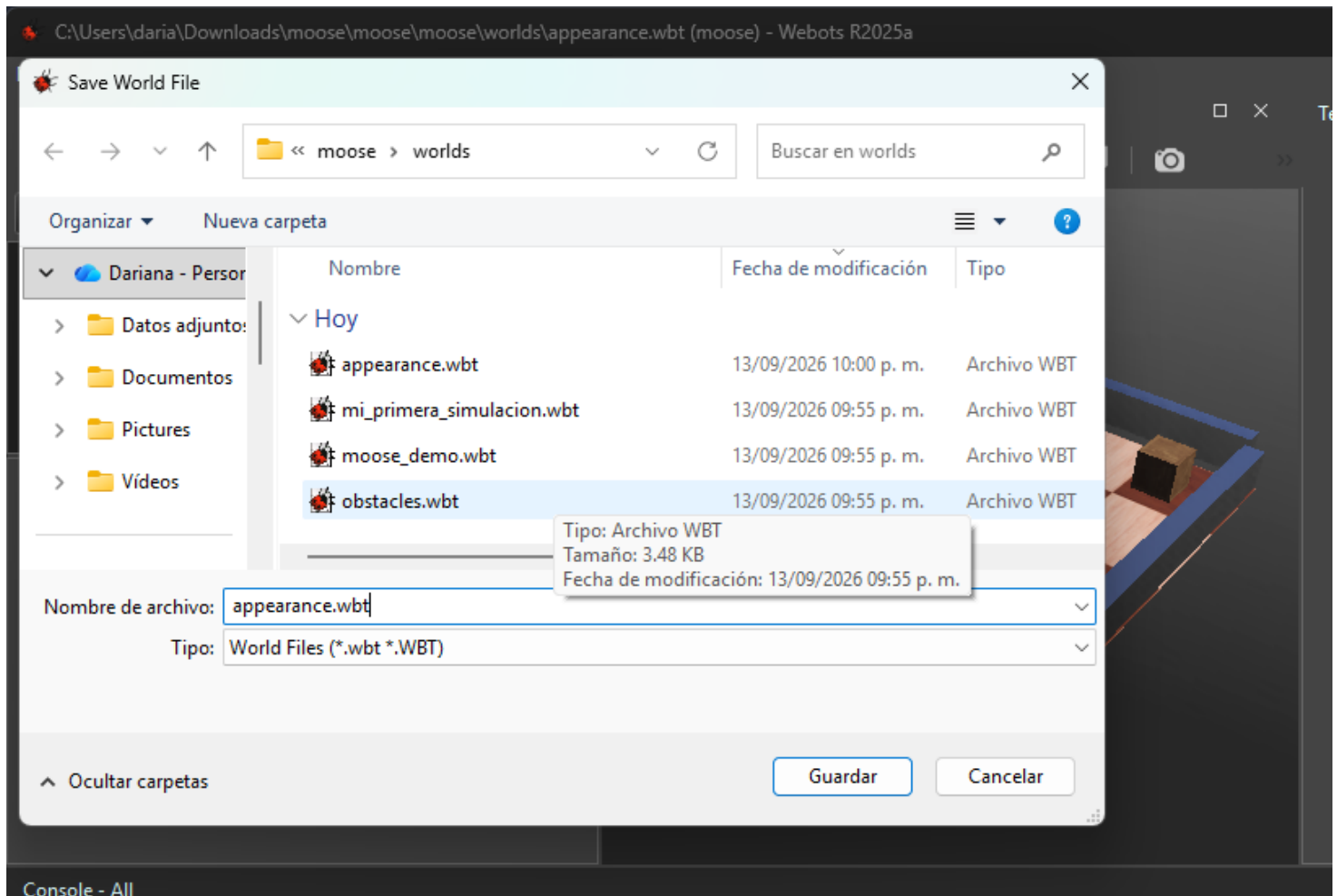
## TUTORIAL #3

En este tercer tutorial nos enfocamos en mejorar la apariencia gráfica de la simulación. Primero guardamos el mundo anterior con un nuevo nombre y empezamos a trabajar con las luces: entendimos la diferencia entre luz direccional, puntual y foco, y cómo afectan el rendimiento y la calidad visual.

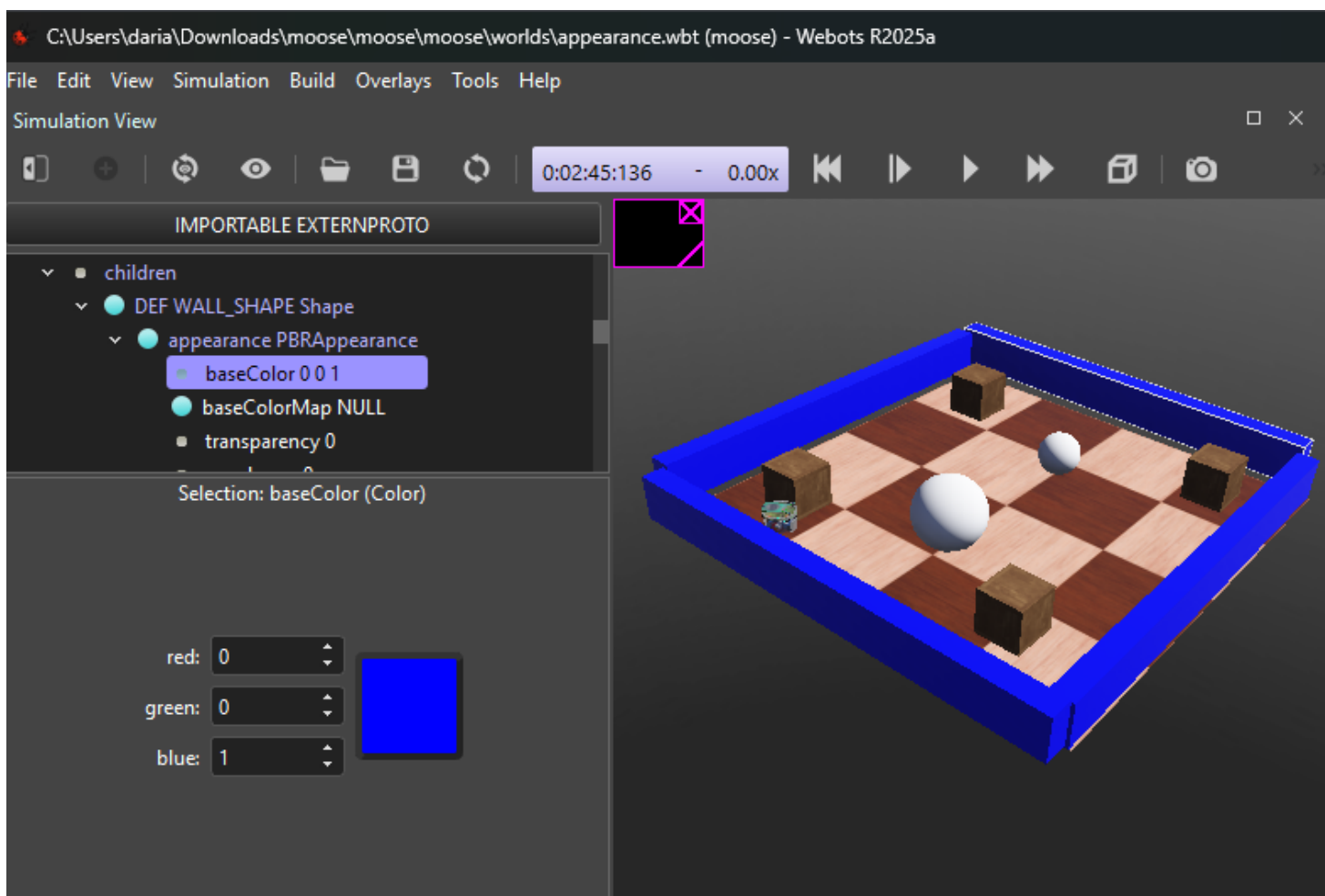
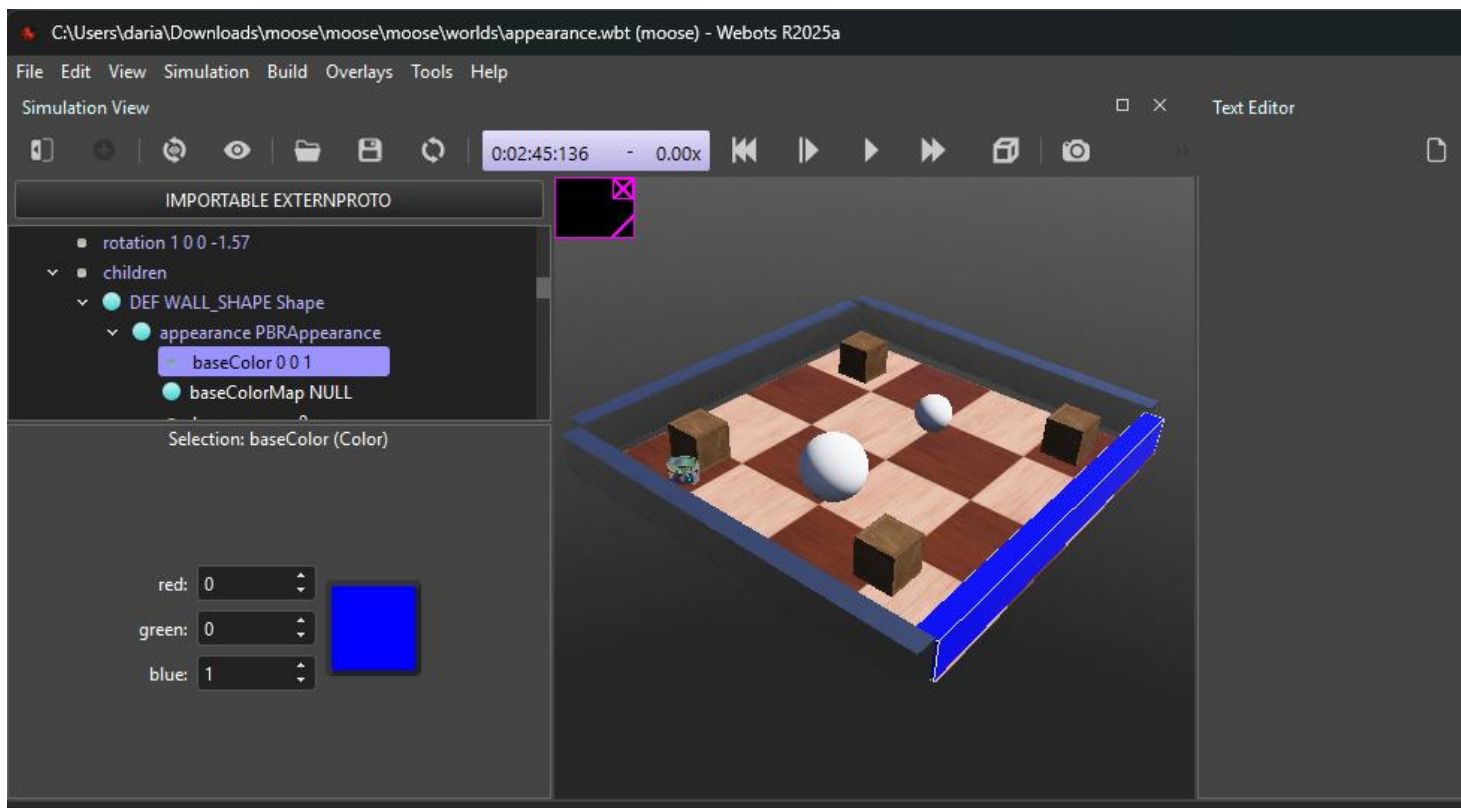
Después modificamos la apariencia de los muros usando PBRAppearance, configurando color azul, metalness en 0 y roughness en 0.5, lo que dio un aspecto más realista. Luego aplicamos un aspecto predefinido a la bola, como *OldSteel*, y finalmente añadimos una textura desde el disco para que la bola mostrara un patrón visible al rodar.

También exploramos las opciones de renderizado, como el modo wireframe y otras vistas opcionales, lo que nos permitió analizar mejor la escena.

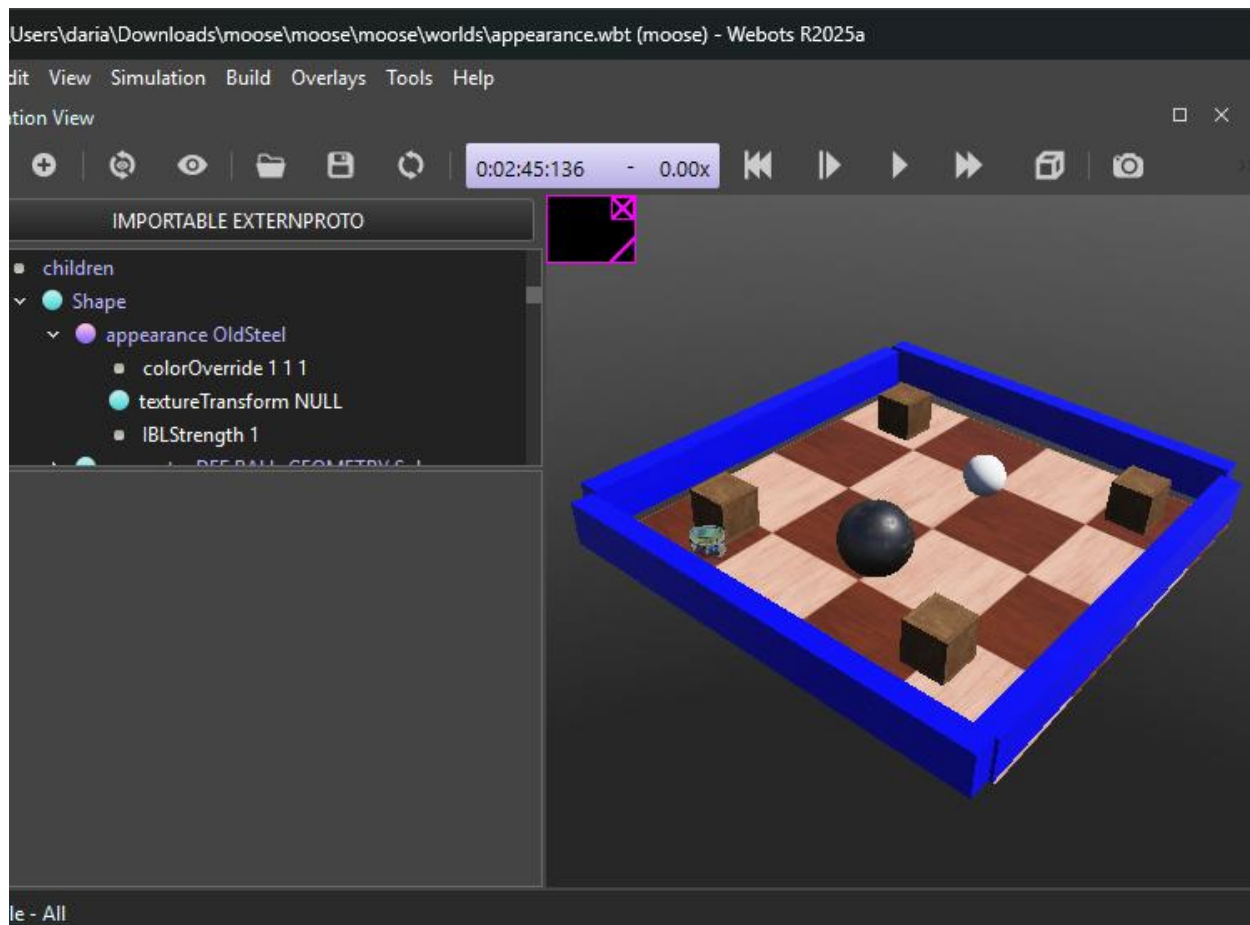
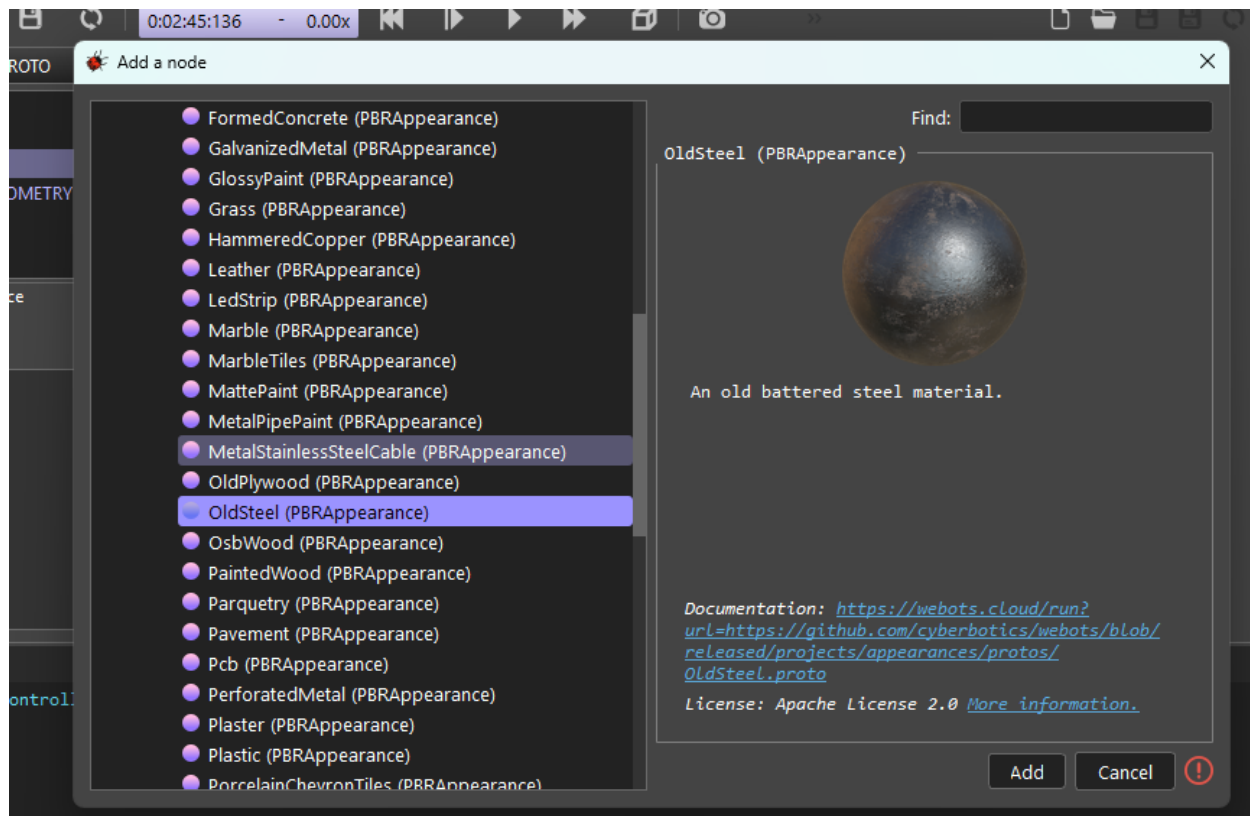
### Hands-on #1: Guardar como appearance.wbt



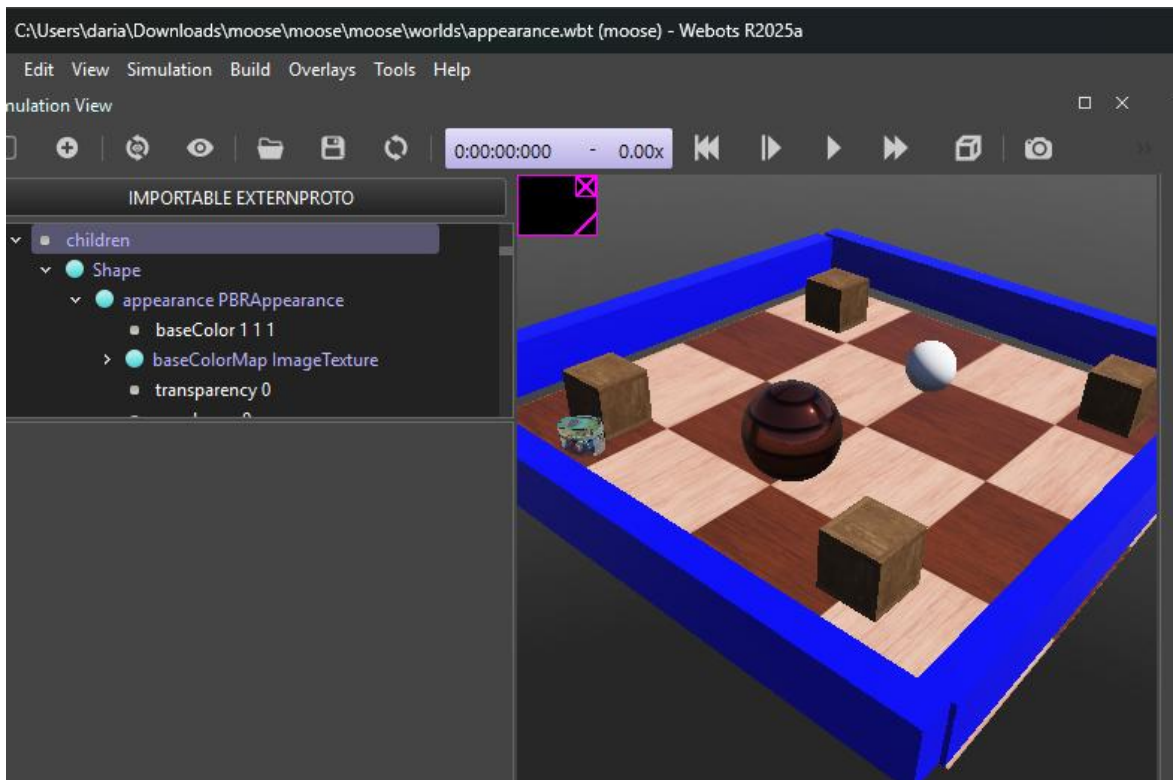
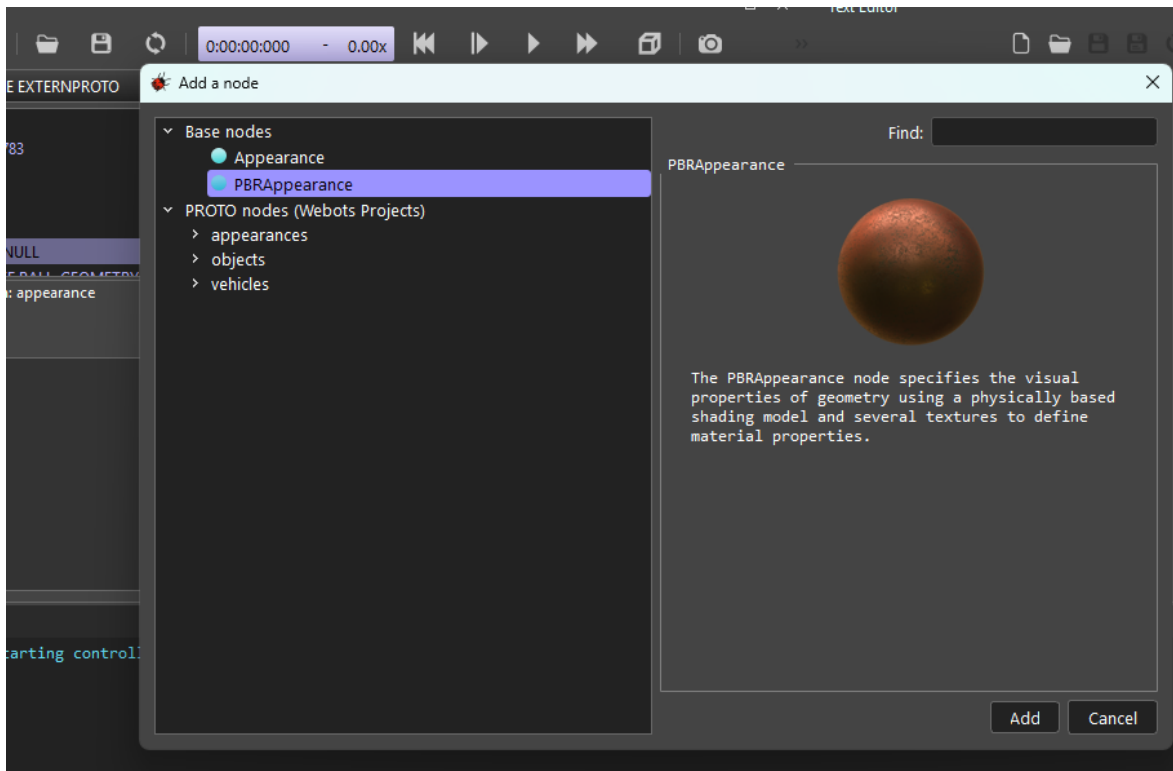
### Hands-on #2: Colorear las paredes de azul



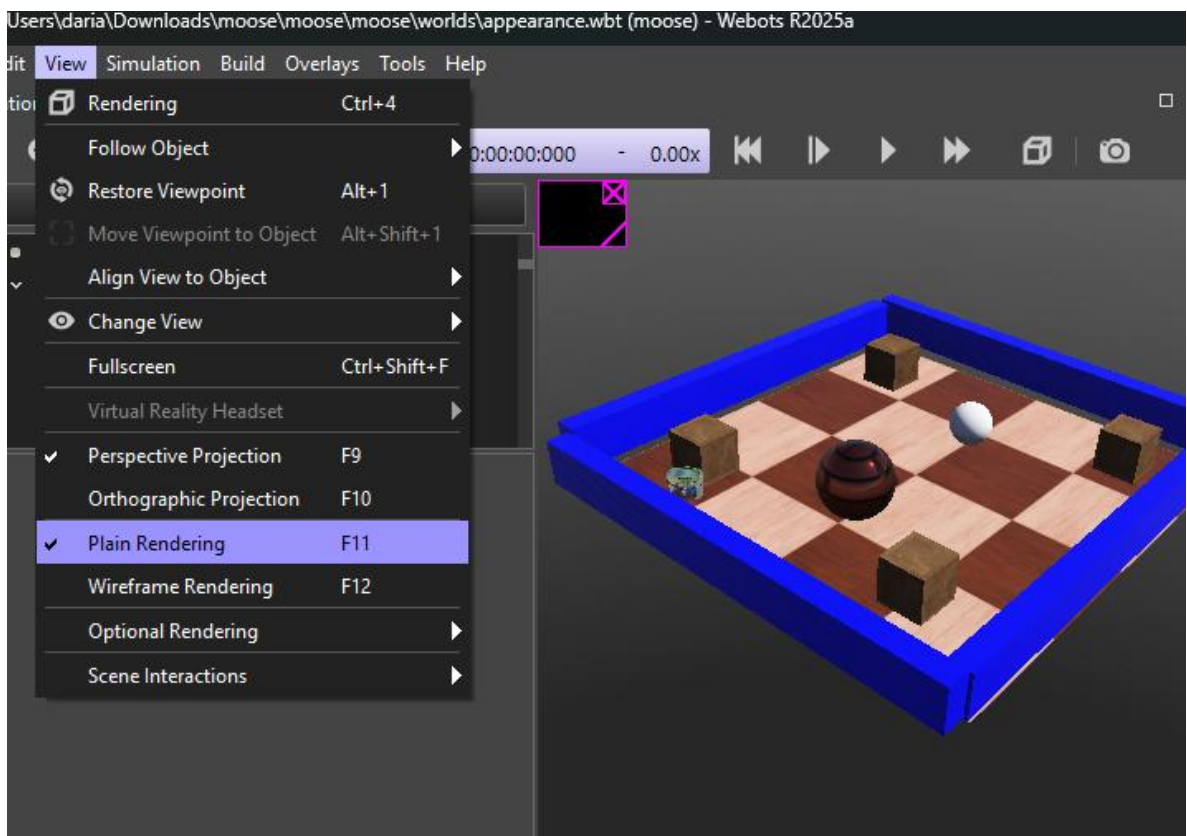
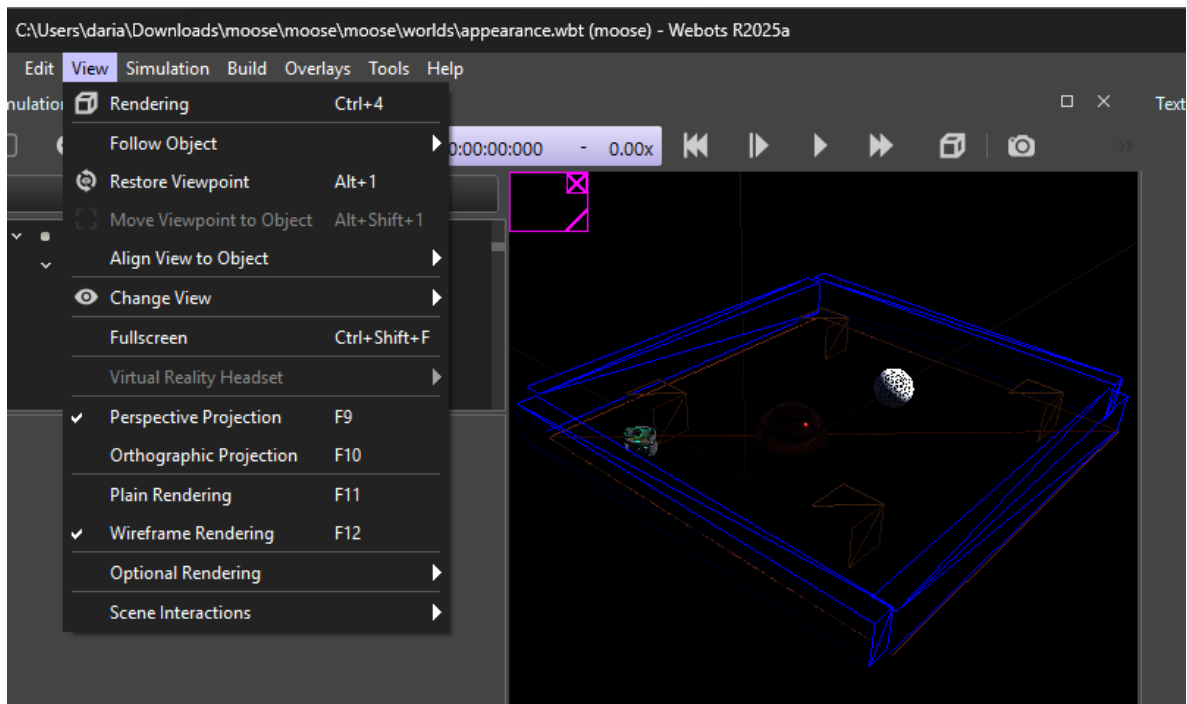
### Hands-on #3: Aplicar OldSteel a la pelota



### Hands-on #4: Aplicar textura desde disco (red\_brick\_wall.jpg)



## Hands-on #5: Modos de renderizado



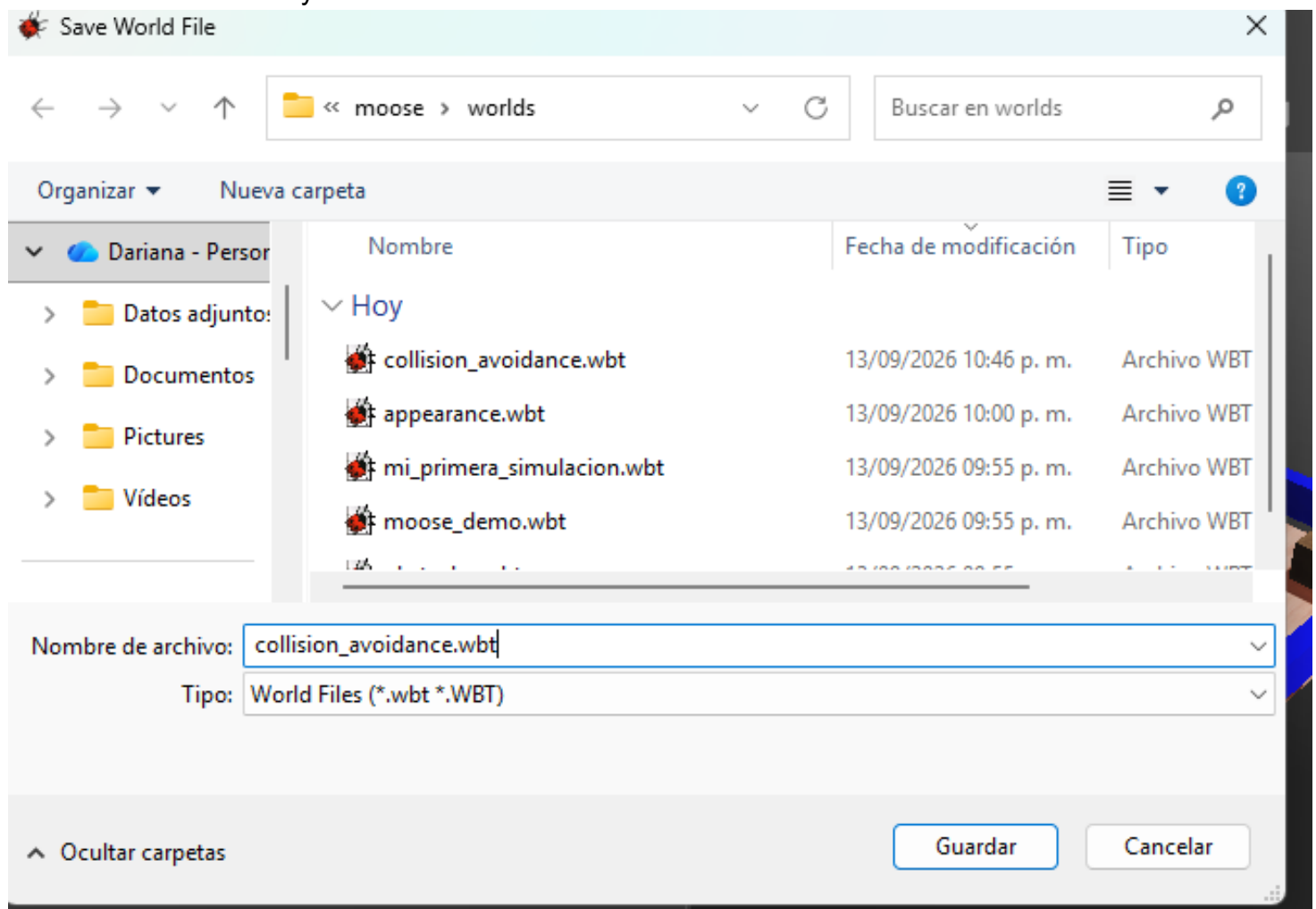
## TUTORIAL #4

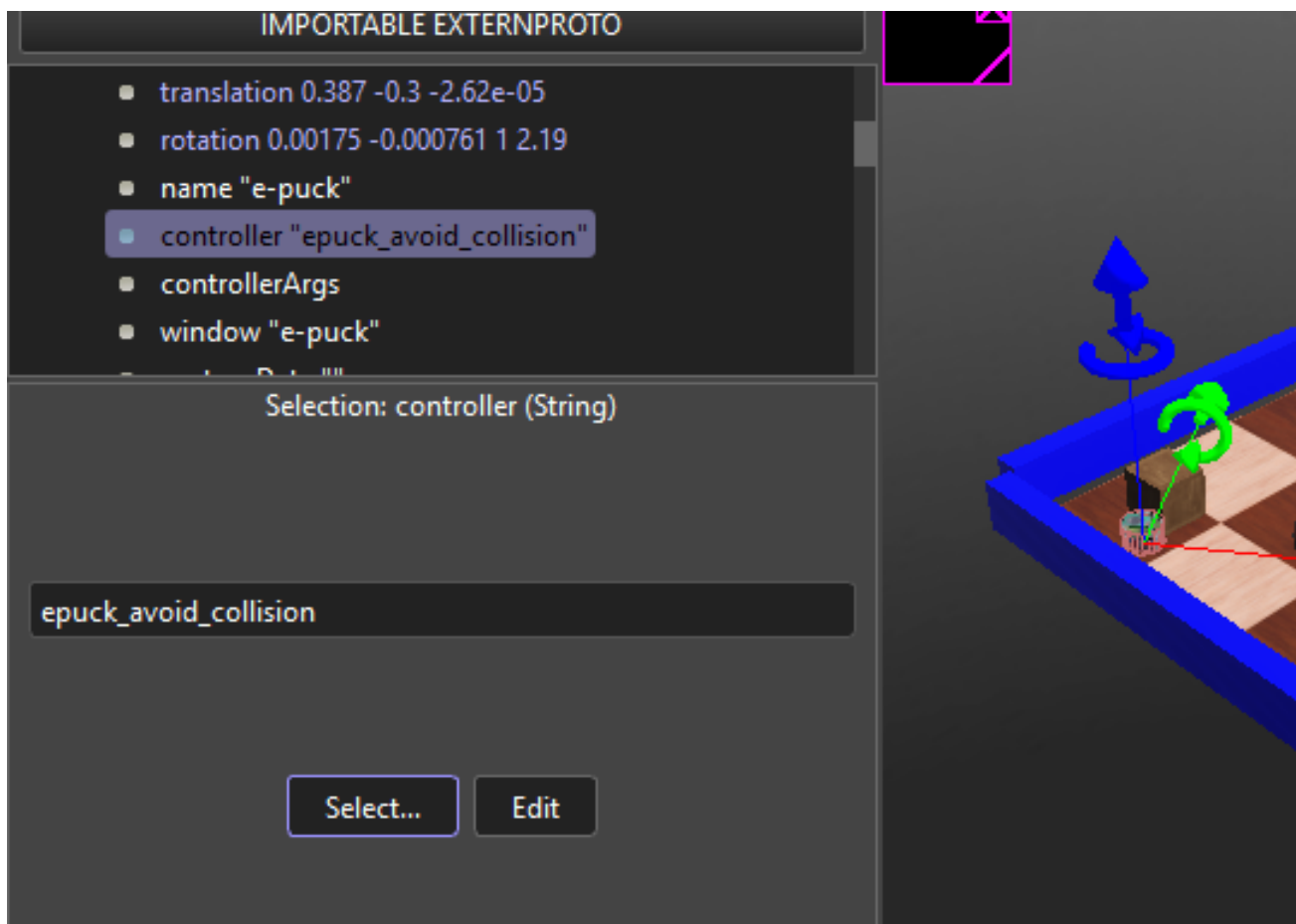
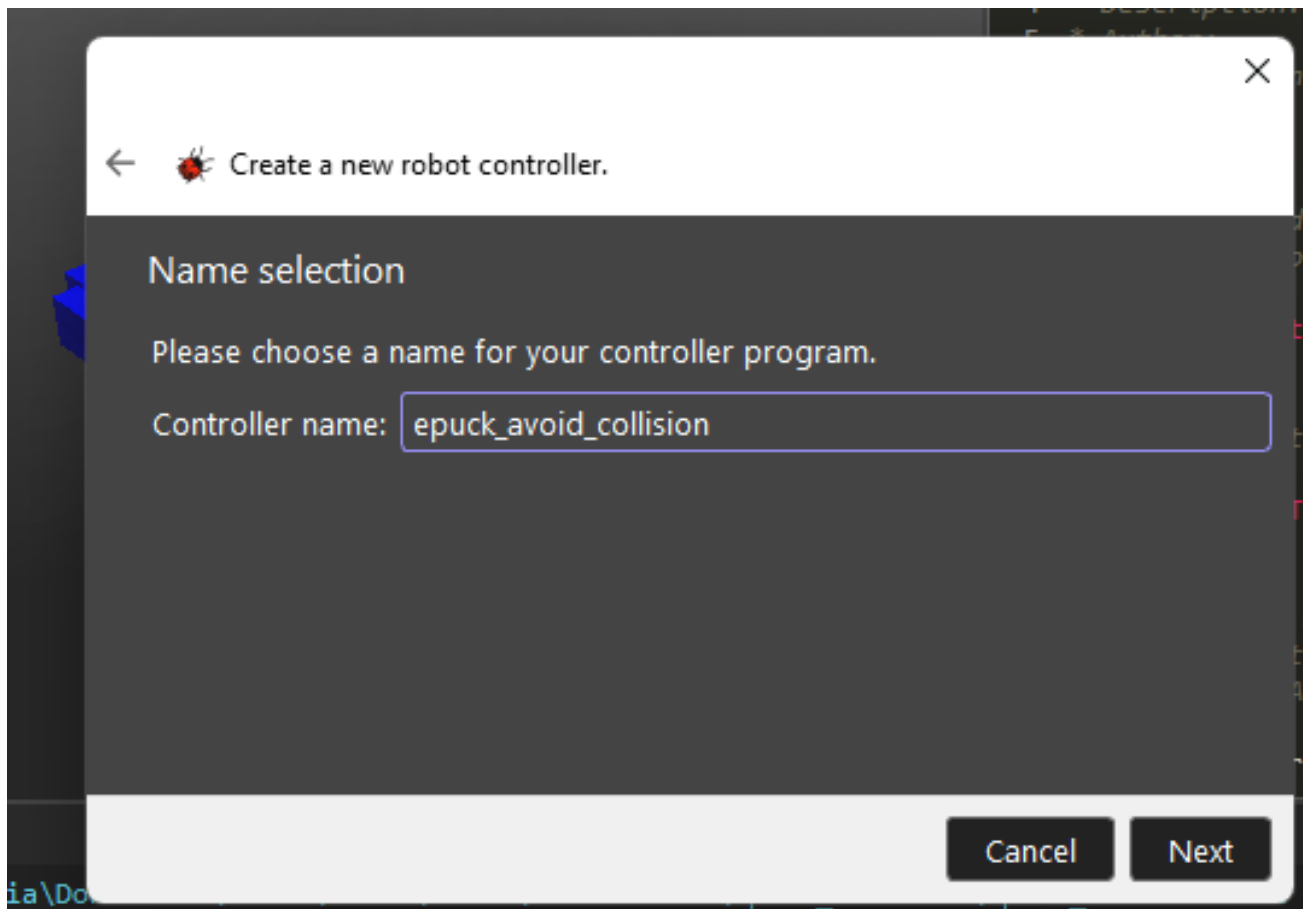
En este cuarto tutorial nos enfocamos en la programación de controladores para el robot e-puck. Primero guardamos el mundo anterior con un nuevo nombre y creamos un controlador llamado *epuck\_avoid\_collision*.

Después entendimos cómo está estructurado el modelo del e-puck: sus 8 sensores de distancia infrarrojos alrededor de la torreta y sus dos motores de ruedas. Con esa información empezamos a programar el controlador en C, inicializando la API de Webots, habilitando los sensores y configurando los motores.

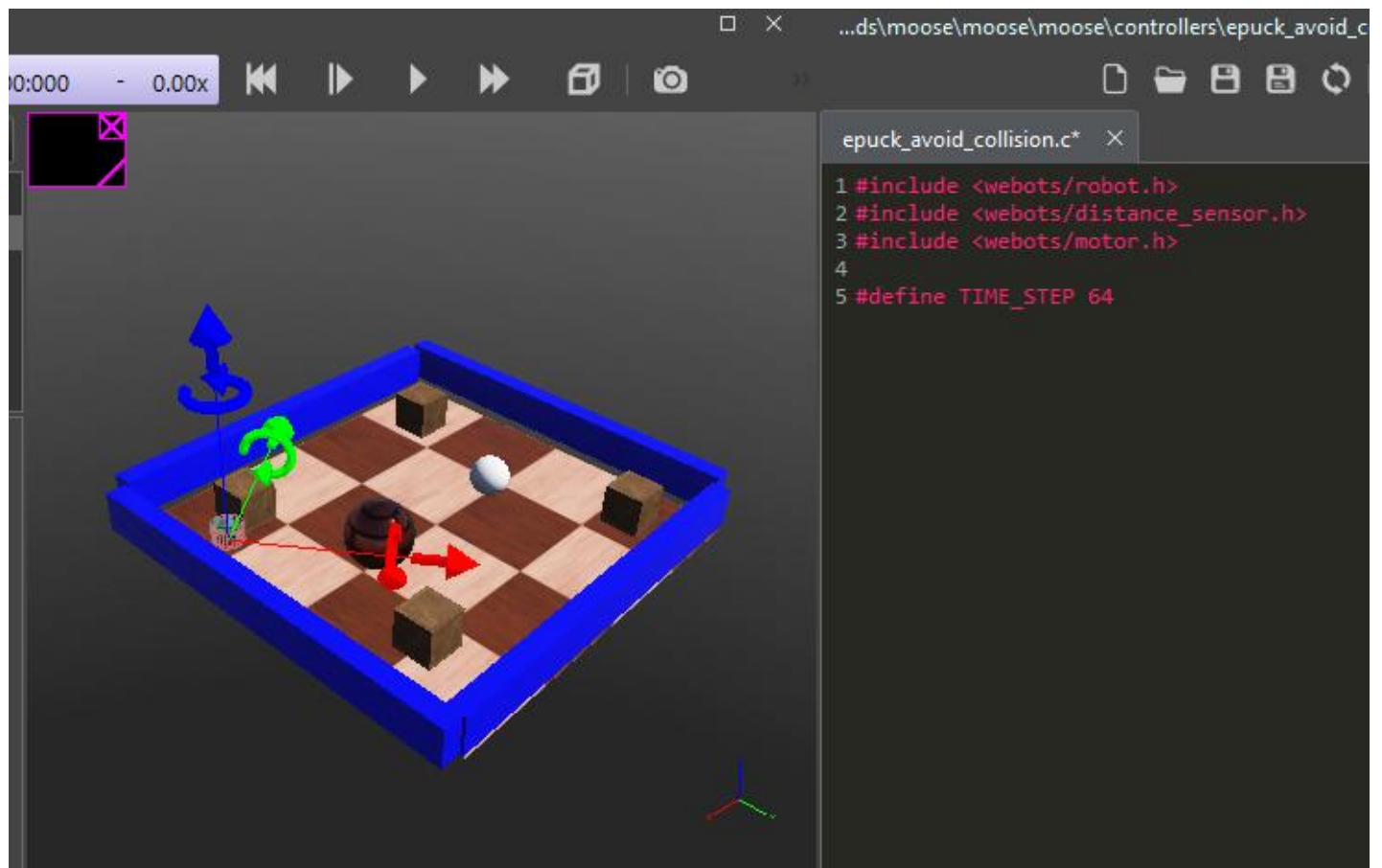
En el bucle principal del programa leímos los valores de los sensores, detectamos obstáculos cuando superaban un umbral y ajustamos la velocidad de las ruedas para girar hacia la dirección libre. Así logramos que el robot avanzara y evitara colisiones de manera sencilla.

### Hands-on #1: Guardar y crear controlador

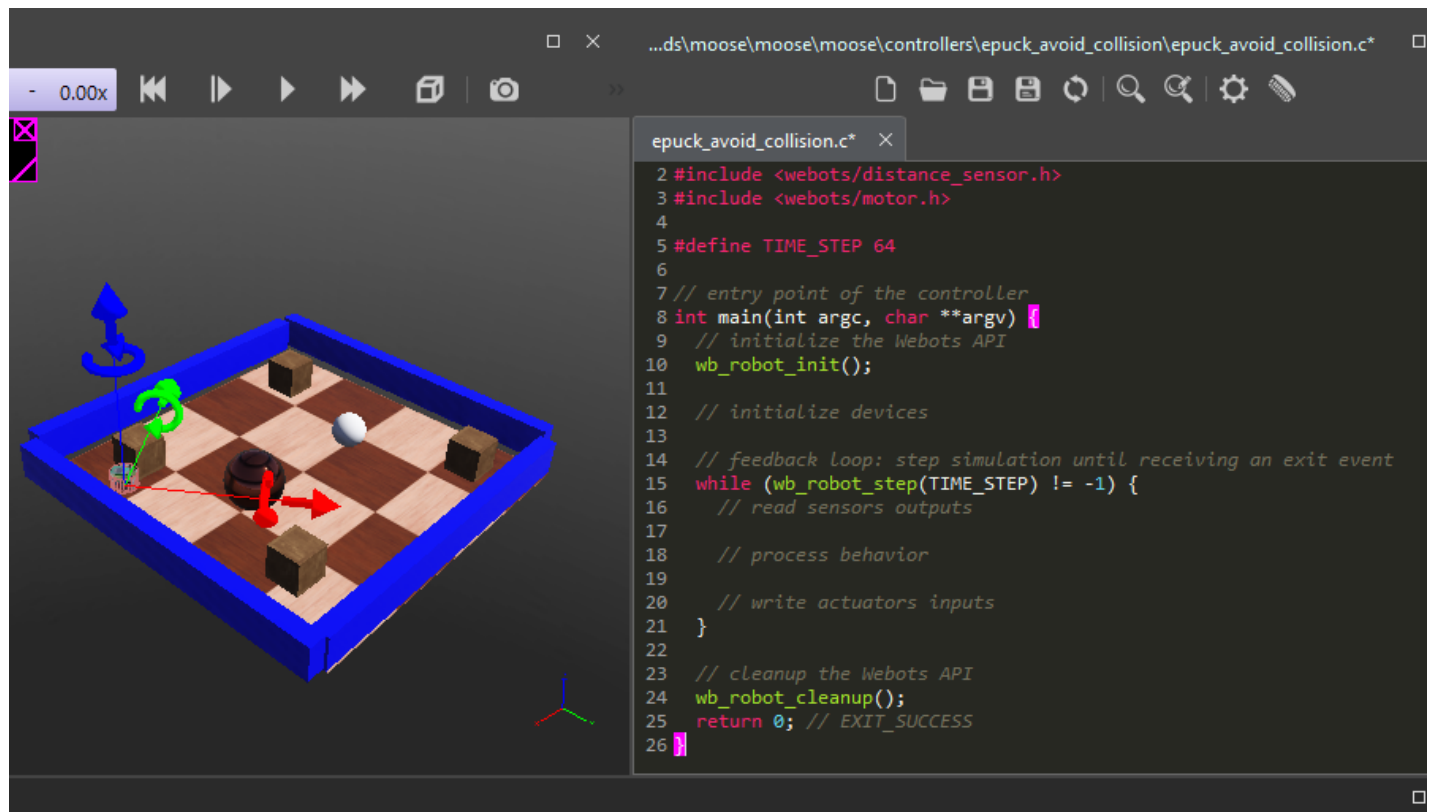




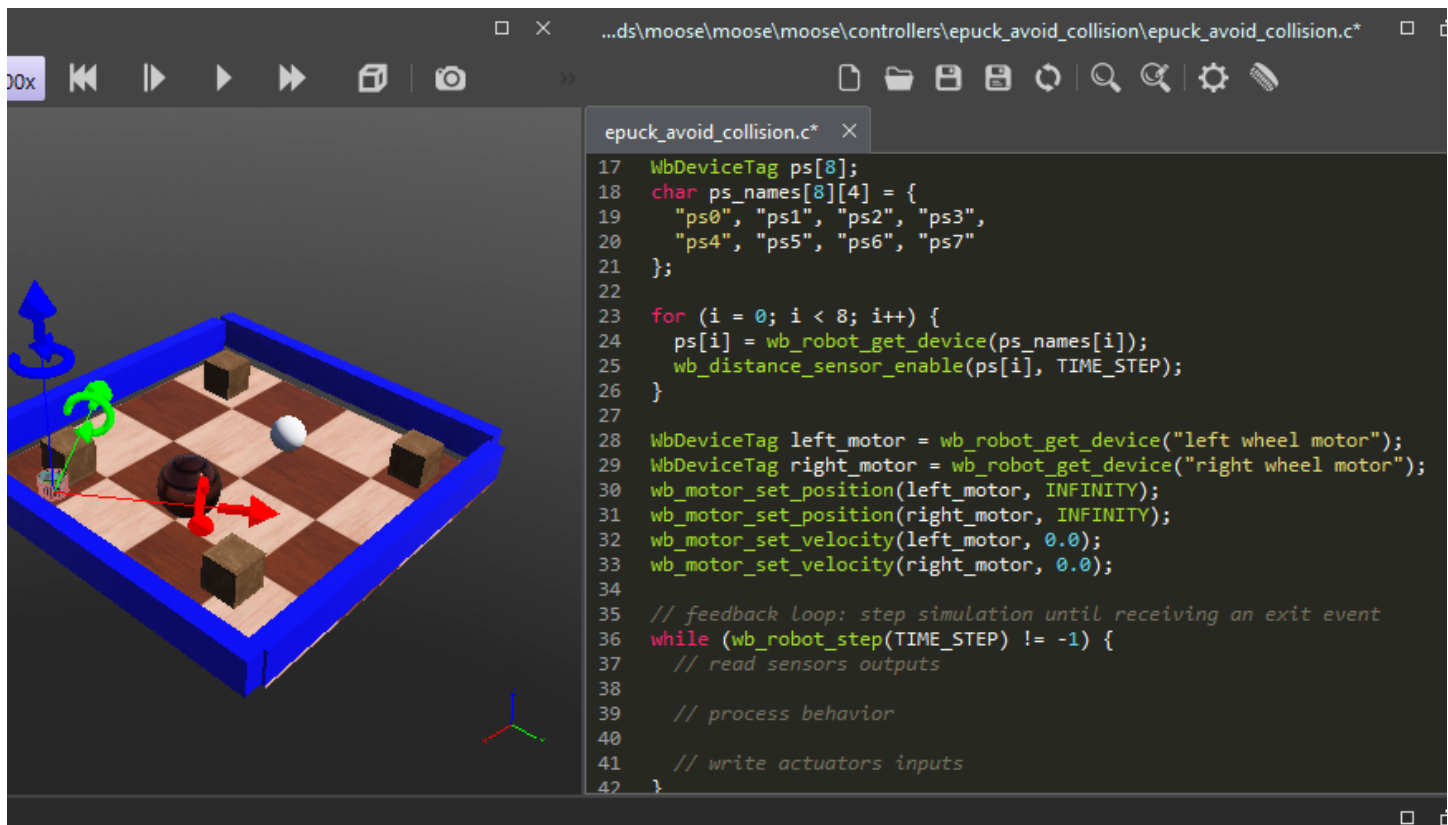
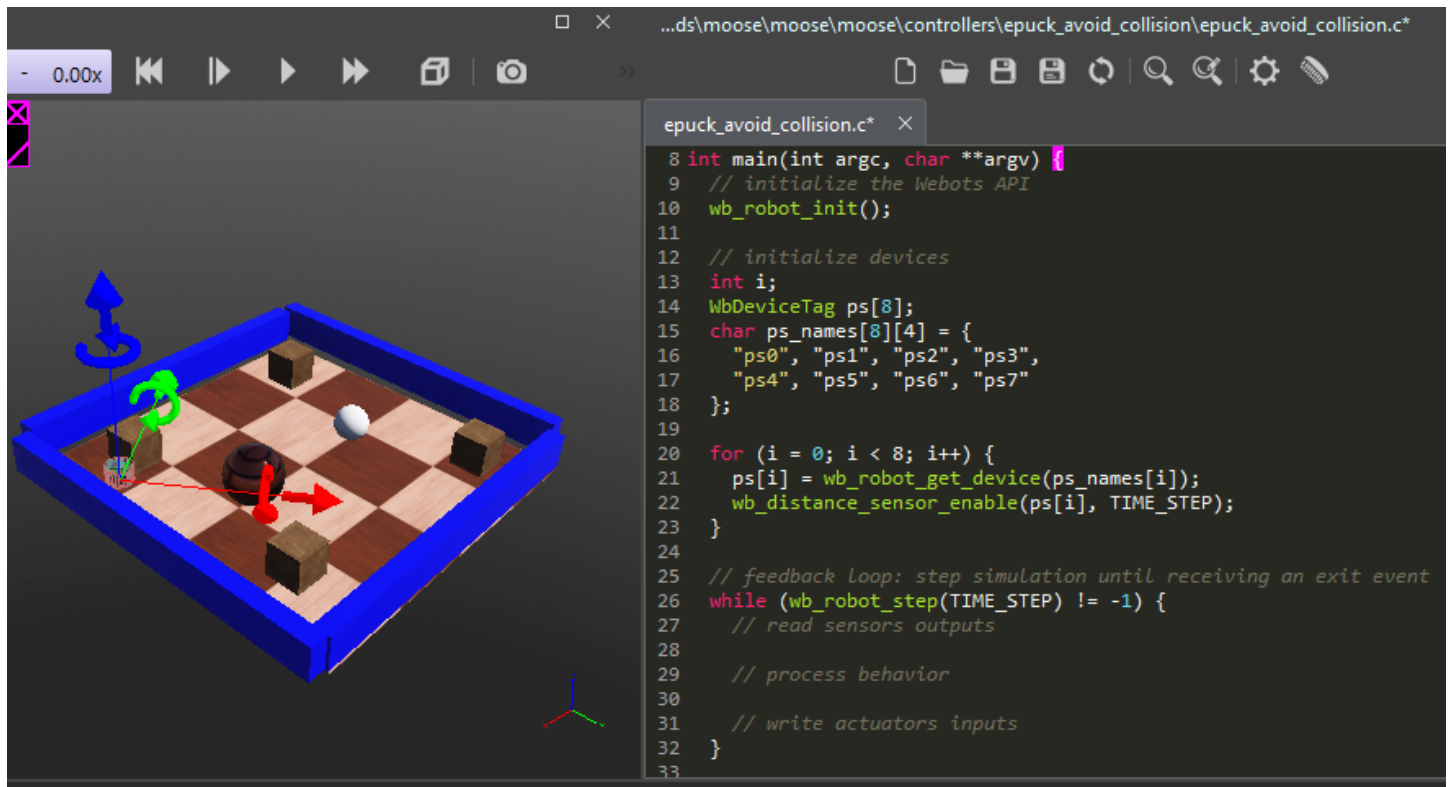
## Hands-on #2: Includes y macros



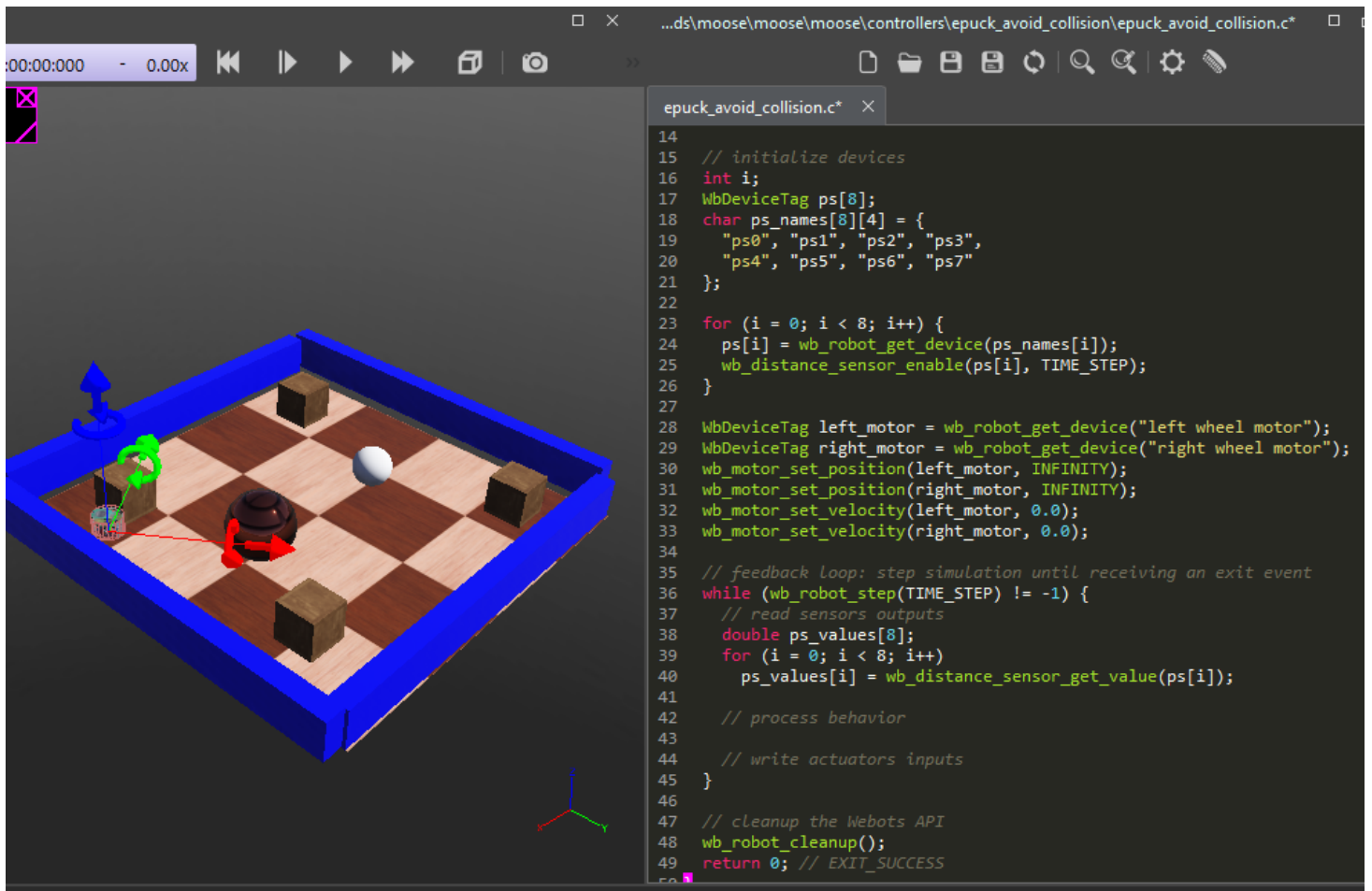
## Hands-on #3: Prototipo de main



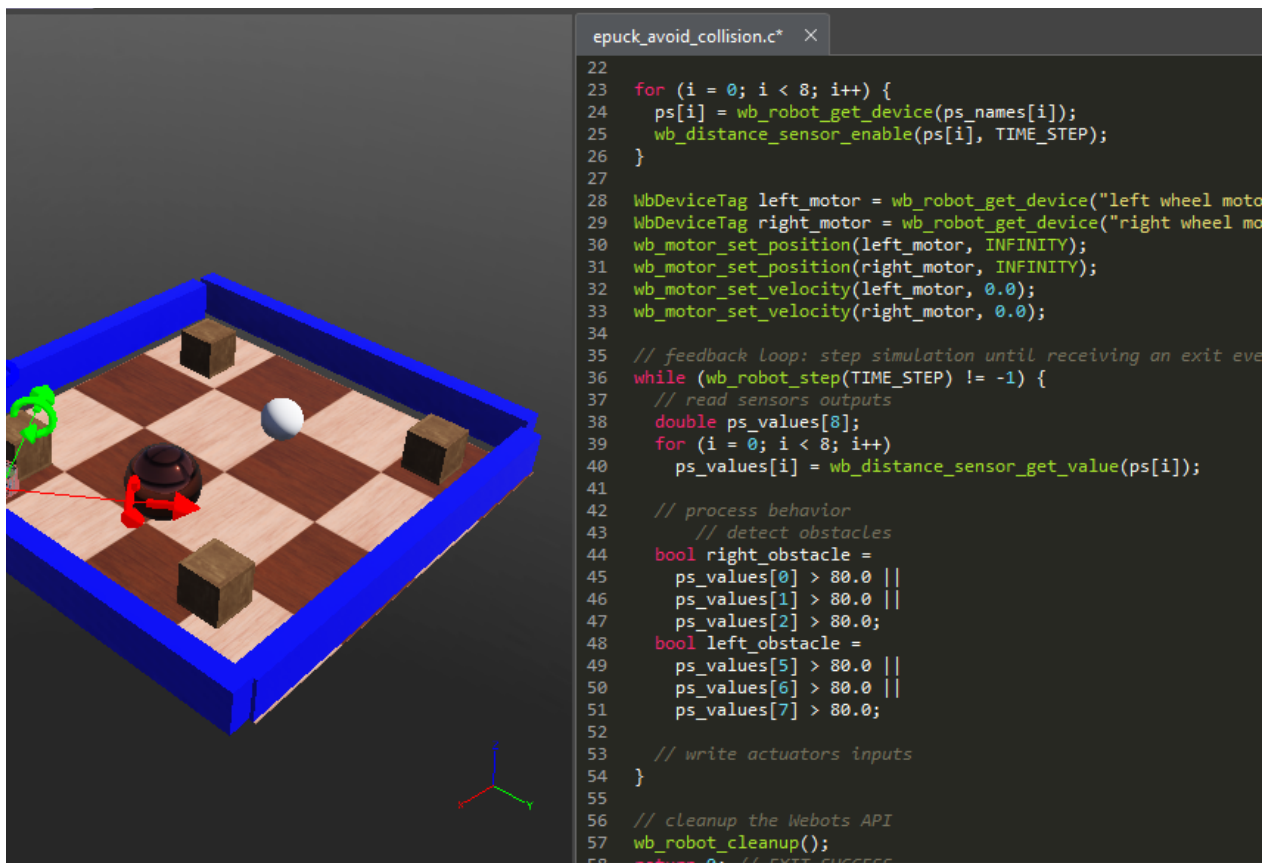
## Hands-on #4: Inicializar dispositivos



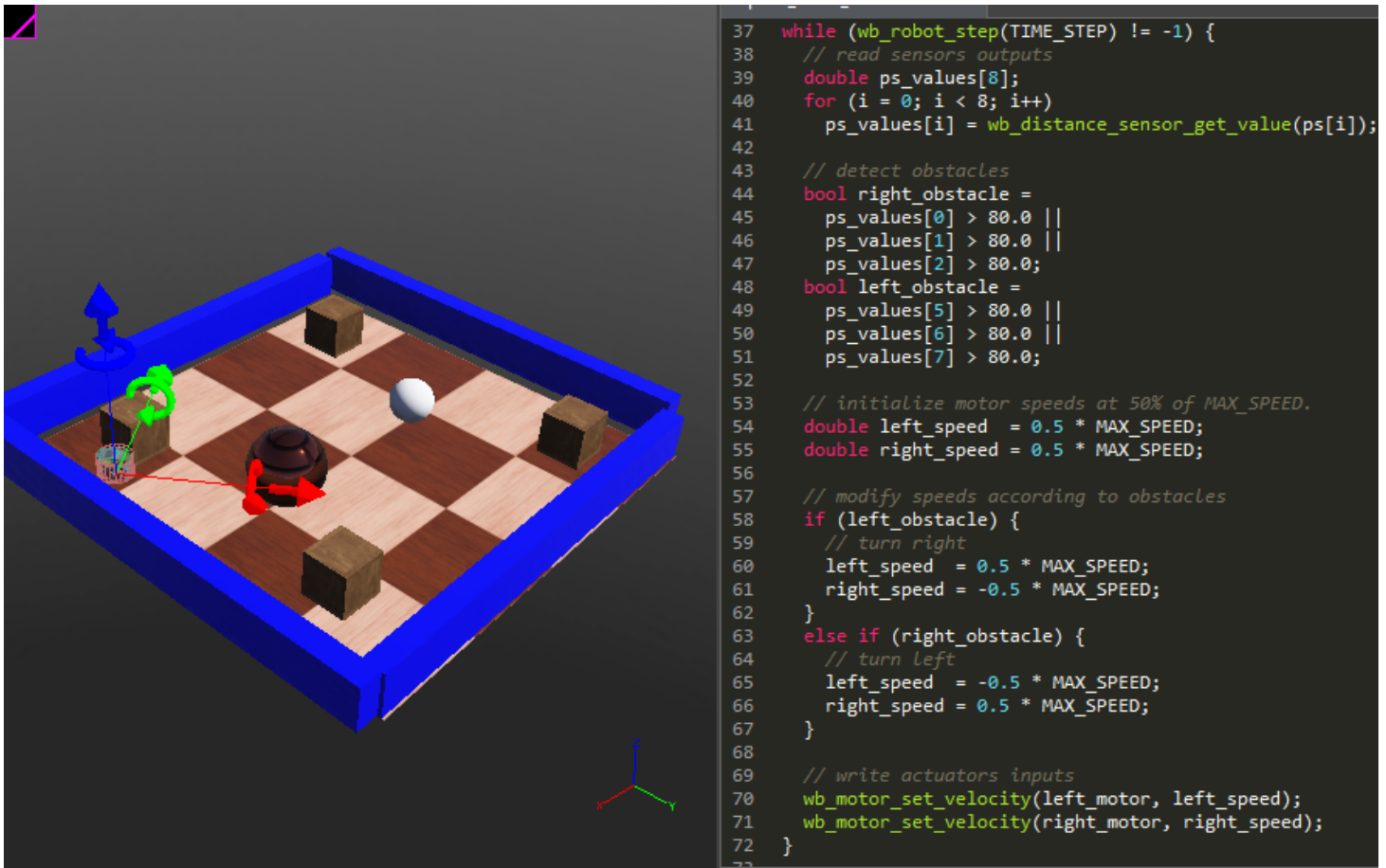
## Hands-on #5: Leer sensores en el bucle



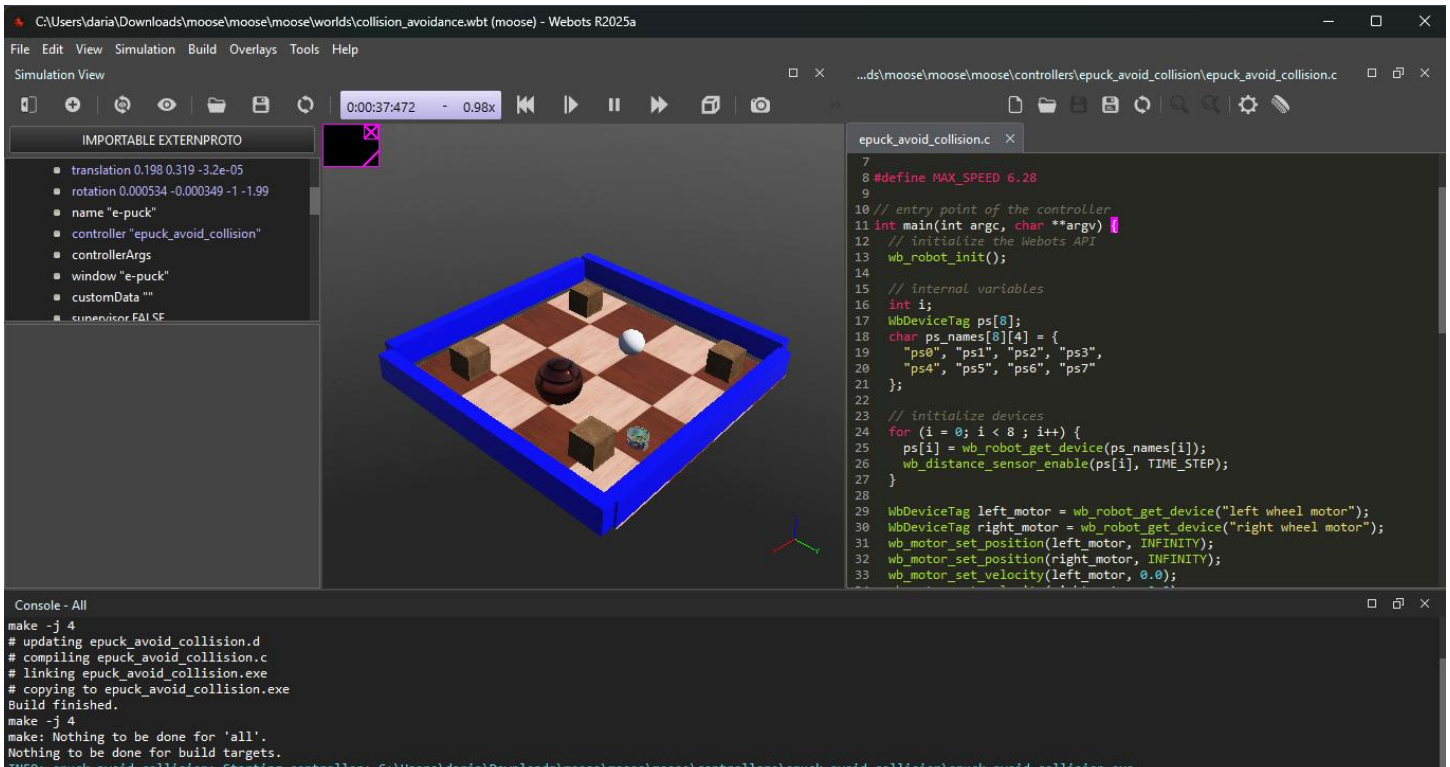
## Hands-on #6: Detectar obstáculos

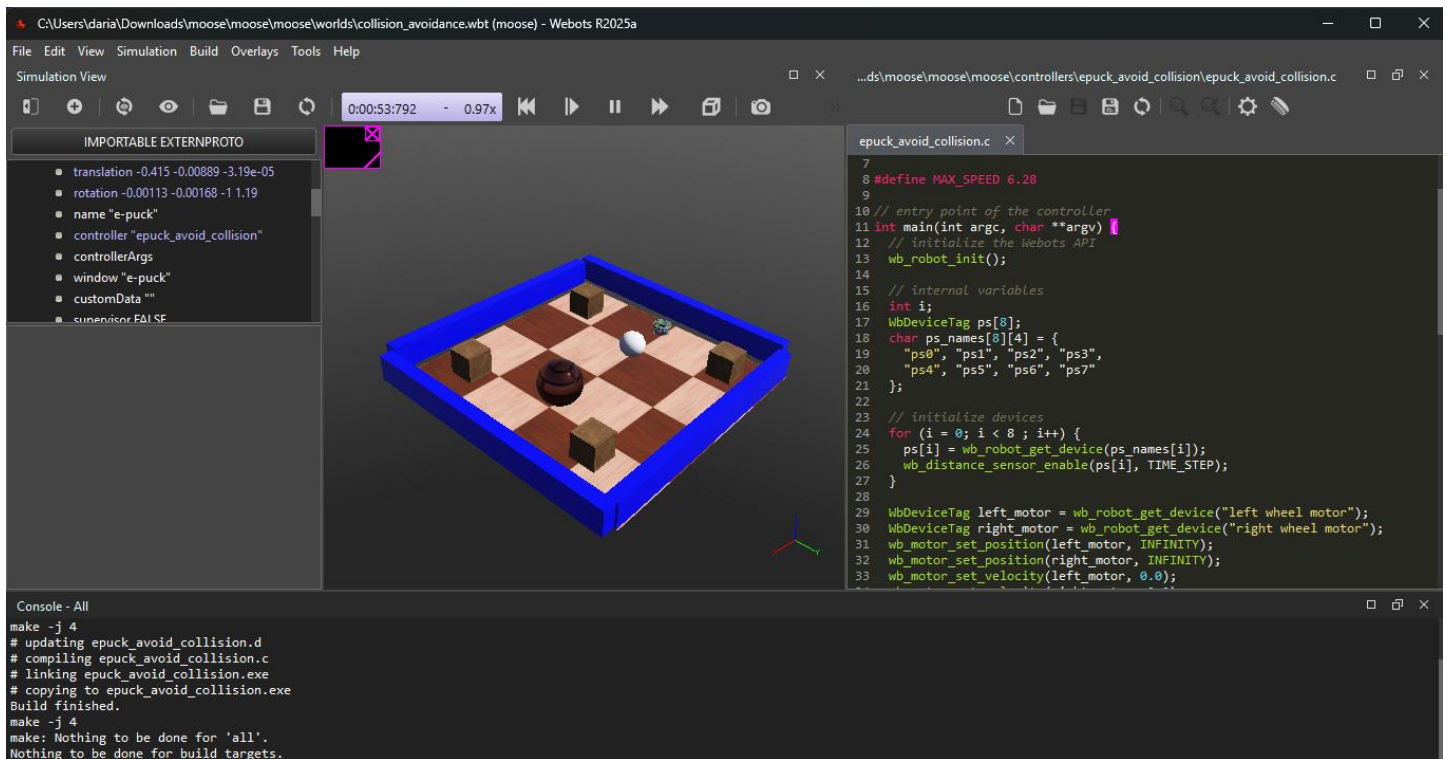


## Hands-on #7: Controlar los motores



## Compilar y Ejecutar





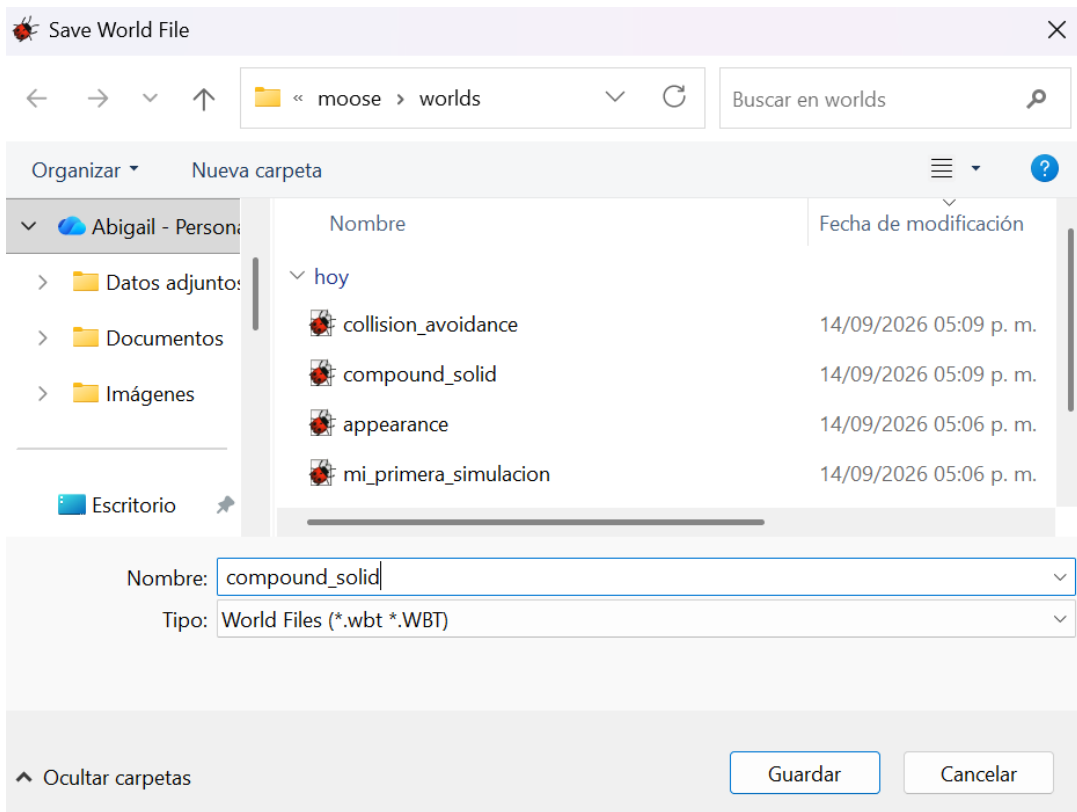
## Tutorial #5

En este quinto tutorial nos concentramos en la simulación física más avanzada. Partimos del mundo anterior y creamos una nueva simulación para construir un sólido compuesto, en este caso una mancuerna formada por un cilindro como mango y dos esferas como pesas en los extremos.

Después configuramos los atributos de física: le asignamos una masa de 2 kg, ajustamos el centro de masa para simular que un lado era más pesado y usamos el campo de rotación para orientar el mango paralelo al suelo. También exploramos cómo elegir objetos delimitadores eficientes (esferas, cajas, cilindros) para mejorar la velocidad de la simulación.

Más adelante modificamos las propiedades de contacto, cambiando la fricción para observar cómo la mancuerna se deslizaba en lugar de rodar. Finalmente revisamos parámetros importantes como el `basicTimeStep`, `ERP` y `CFM`, que influyen en la precisión y estabilidad de la simulación.

1 Creación de una nueva simulación a partir del tutorial anterior. Llamado `compound_solid.wbt`

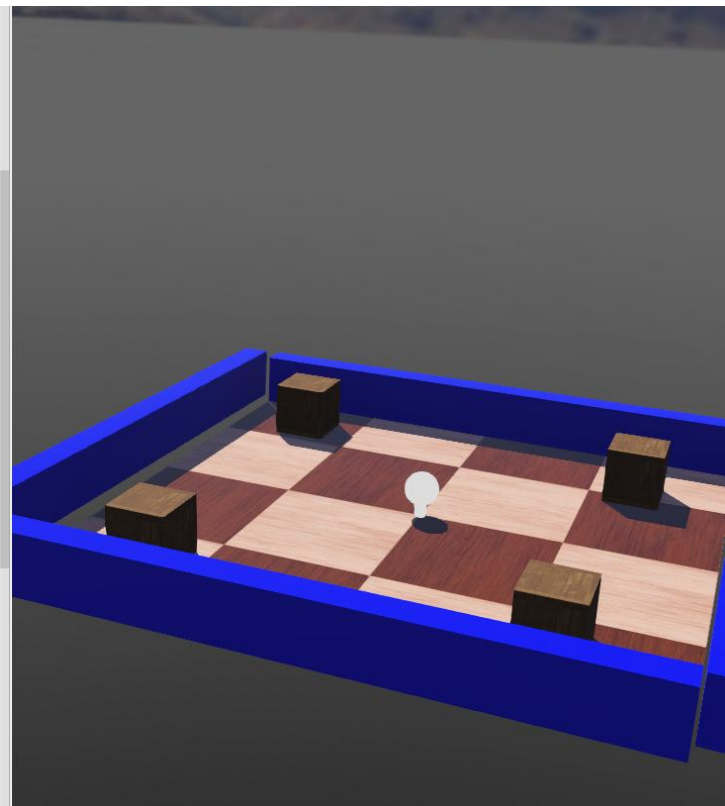


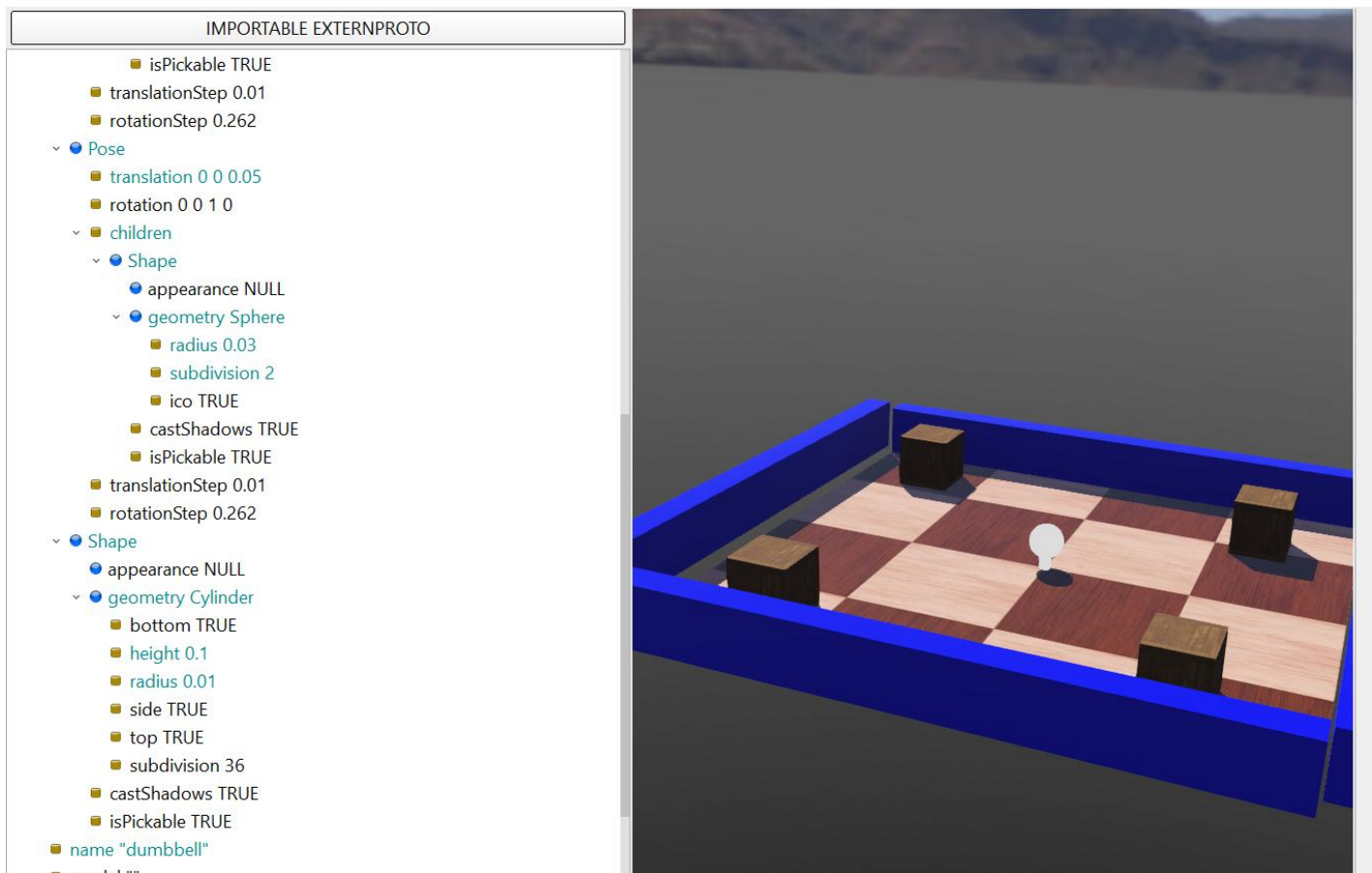
2: Crea la mancuerna siguiendo la figura. Crea primero el mango sin colocarlo en un nodo de pose (así el eje del mango tendrá la misma dirección que el eje z del sólido). El mango debe tener una longitud de 0,1 m y un radio de 0,01 m. Los pesos deben tener un radio de 0,03 m y una subdivisión de 2.

```

Solid "dumbbell"
  translation 0 0 0
  rotation 0 0 1 0
  children
    Pose
      translation 0 0 -0.05
      rotation 0 0 1 0
      children
        Shape
          appearance NULL
          geometry Sphere
            radius 0.03
            subdivision 2
            ico TRUE
            castShadows TRUE
            isPickable TRUE
          translationStep 0.01
          rotationStep 0.262
    Pose
      translation 0 0 0.05
      rotation 0 0 1 0
      children
        Shape
          appearance NULL
          geometry Sphere
            radius 0.03
            subdivision 2
            ico TRUE
            castShadows TRUE

```





3. Configuración de la masa de la mancuerna a 2 kg.

- boundingObject NULL
- physics Physics
  - density 2
  - mass -1
  - centerOfMass
  - translation 0 0 0

#### 4. Modificación del centro de masa de la mancuerna sobre el eje Z.

Selection: centerOfMass (Vector3 #1)

x:  m

y:  m

z:  m

#### 5. Rotación de la mancuerna para colocar el eje del mango paralelo al suelo.

IMPORTABLE EXTERNPROTO

- > Floor "floor"
- > WoodenBox "wooden box(4)"
- > WoodenBox "wooden box(2)"
- > WoodenBox "wooden box(1)"
- > WoodenBox "wooden box"
- > Solid "solid(2)"
- > Solid "solid(5)"
- > Solid "solid(4)"
- > Solid "solid(3)"
- > Solid "dumbbell"
  - translation 0 0 0
  - rotation 0 1 0 1.57
  - children
    - name "dumbbell"
    - model ""
    - description ""
    - contactMaterial "default"
    - immersionProperties
    - boundingObject NULL

Selection: rotation (Rotation)

Rotation type:

x:  m

y:  m

z:  m

angle:  rad

## 6. Configuración de las propiedades de contacto para modificar la fricción de la mancuerna.

### contactProperties

#### ● ContactProperties

■ material1 "dumbbell"

■ material2 "default"

▼ ■ coulombFriction

0

■ frictionRotation 0 0

■ rollingFriction 0 0 0

■ bounce 0.5

File Edit View Simulation Build Overlays Tools Help

Simulation View

0:27:25:248 - N/A

IMPORTABLE EXTERNPROTO

■ translation 0 0 -0.05

■ rotation 0 0 1 0

▼ ■ children

▼ ● Shape

● appearance NULL

► ● geometry DEF G2 Sph

■ castShadows TRUE

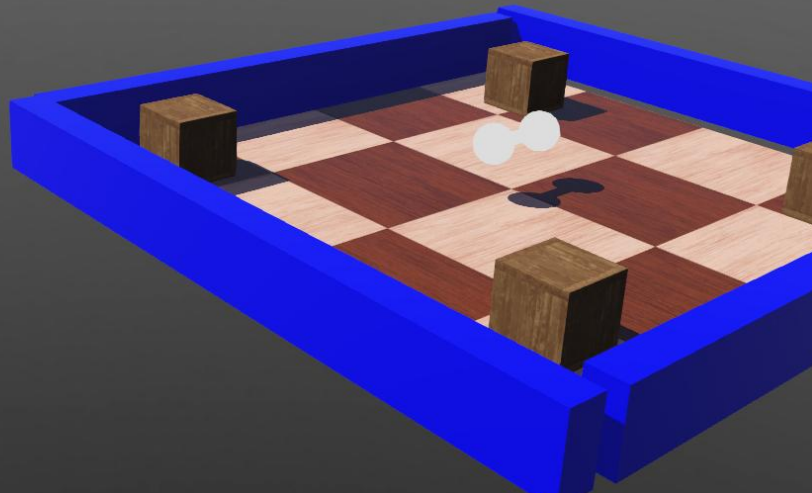
■ isPickable TRUE

■ translationStep 0.01

■ rotationStep 0.262

▼ ● Pose

■ translation 0 0 0.05



## 7. Configuración de parámetros adicionales de física del sólido.

The image displays a software interface for configuring physics parameters for a solid object. It is divided into three main sections: a left-hand tree view, a right-hand detail view, and a bottom configuration panel.

**Left Panel (Tree View):**

- physics Physics
  - density -1
  - mass 2
  - centerOfMass
  - inertiaMatrix
  - damping Damping
    - linear 0.2
    - angular 0.2

**Right Panel (Detail View):**

- physics Physics
  - density -1
  - mass 2
  - centerOfMass
  - inertiaMatrix
    - 1 1 1
    - 0 0 0

**Bottom Panel (WorldInfo):**

- WorldInfo
  - info
  - title ""
  - window "<none>"
  - gravity 9.81
  - CFM 1e-05
  - ERP 0.2
  - physics "<none>"
  - basicTimeStep 16
  - FPS 60
  - optimalThreadCount 1
  - physicsDisableTime 1

## Tutorial #6

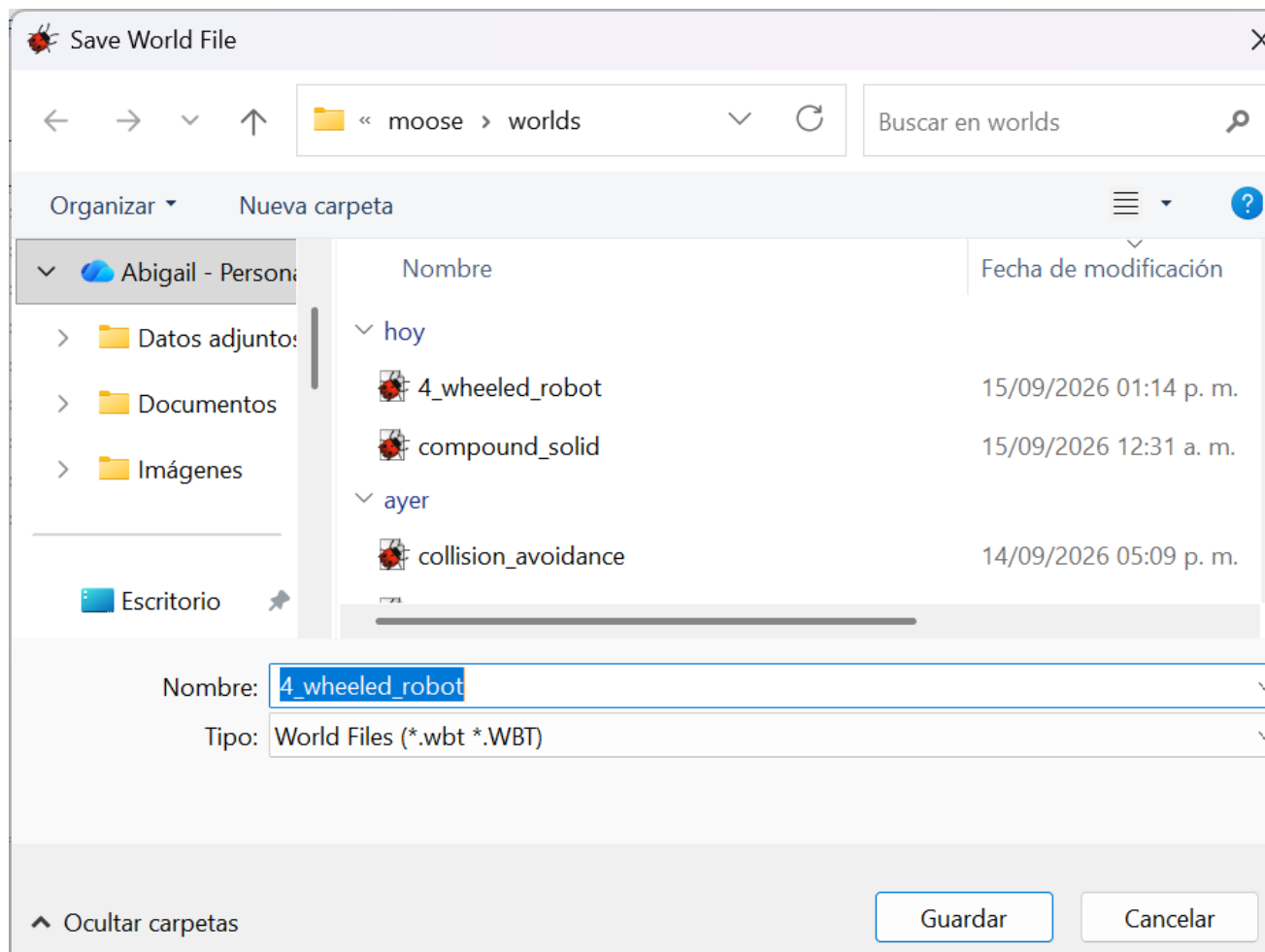
En este sexto tutorial dimos el salto a crear nuestro primer robot desde cero. Partimos del mundo anterior y lo guardamos con un nuevo nombre, eliminando los objetos que ya no necesitábamos.

Diseñamos la estructura del robot con un nodo Robot como raíz, una carrocería en forma de caja roja y cuatro ruedas verdes conectadas mediante HingeJoint con sus respectivos motores rotacionales. Aprendimos a configurar las posiciones, anclas y ejes de las articulaciones para que las ruedas pudieran girar correctamente.

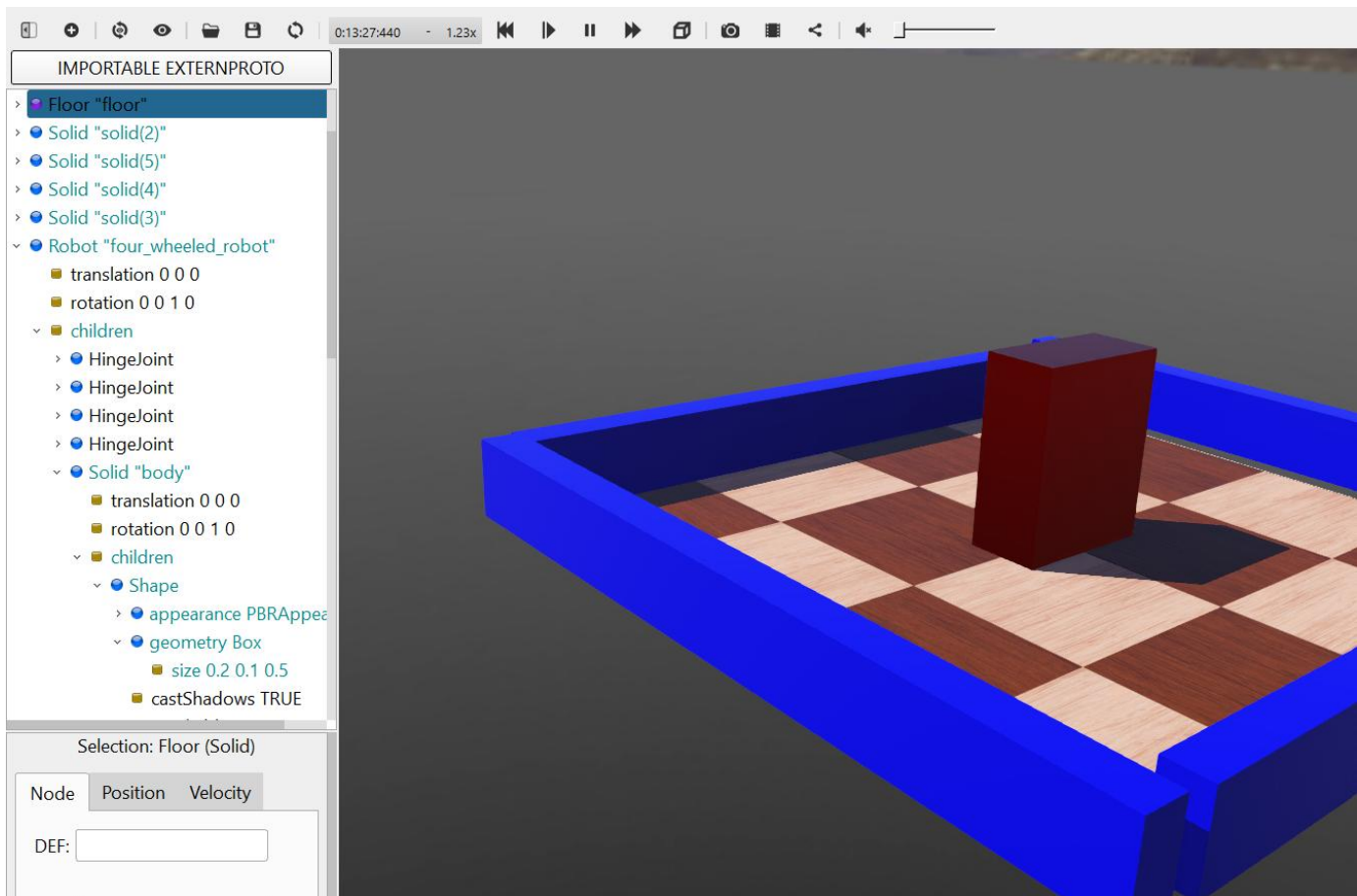
Después añadimos dos sensores de distancia en la parte frontal, orientados con un ángulo específico, y les dimos forma de cubo azul. Con esto, el robot quedó completo en su estructura física y sensorial.

Finalmente programamos un controlador que actuaba sobre los motores de las ruedas y usaba los sensores para detectar obstáculos. El robot avanzaba y, al encontrar un obstáculo, giraba para evitarlo.

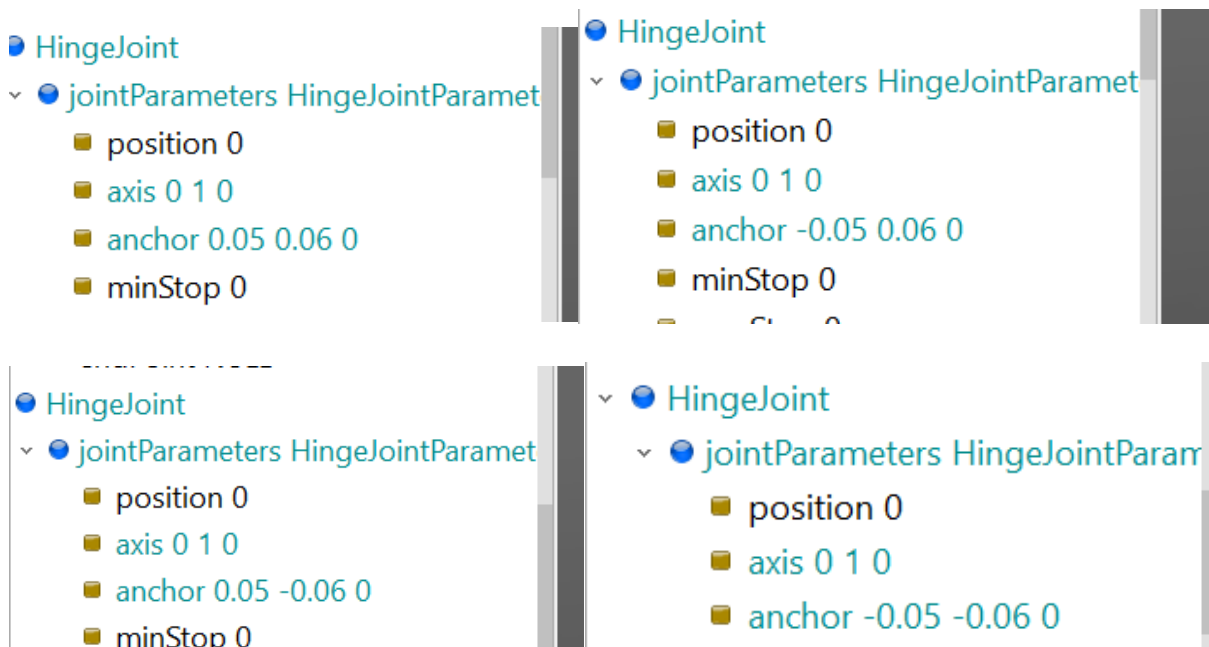
### 1 Creación del archivo 4\_wheeled\_robot



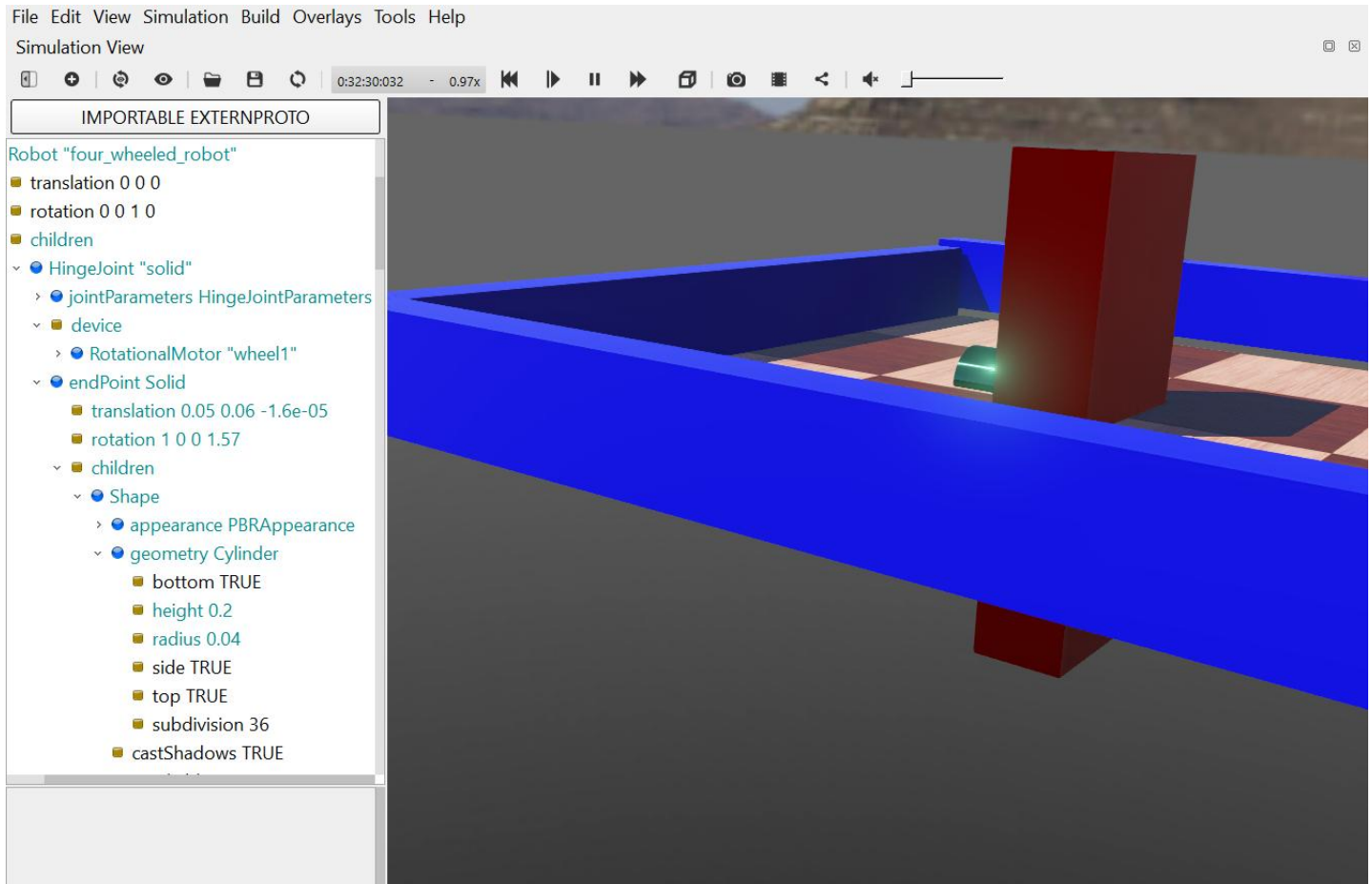
### 2. Creación de la estructura principal del robot de cuatro ruedas.



### 3. Configuración de la articulación de las ruedas y sus ejes de rotación.

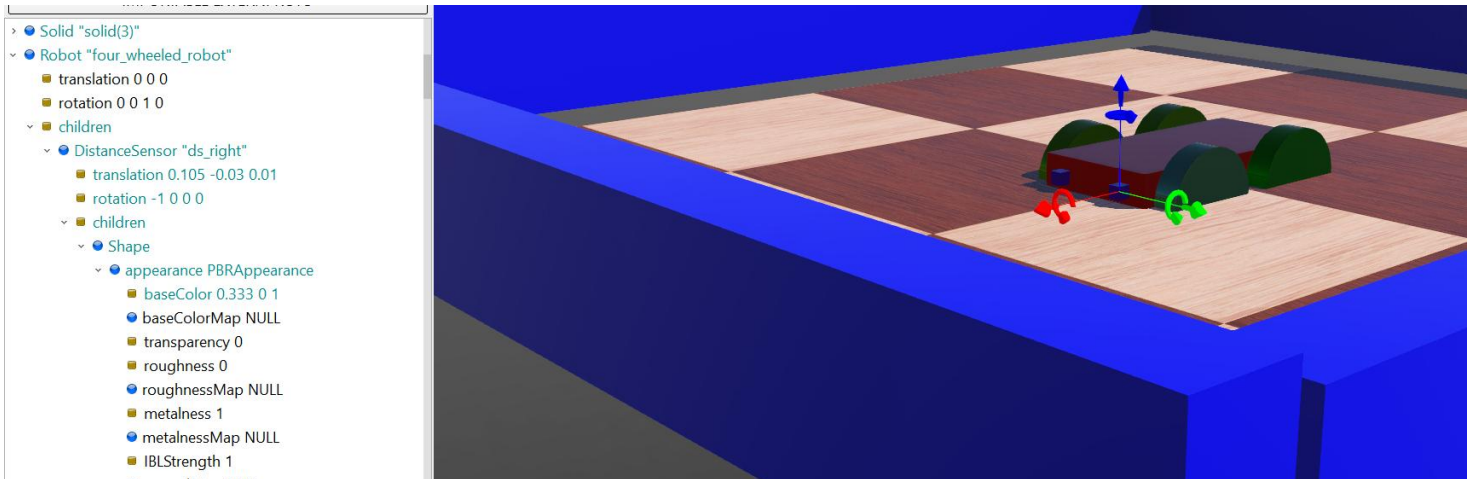


#### 4. Construcción de las cuatro ruedas, motores y elementos físicos del robot.



#### 5. Incorporación y orientación de los dos sensores de distancia en la parte frontal del robot.

- ▼ HingeJoint
  - ▼ jointParameters HingeJointParam
    - position 0
    - axis 0 1 0
    - anchor -0.05 -0.06 0



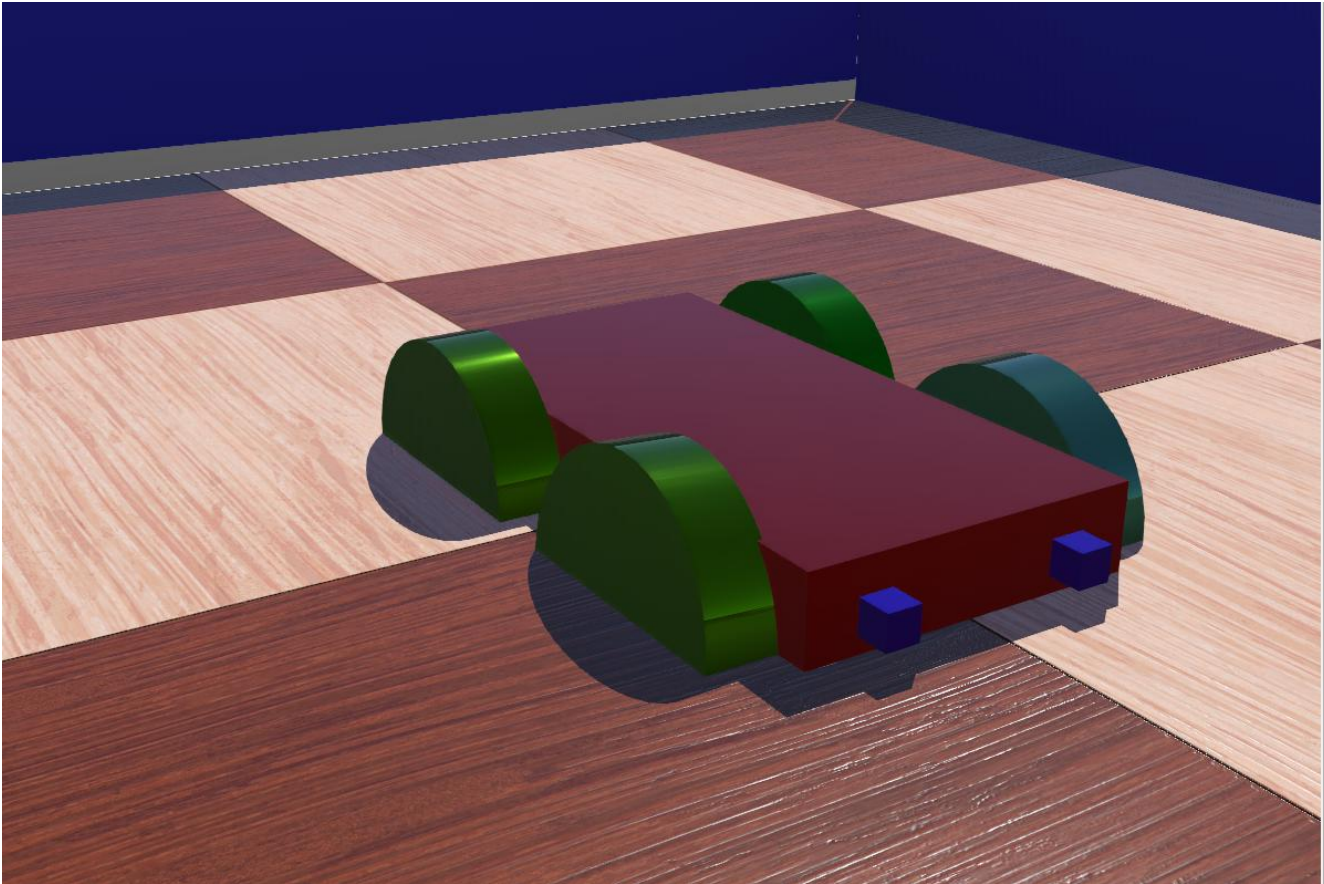
## 6. Implementación del controlador para avanzar y evitar obstáculos mediante sensores de distancia.

File Edit View Simulation Build Overlays Tools Help  
 C:\Users\abyhd\Downloads\moose\controllers\four\_wheeled\_collision\_avoidance\four\_wheeled\_collision\_avoidance.c

four\_wheeled\_collision\_avoidance.c

```
1#include <webots/distance_sensor.h>
2#include <webots/motor.h>
3#include <webots/robot.h>
4
5#define TIME_STEP 64
6
7int main(int argc, char **argv) {
8  wb_robot_init();
9  int i;
10  bool avoid_obstacle_counter = 0;
11  WbDeviceTag ds[2];
12  char ds_names[2][10] = {"ds_left", "ds_right"};
13  for (i = 0; i < 2; i++) {
14    ds[i] = wb_robot_get_device(ds_names[i]);
15    wb_distance_sensor_enable(ds[i], TIME_STEP);
16  }
17  WbDeviceTag wheels[4];
18  char wheels_names[4][8] = {"wheel1", "wheel2", "wheel3", "wheel4"};
19  for (i = 0; i < 4; i++) {
20    wheels[i] = wb_robot_get_device(wheels_names[i]);
21    wb_motor_set_position(wheels[i], INFINITY);
22  }
23  while (wb_robot_step(TIME_STEP) != -1) {
24    double left_speed = 1.0;
25    double right_speed = 1.0;
26    if (avoid_obstacle_counter > 0) {
27      avoid_obstacle_counter--;
28      left_speed = 1.0;
29      right_speed = -1.0;
30    } else { // read sensors
31      double ds_values[2];
32      for (i = 0; i < 2; i++)
33        ds_values[i] = wb_distance_sensor_get_value(ds[i]);
34      if (ds_values[0] < 950.0 || ds_values[1] < 950.0)
35        avoid_obstacle_counter = 100;
```

Vista 3D del robot de las 4 ruedas.



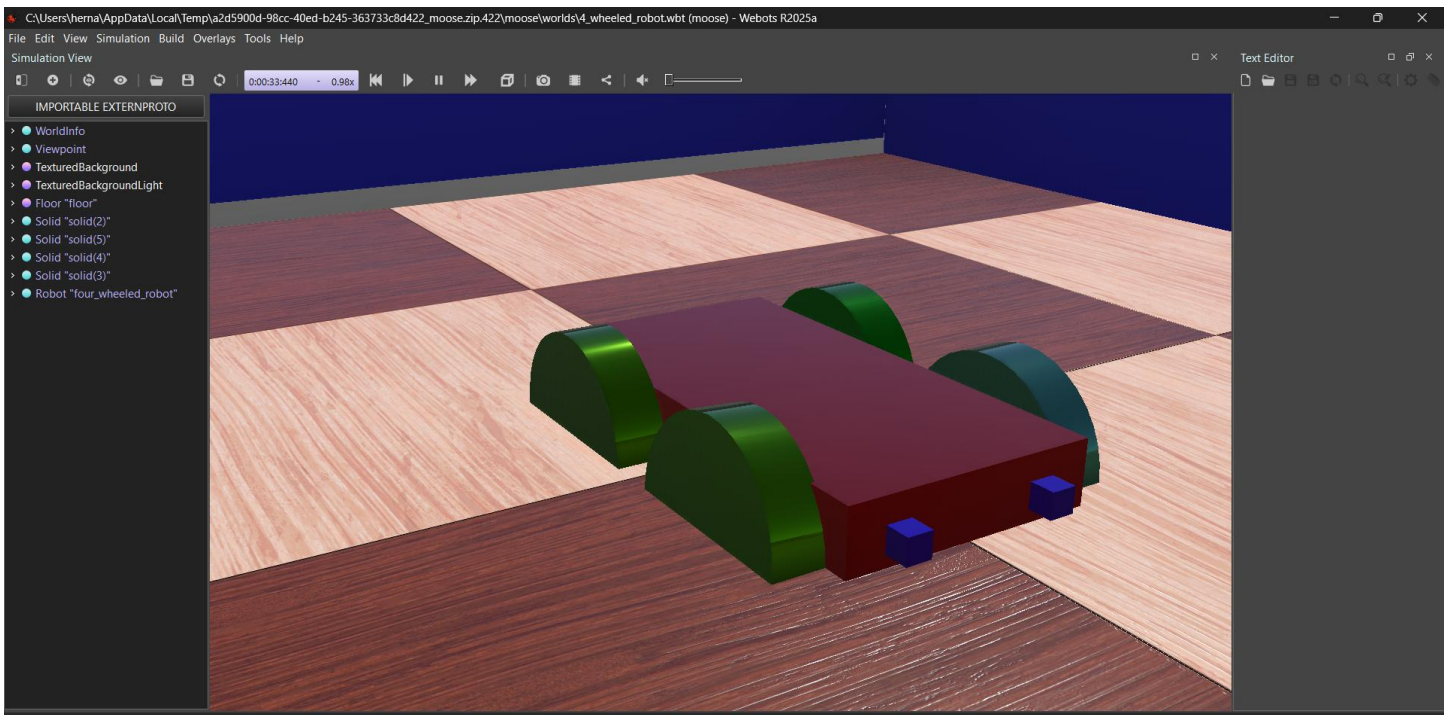
## Tutorial #7

En este séptimo tutorial aprendimos a crear nuestro primer archivo PROTO para el robot de cuatro ruedas. Lo que hicimos fue tomar la definición completa del robot que estaba dentro del archivo del mundo y moverla a un archivo independiente llamado *FourWheelsRobot.proto*.

Primero copiamos la estructura básica del PROTO y pegamos el bloque del nodo Robot dentro del cuerpo. Luego abrimos el mundo y añadimos el nuevo nodo PROTO, lo que nos permitió reutilizar el robot sin necesidad de copiar y pegar su definición.

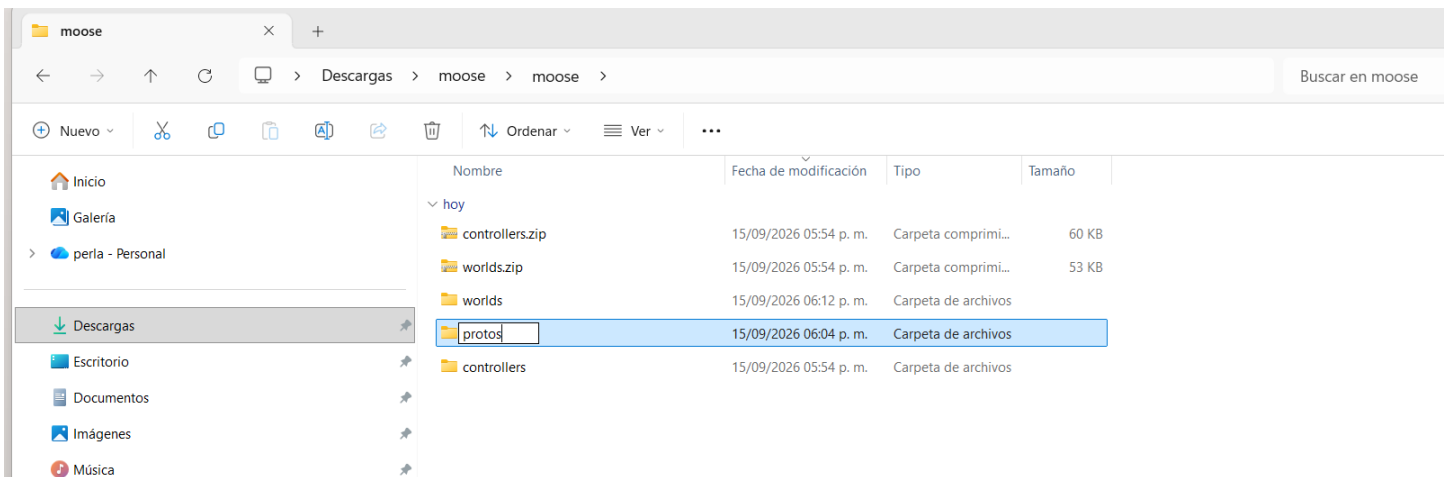
Después sumamos campos abiertos como translation, rotation y bodyMass, y los vinculamos con los internos del robot usando la palabra clave IS. De esta forma, pudimos modificar la posición, orientación y masa del robot directamente desde la interfaz de Webots.

Lo primero que se hizo fue buscar el mundo creado en el tutorial anterior, pues este será nuestra base para el tutorial 7.

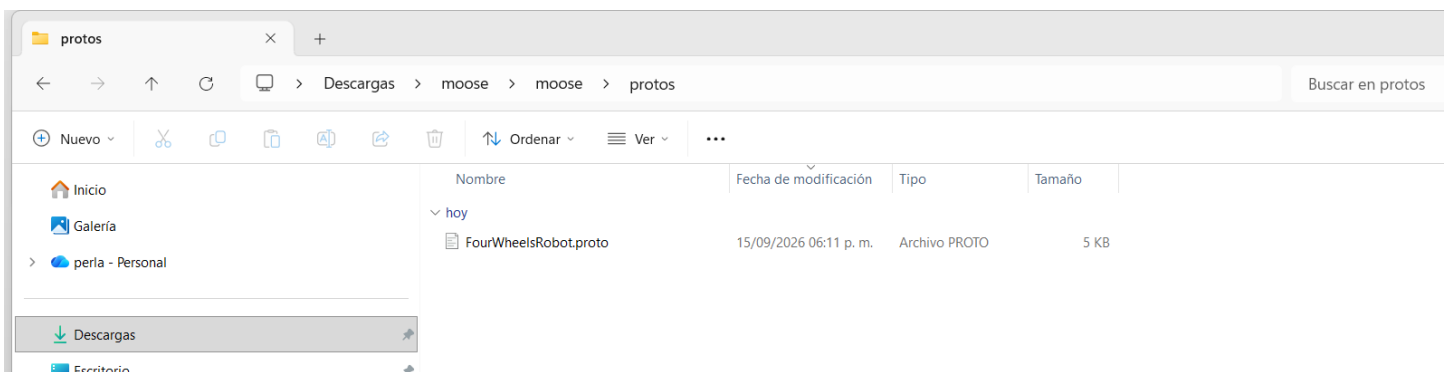


## PRACTICA1:

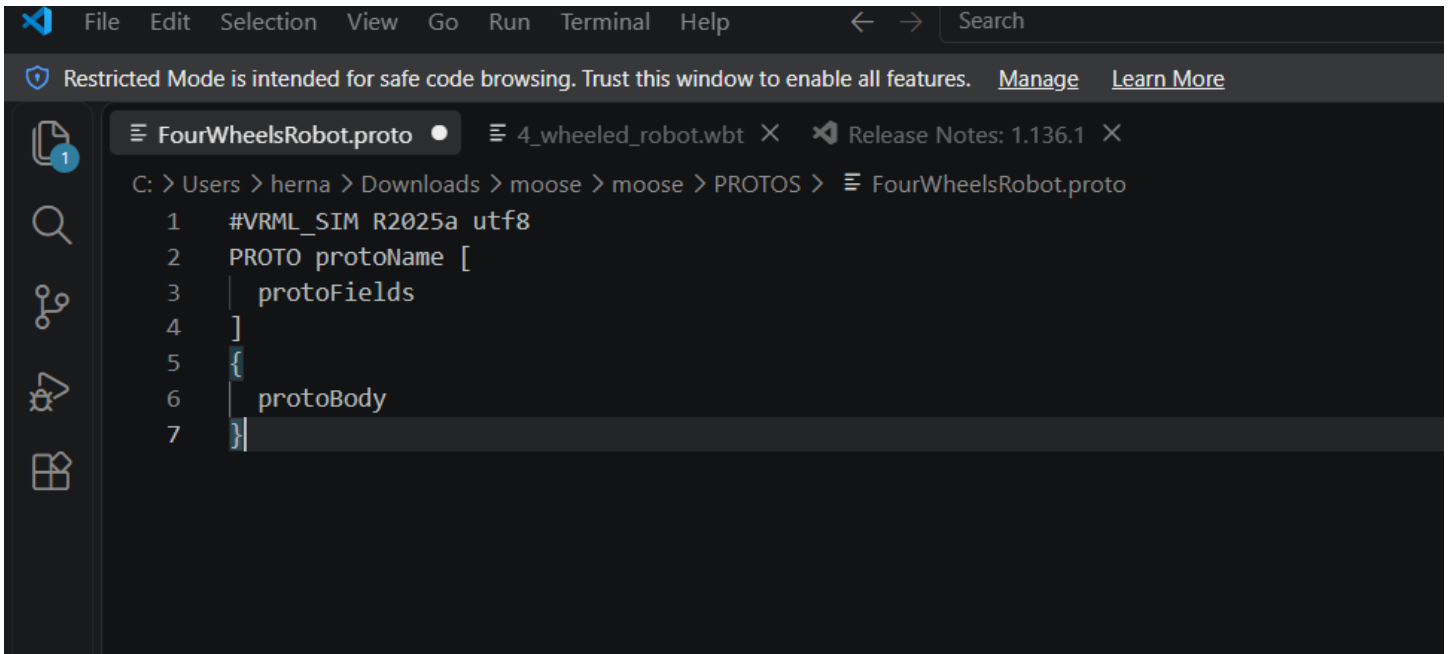
- 1- Primero creamos una carpeta protos en el mismo nivel de la carpeta worlds que como su nombre lo indica contiene el mundo en el que trabajamos anteriormente.



- 2- El siguiente paso es crear un archivo .proto en la carpeta que creamos que contendrá el código del 2do robot.



3- Escribimos el código base que debe tener un archivo proto.

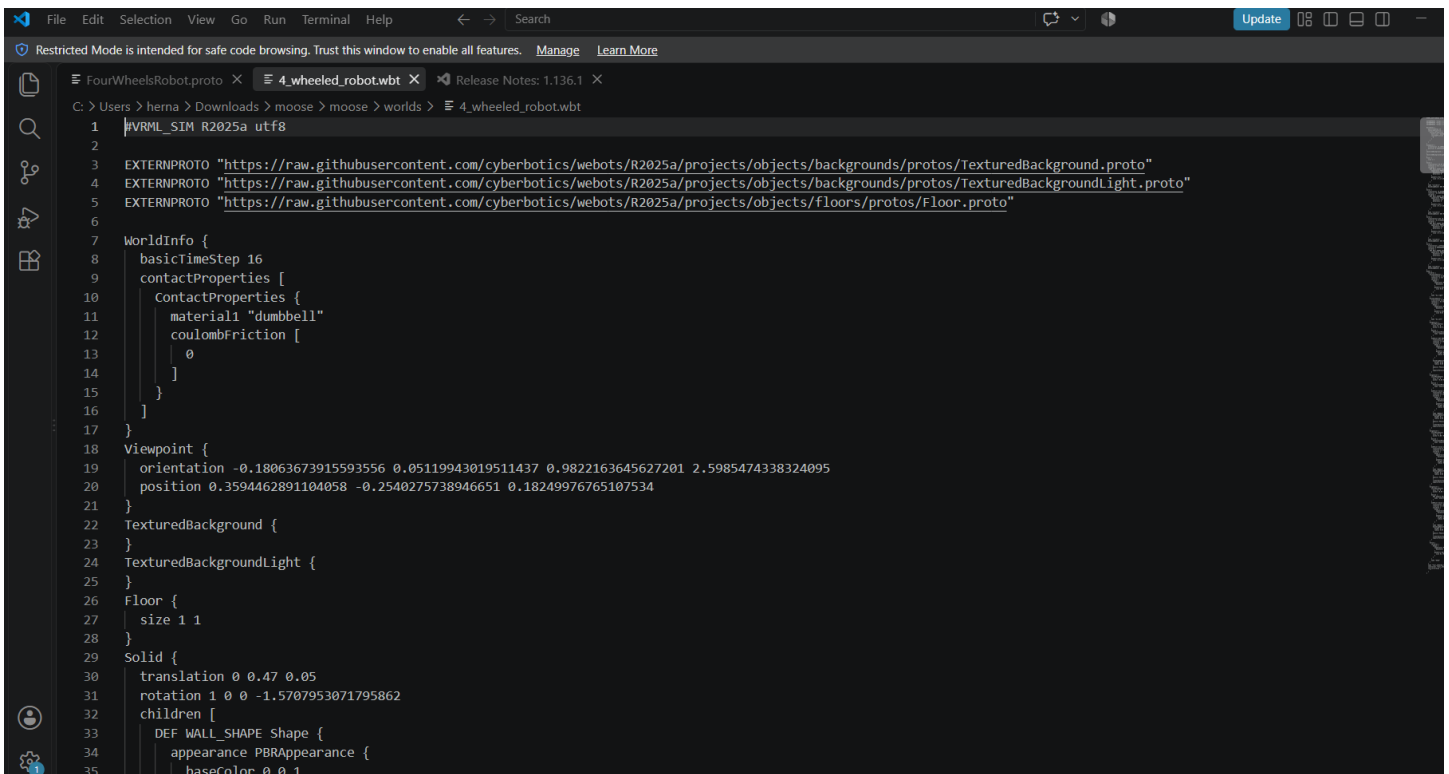


The screenshot shows the Visual Studio Code editor with the file `FourWheelsRobot.proto` open. The editor is in a dark theme. The menu bar at the top includes File, Edit, Selection, View, Go, Run, Terminal, and Help. A status bar at the bottom indicates 'Restricted Mode is intended for safe code browsing. Trust this window to enable all features.' with links to 'Manage' and 'Learn More'. The file explorer on the left shows the project structure. The main editor area displays the following code:

```
1  #VRML_SIM R2025a utf8
2  PROTO protoName [
3    | protoFields
4    | ]
5    {
6    | protoBody
7    | }
```

## PRACTICA2

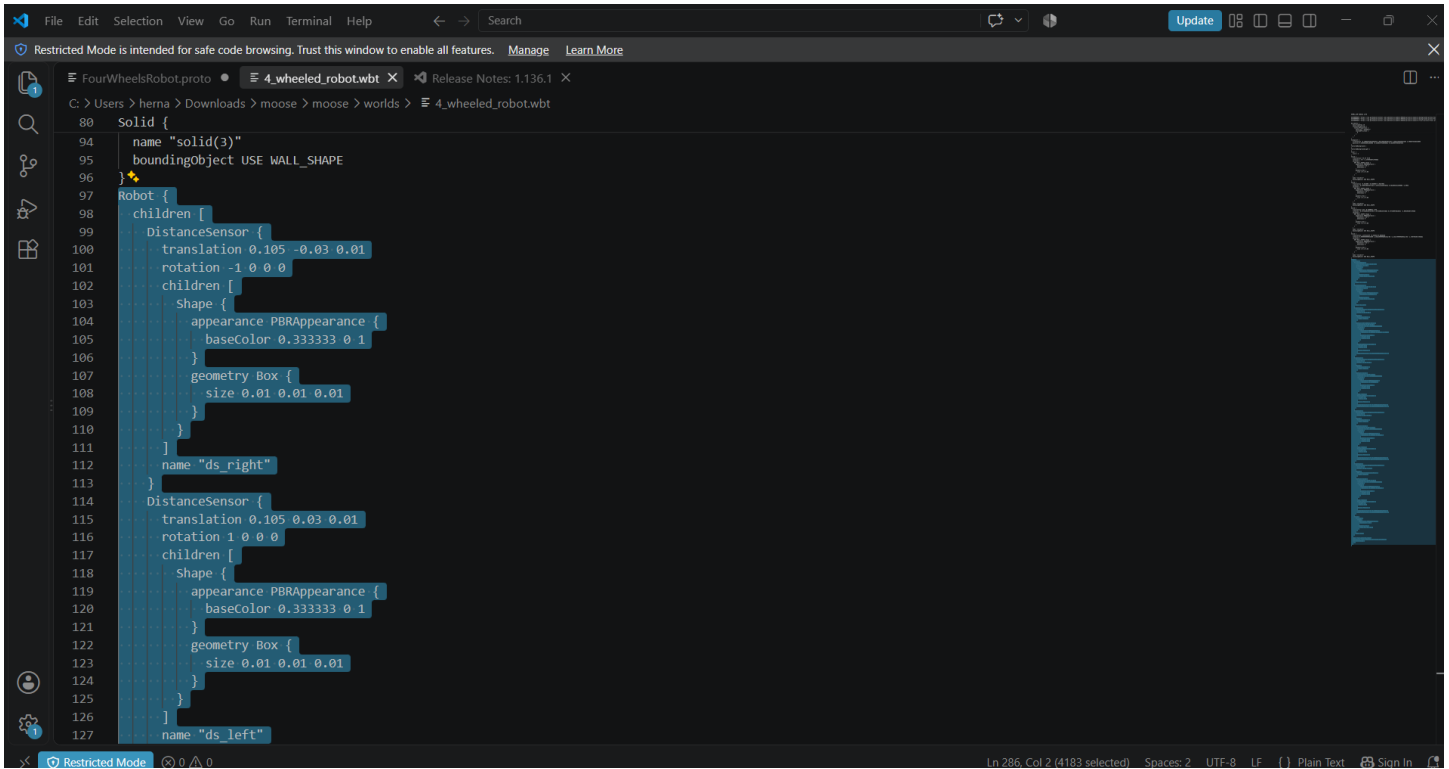
1- Abrimos el código del mundo en el editor de nuestra preferencia, en este caso vs.



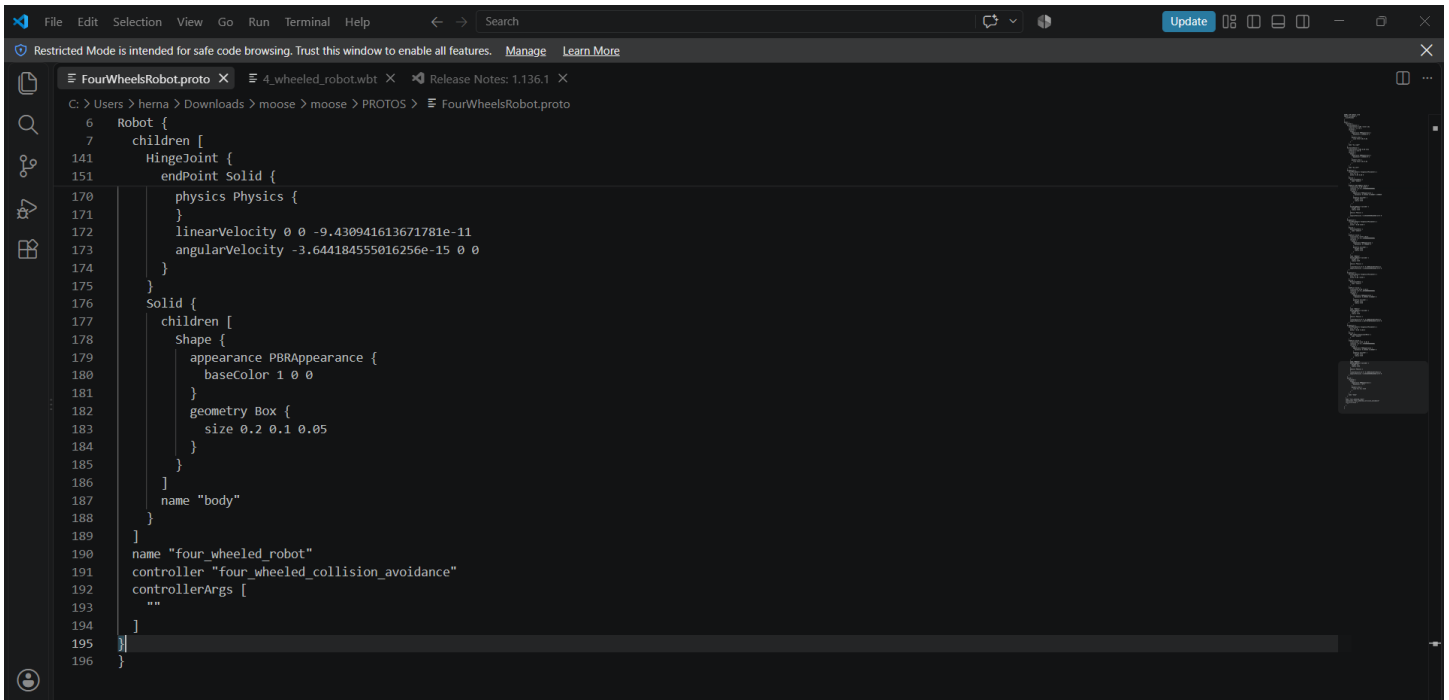
The screenshot shows the Visual Studio Code editor with the file `4_wheeled_robot.wbt` open. The editor is in a dark theme. The menu bar at the top includes File, Edit, Selection, View, Go, Run, Terminal, and Help. A status bar at the bottom indicates 'Restricted Mode is intended for safe code browsing. Trust this window to enable all features.' with links to 'Manage' and 'Learn More'. The file explorer on the left shows the project structure. The main editor area displays the following code:

```
1  #VRML_SIM R2025a utf8
2
3  EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2025a/projects/objects/backgrounds/protos/TexturedBackground.proto"
4  EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2025a/projects/objects/backgrounds/protos/TexturedBackgroundLight.proto"
5  EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2025a/projects/objects/floors/protos/Floor.proto"
6
7  WorldInfo {
8    | basicTimeStep 16
9    | contactProperties [
10     | | ContactProperties {
11     | | | material1 "dumbbell"
12     | | | coulombFriction [
13     | | | | 0
14     | | | ]
15     | | }
16     | ]
17   }
18   Viewpoint {
19     | orientation -0.18063673915593556 0.05119943019511437 0.9822163645627201 2.5985474338324095
20     | position 0.3594462891104058 -0.2540275738946651 0.18249976765107534
21   }
22   TexturedBackground {
23   }
24   TexturedBackgroundLight {
25   }
26   Floor {
27     | size 1 1
28   }
29   Solid {
30     | translation 0 0.47 0.05
31     | rotation 1 0 0 -1.5707953071795862
32     | children [
33     | | DEF WALL_SHAPE Shape {
34     | | | appearance PBRAppearance {
35     | | | | baseColor 0 0 1
```

2- En este paso copiaremos el código contenido del robot de archivo del mundo al archivo de proto.



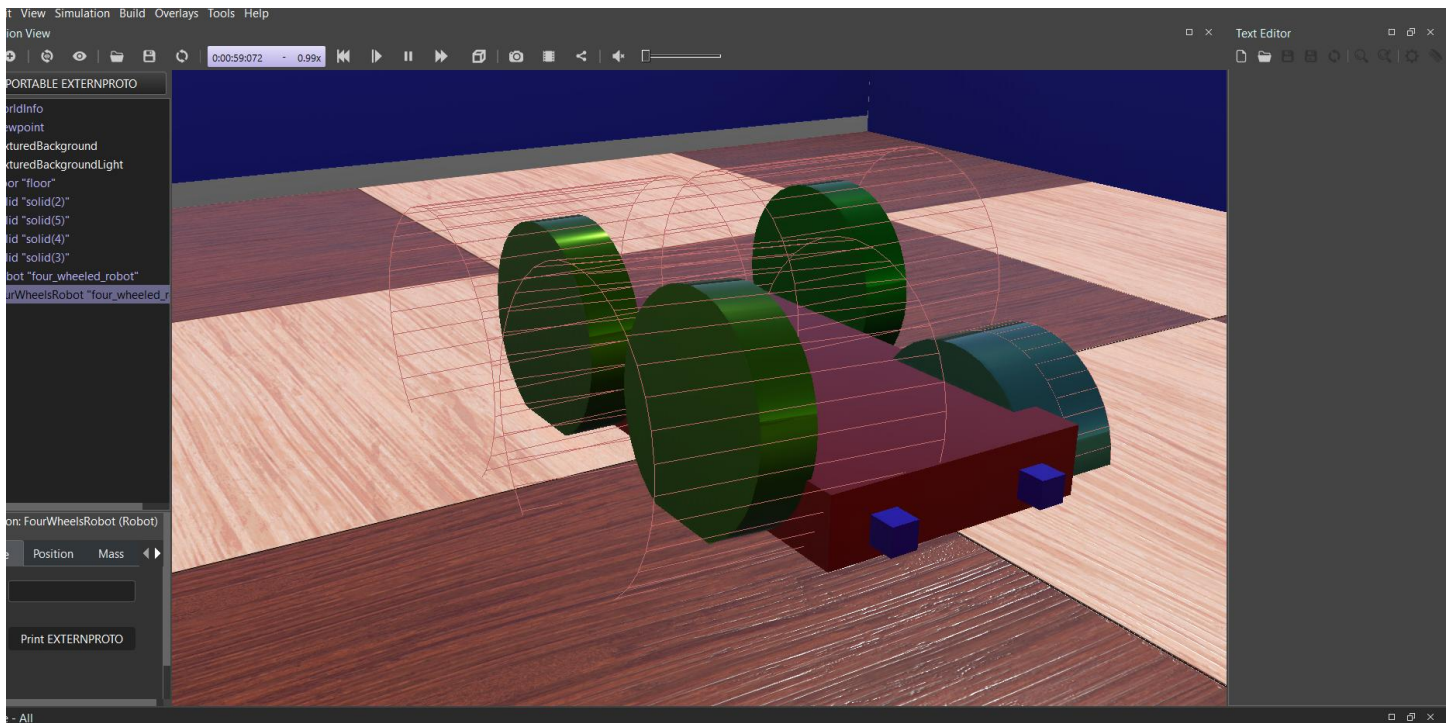
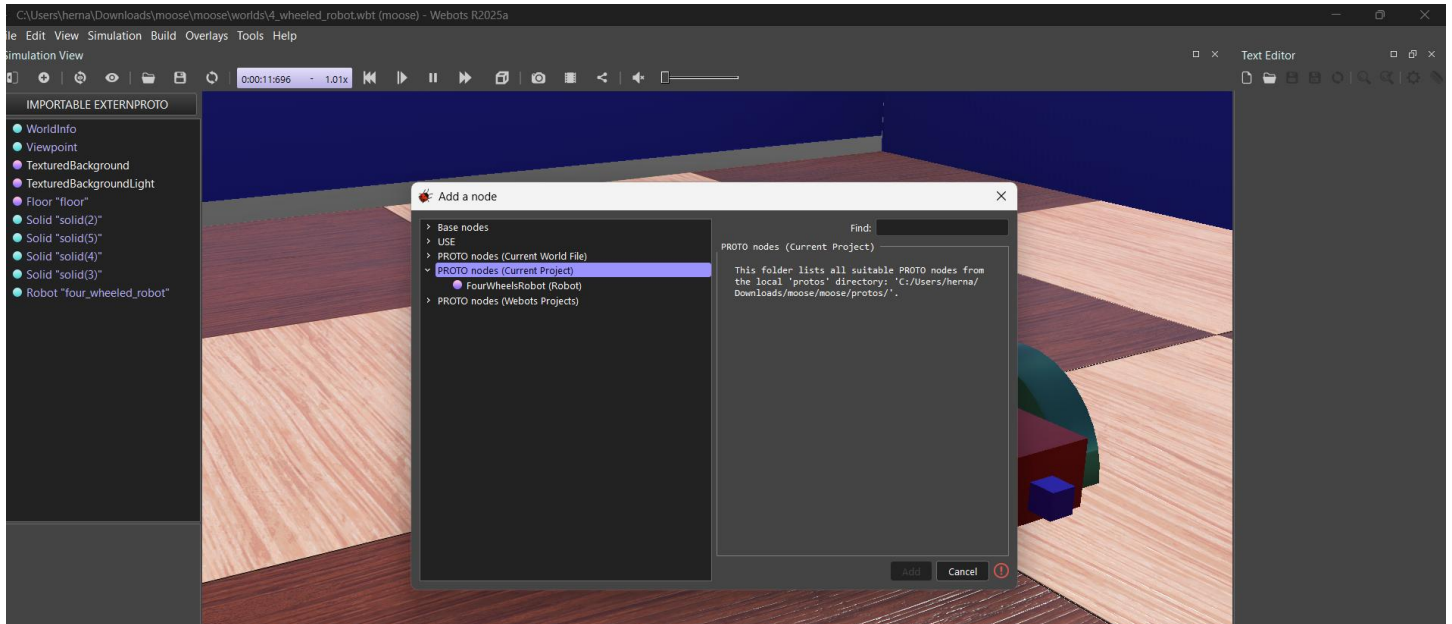
```
80 Solid {
81   name "solid(3)"
82   boundingObject USE WALL_SHAPE
83 }
84
85 Robot {
86   children [
87     DistanceSensor {
88       translation 0.105 -0.03 0.01
89       rotation -1 0 0 0
90       children [
91         Shape {
92           appearance PBRAppearance {
93             baseColor 0.333333 0 1
94           }
95           geometry Box {
96             size 0.01 0.01 0.01
97           }
98         }
99       ]
100     }
101     name "ds_right"
102     DistanceSensor {
103       translation 0.105 0.03 0.01
104       rotation 1 0 0 0
105       children [
106         Shape {
107           appearance PBRAppearance {
108             baseColor 0.333333 0 1
109           }
110           geometry Box {
111             size 0.01 0.01 0.01
112           }
113         }
114       ]
115     }
116     name "ds_left"
117   ]
118 }
```



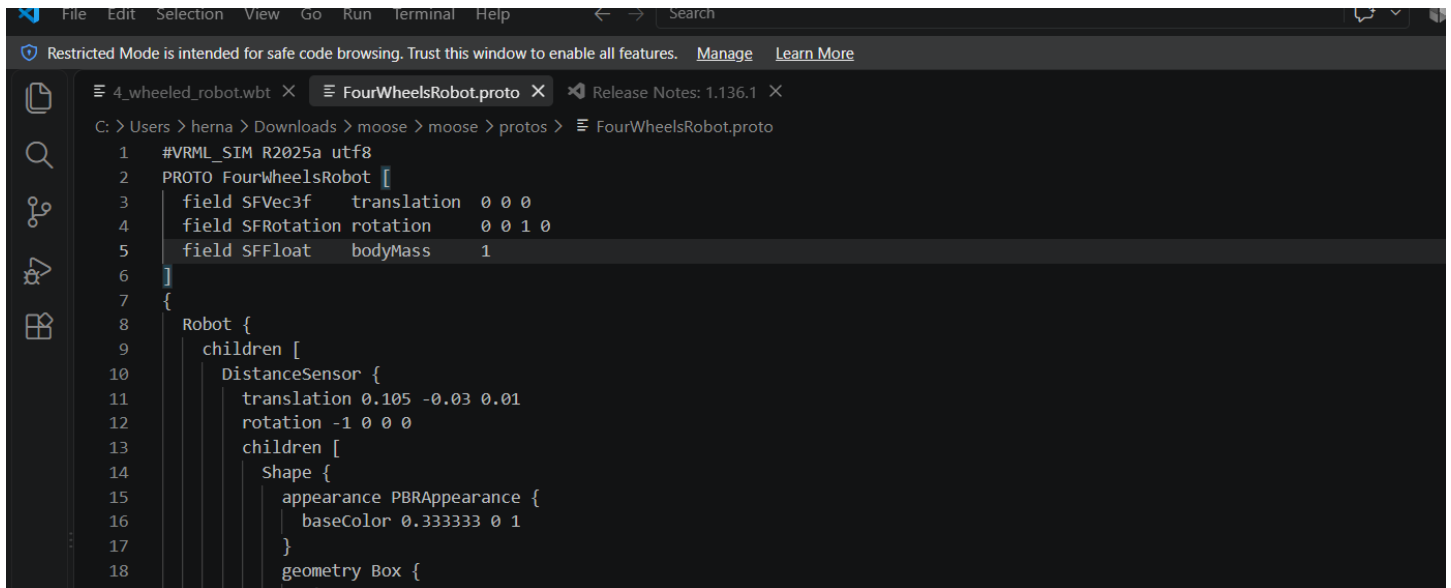
```
6 Robot {
7   children [
8     HingeJoint {
9       endPoint Solid {
10         physics Physics {
11           linearVelocity 0 0 -9.430941613671781e-11
12           angularVelocity -3.644184555016256e-15 0 0
13         }
14       }
15       Solid {
16         children [
17           Shape {
18             appearance PBRAppearance {
19               baseColor 1 0 0
20             }
21             geometry Box {
22               size 0.2 0.1 0.05
23             }
24           }
25         ]
26         name "body"
27       }
28     }
29   ]
30   name "four_wheeled_robot"
31   controller "four_wheeled_collision_avoidance"
32   controllerArgs [
33     ""
34   ]
35 }
36 }
```

## PRACTICA3

1- Con nuestro codigo terminado, cargaremos el proto al mundo en webots.



## PRACTICA4

A screenshot of a code editor window. The title bar shows 'File Edit Selection View Go Run Terminal Help' and a search bar. Below the title bar, a message states: 'Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More'. The editor has three tabs: '4\_wheeled\_robot.wbt', 'FourWheelsRobot.proto', and 'Release Notes: 1.136.1'. The active tab is 'FourWheelsRobot.proto'. The file path is 'C: > Users > herna > Downloads > moose > moose > protos > FourWheelsRobot.proto'. The code is a .proto file for a robot. It starts with a VRML\_SIM header, followed by a PROTO definition for FourWheelsRobot. The definition includes fields for translation, rotation, and bodyMass. It then defines a Robot struct with children: a DistanceSensor (with translation, rotation, and a Shape child) and a geometry Box. The Shape child has a PBRAppearance with a baseColor. The code is as follows:

```
1 #VRML_SIM R2025a utf8
2 PROTO FourWheelsRobot [
3     field SFVec3f translation 0 0 0
4     field SFRotation rotation 0 0 1 0
5     field SFFloat bodyMass 1
6 ]
7 {
8     Robot {
9         children [
10             DistanceSensor {
11                 translation 0.105 -0.03 0.01
12                 rotation -1 0 0 0
13                 children [
14                     Shape {
15                         appearance PBRAppearance {
16                             baseColor 0.333333 0 1
17                         }
18                     geometry Box {
```

CODIGO FINAL DEL ARCHIVO .proto:

```
#VRML_SIM R2025a utf8
```

```
PROTO FourWheelsRobot [
```

```
    field SFVec3f translation 0 0 0
```

```
    field SFRotation rotation 0 0 1 0
```

```
    field SFFloat bodyMass 1
```

```
]
```

```
{
```

```
    Robot {
```

```
        translation IS translation
```

```
        rotation IS rotation
```

```
        children [
```

```
            DistanceSensor {
```

```
                translation 0.105 -0.03 0.01
```

```
                rotation -1 0 0 0
```

```
                children [
```

```
                    Shape {
```

```
                        appearance PBRAppearance {
```

```
                            baseColor 0.333333 0 1
```

```
                        }
```

```
                    geometry Box {
```

```

        size 0.01 0.01 0.01
    }
}
]
name "ds_right"
}
DistanceSensor {
    translation 0.105 0.03 0.01
    rotation 1 0 0 0
    children [
        Shape {
            appearance PBRAppearance {
                baseColor 0.333333 0 1
            }
            geometry Box {
                size 0.01 0.01 0.01
            }
        }
    ]
    name "ds_left"
}
HingeJoint {
    jointParameters HingeJointParameters {
        axis 0 1 0
        anchor 0.05 0.06 0
    }
    device [
        RotationalMotor {
            name "wheel1"
        }
    ]
    endPoint DEF WHEEL1 Solid {

```

```
translation 0.05 0.06 0
rotation 1 0 0 1.5708
children [
  Shape{
    appearance PBRAppearance {
      baseColor 0.333333 0.666667 0.498039
    }
    geometry Cylinder {
      height 0.02
      radius 0.04
    }
  }
]
boundingObject Cylinder {
  height 0.02
  radius 0.04
}
physics Physics {
}
}
HingeJoint {
  jointParameters HingeJointParameters {
    axis 0 1 0
    anchor -0.05 0.06 0
  }
  device [
    RotationalMotor {
      name "wheel2"
    }
  ]
  endPoint Solid {
```

```
translation -0.05 0.06 0
rotation 1 0 0 1.5708
children [
  Shape{
    appearance PBRAppearance {
      baseColor 0 0.666667 0
    }
    geometry Cylinder {
      height 0.02
      radius 0.04
    }
  }
]
name "WHEEL2"
boundingObject Cylinder {
  height 0.1
  radius 0.05
}
physics Physics {
}
}
HingeJoint {
  jointParameters HingeJointParameters {
    axis 0 1 0
    anchor 0.05 -0.06 0
  }
  device [
    RotationalMotor {
      name "wheel3"
    }
  ]
}
```

```

endPoint Solid {
  translation 0.05 -0.06 0
  rotation 1 0 0 1.5078
  children [
    Shape {
      appearance PBRAppearance {
        baseColor 0.333333 0.666667 0
      }
      geometry Cylinder {
        height 0.02
        radius 0.04
      }
    }
  ]
  name "WHEEL3"
  boundingObject Cylinder {
    height 0.1
    radius 0.05
  }
  physics Physics {
  }
}

HingeJoint {
  jointParameters HingeJointParameters {
    axis 0 1 0
    anchor -0.05 -0.06 0
  }
  device [
    DEF wheel4 RotationalMotor {
      name "wheel4"
    }
  ]
}

```

```

]
endPoint Solid {
  translation -0.05 -0.06 0
  rotation 1 0 0 1.5708
  children [
    Shape {
      appearance PBRAppearance {
        baseColor 0.333333 0.666667 0
      }
      geometry Cylinder {
        height 0.02
        radius 0.04
      }
    }
  ]
  name "WHEEL4"
  boundingObject Cylinder {
    height 0.1
    radius 0.05
  }
  physics Physics {
  }
}
}

Solid {
  children [
    Shape {
      appearance PBRAppearance {
        baseColor 1 0 0
      }
      geometry Box {
        size 0.2 0.1 0.05

```

```
    }  
  }  
]  
  name "BODY"  
}  
]  
boundingObject USE BODY  
physics Physics{  
  density -1  
  mass IS bodyMass  
}  
name "four_wheeled_robot"  
controller "four_wheeled_collision_avoidance"  
controllerArgs [  
  ""  
]  
}  
}
```

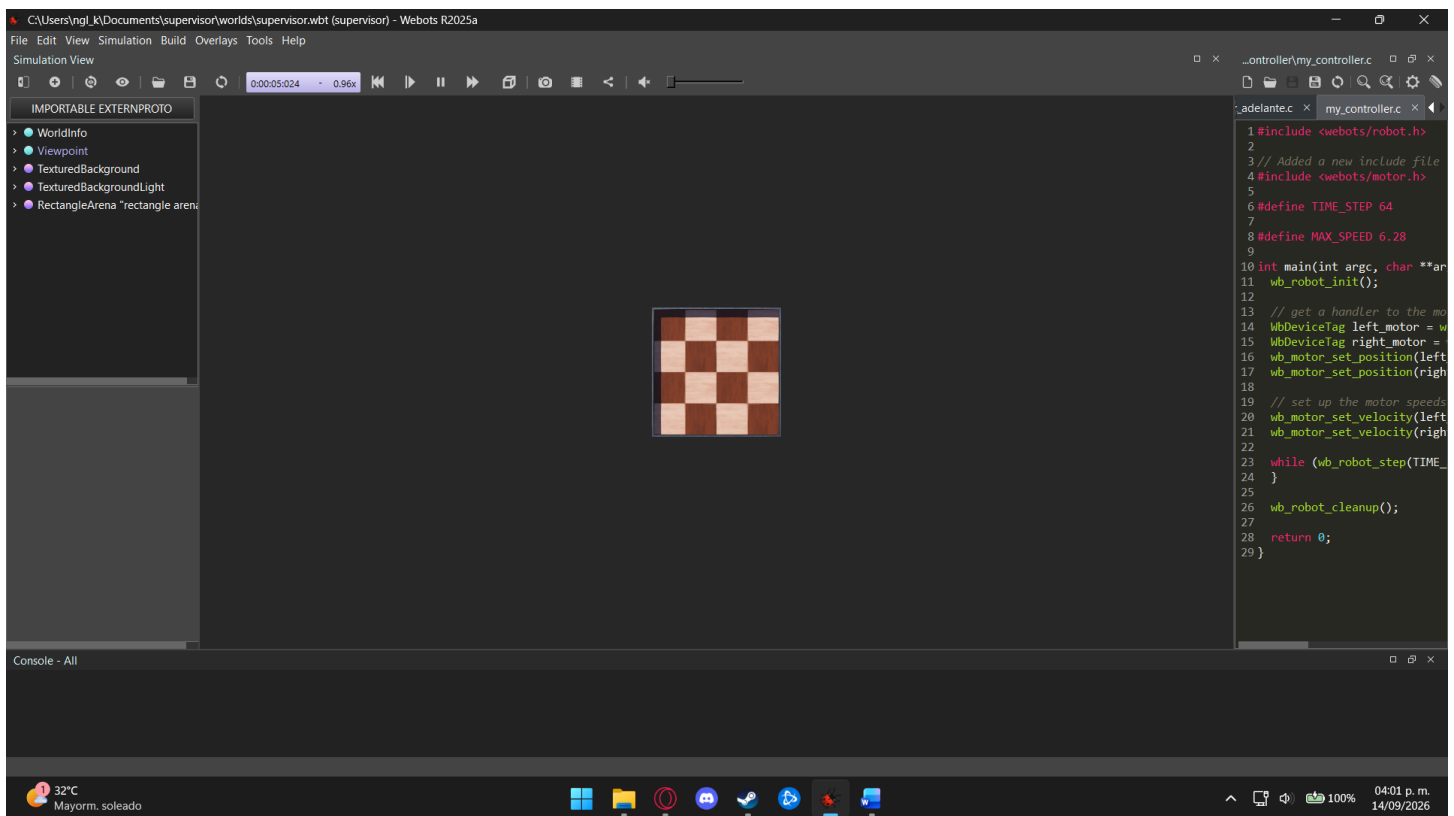
## Tutorial #8

En este octavo tutorial aprendimos a usar el Supervisor en Webots, que es básicamente un robot con poderes especiales para controlar y modificar el mundo.

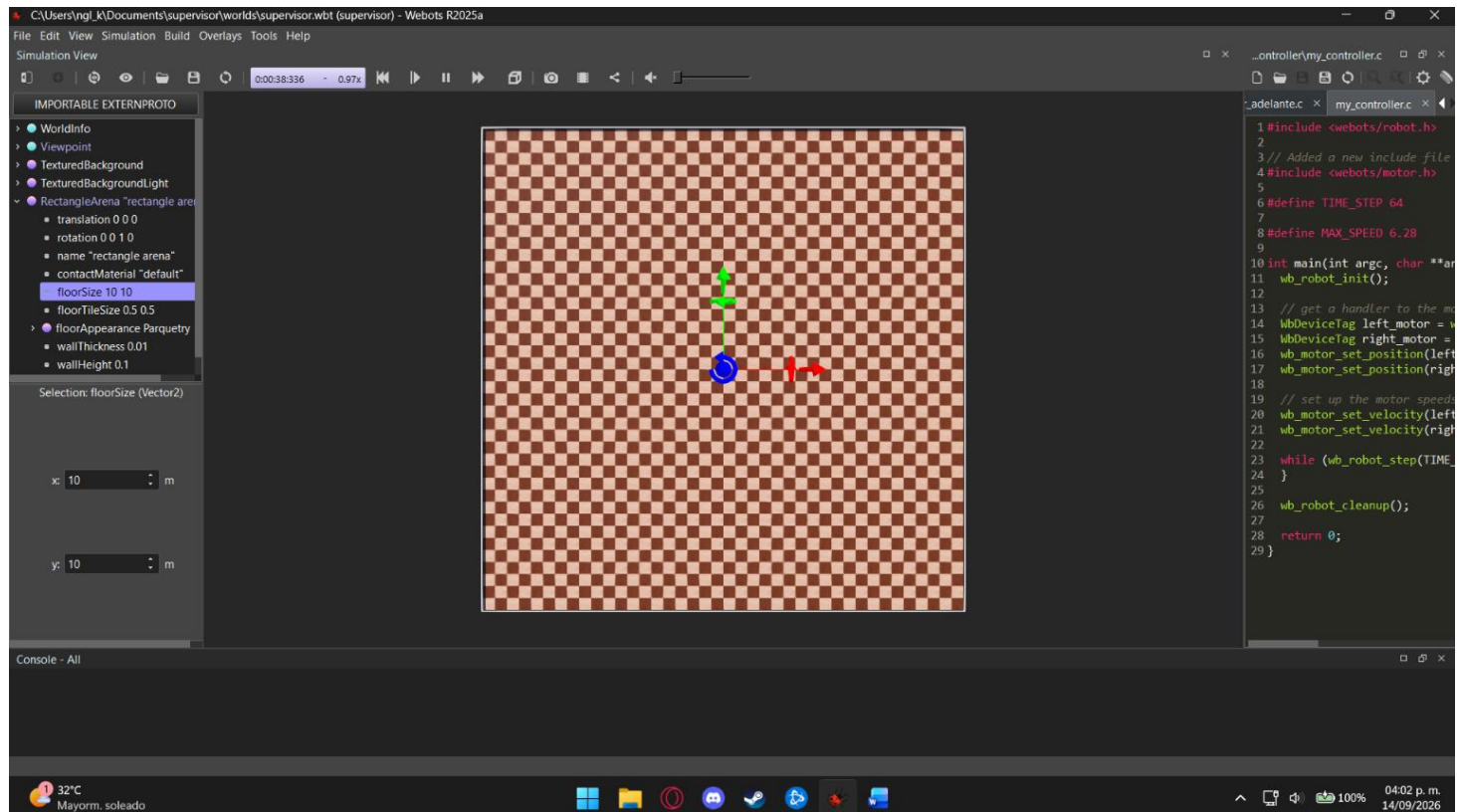
Primero configuramos un nuevo proyecto, añadimos una arena más grande y colocamos un robot BB-8, además de crear un nodo supervisor con su propio controlador. Luego programamos al supervisor para mover al BB-8 directamente a otra posición, ignorando la física, lo que nos mostró cómo puede “hacer trampas” en la simulación.

Después experimentamos con la eliminación y aparición de nodos: hicimos que el supervisor quitara al BB-8 y luego insertara un robot Nao en su lugar. También añadimos una pelota y usamos el supervisor para seguir su posición mientras caía, cambiando su color a rojo cuando tocaba el suelo.

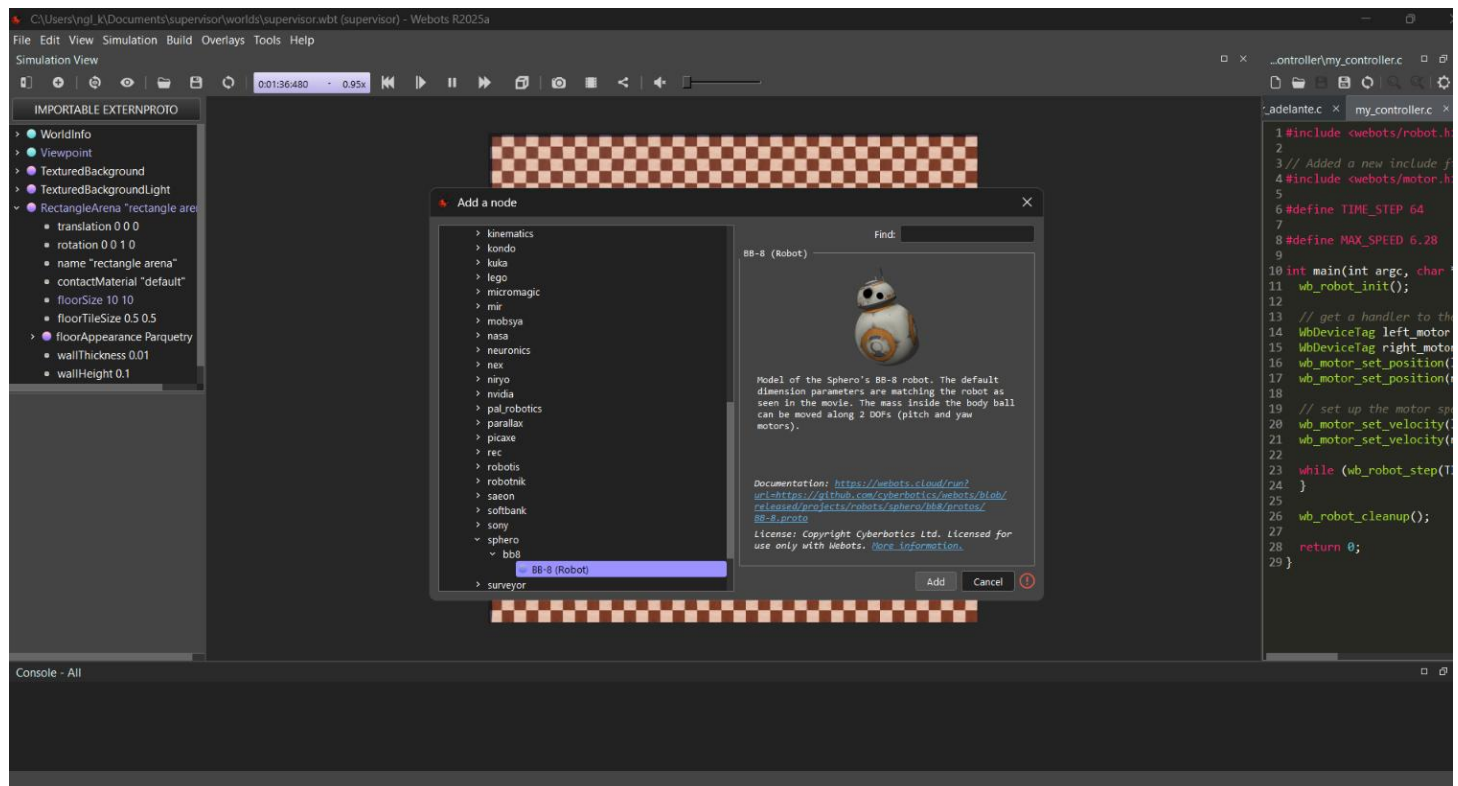
### 1. Creación del mundo



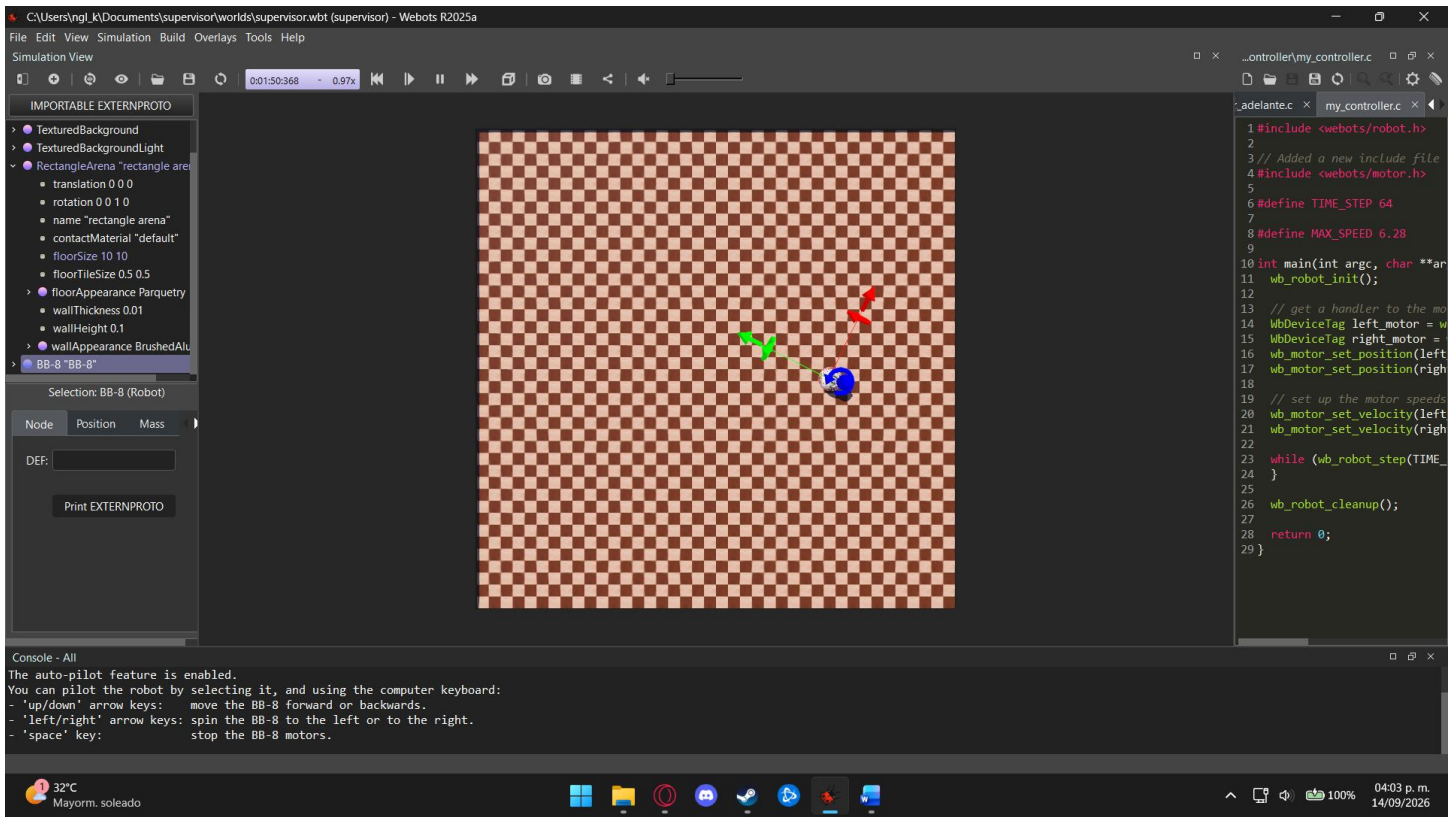
## 2.- Modificación del tamaño del suelo a 10 10



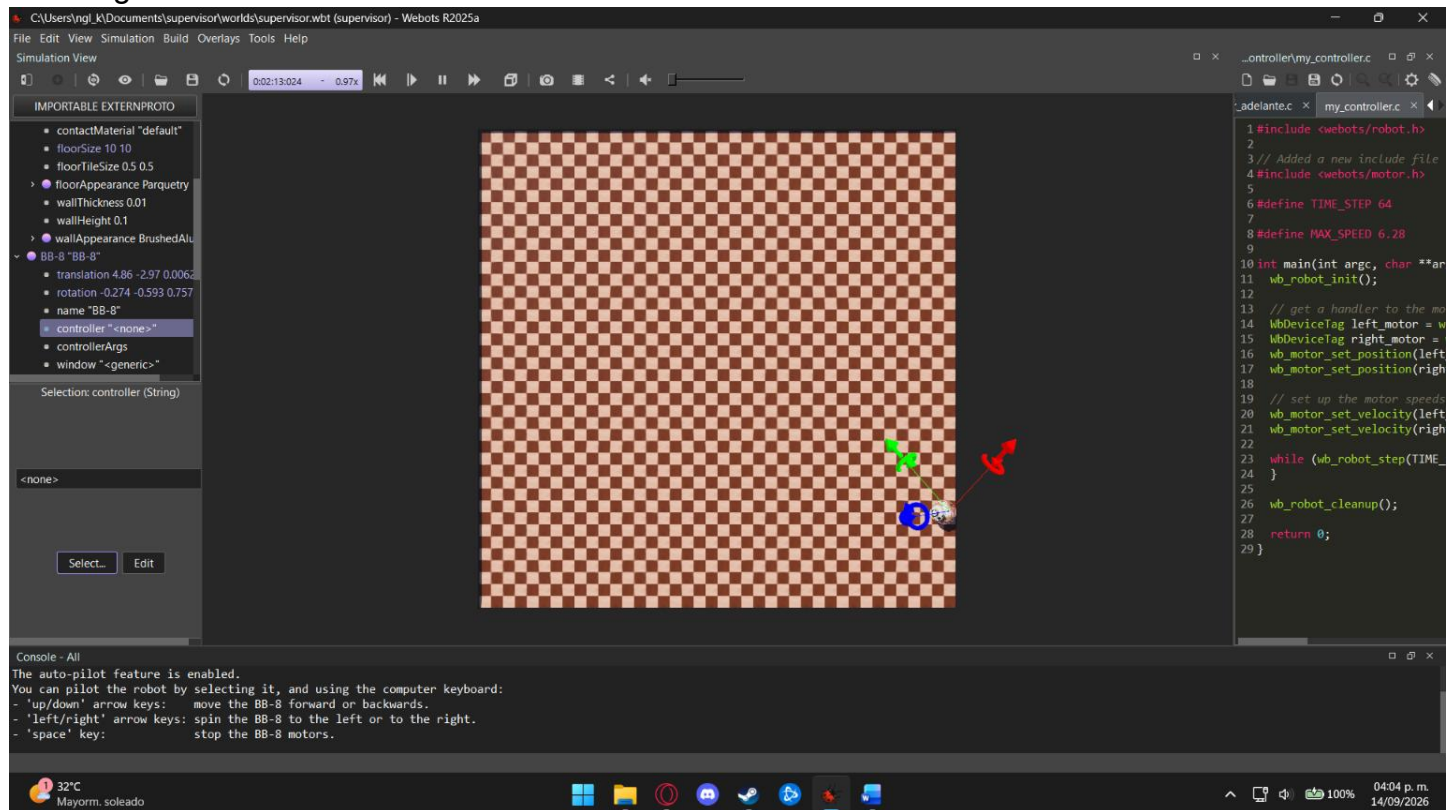
## 3.- Integración de robot BB-8 al mundo



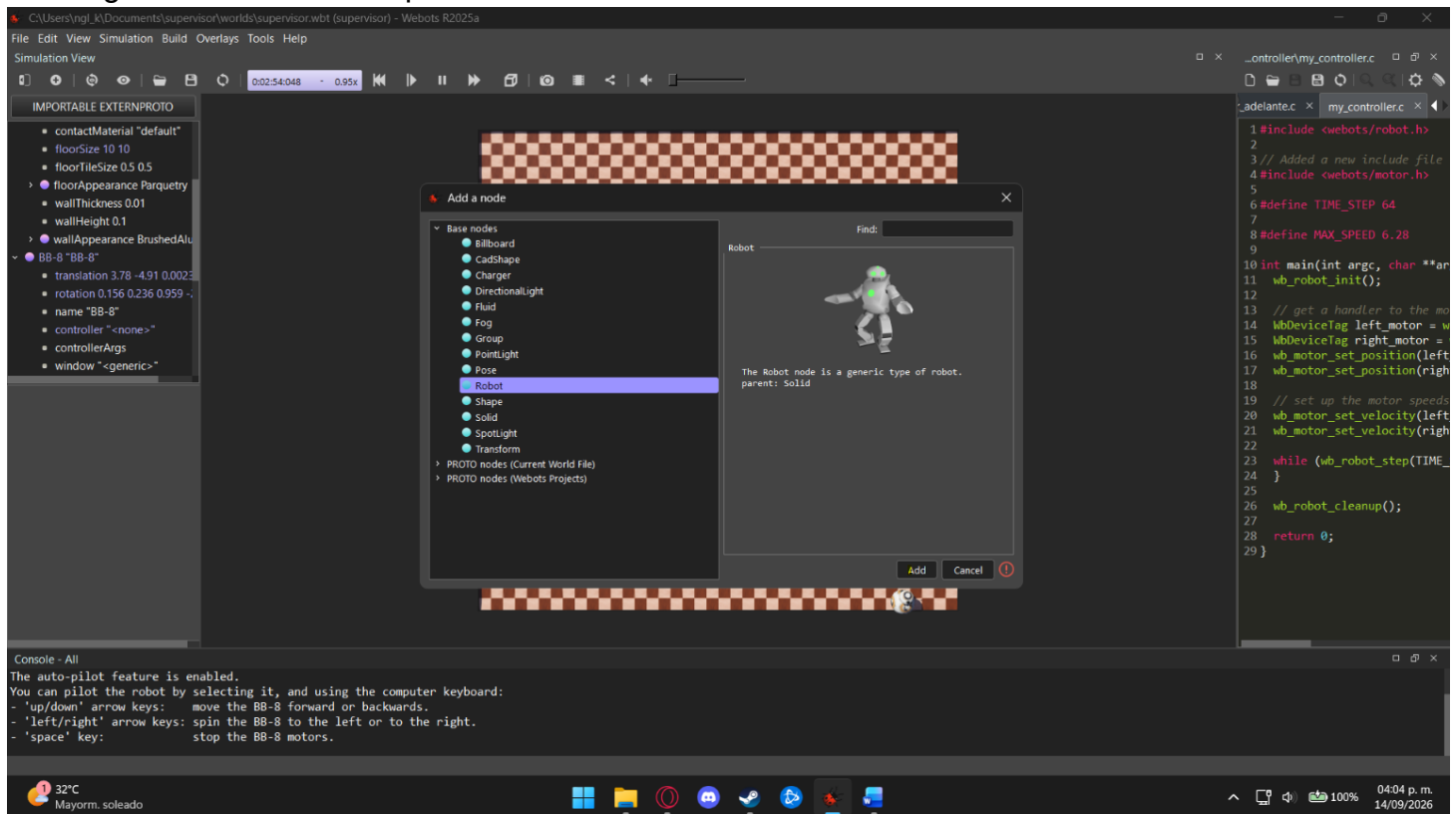
Robot BB-8



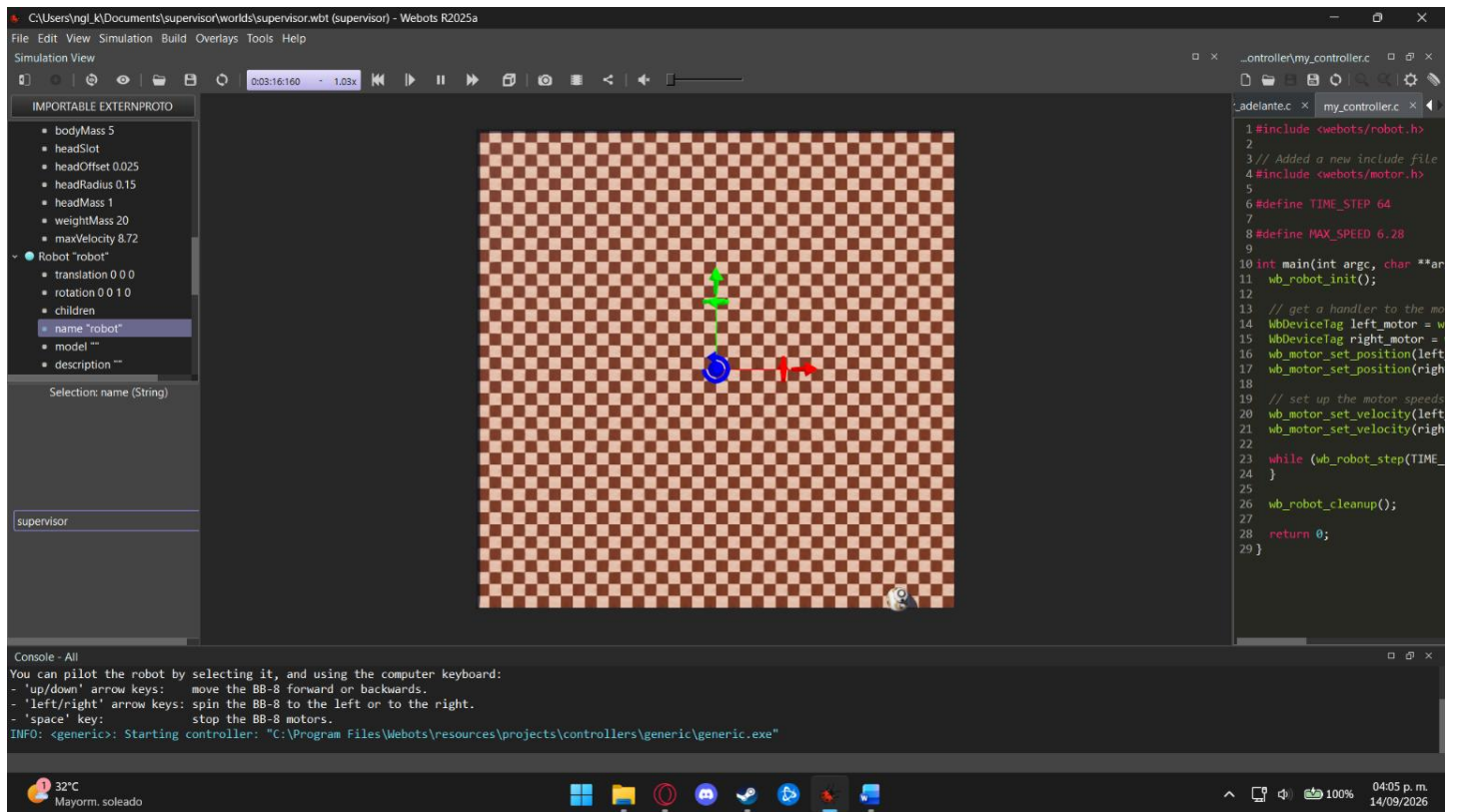
#### 4.- Designación del controlador al Robot BB-8



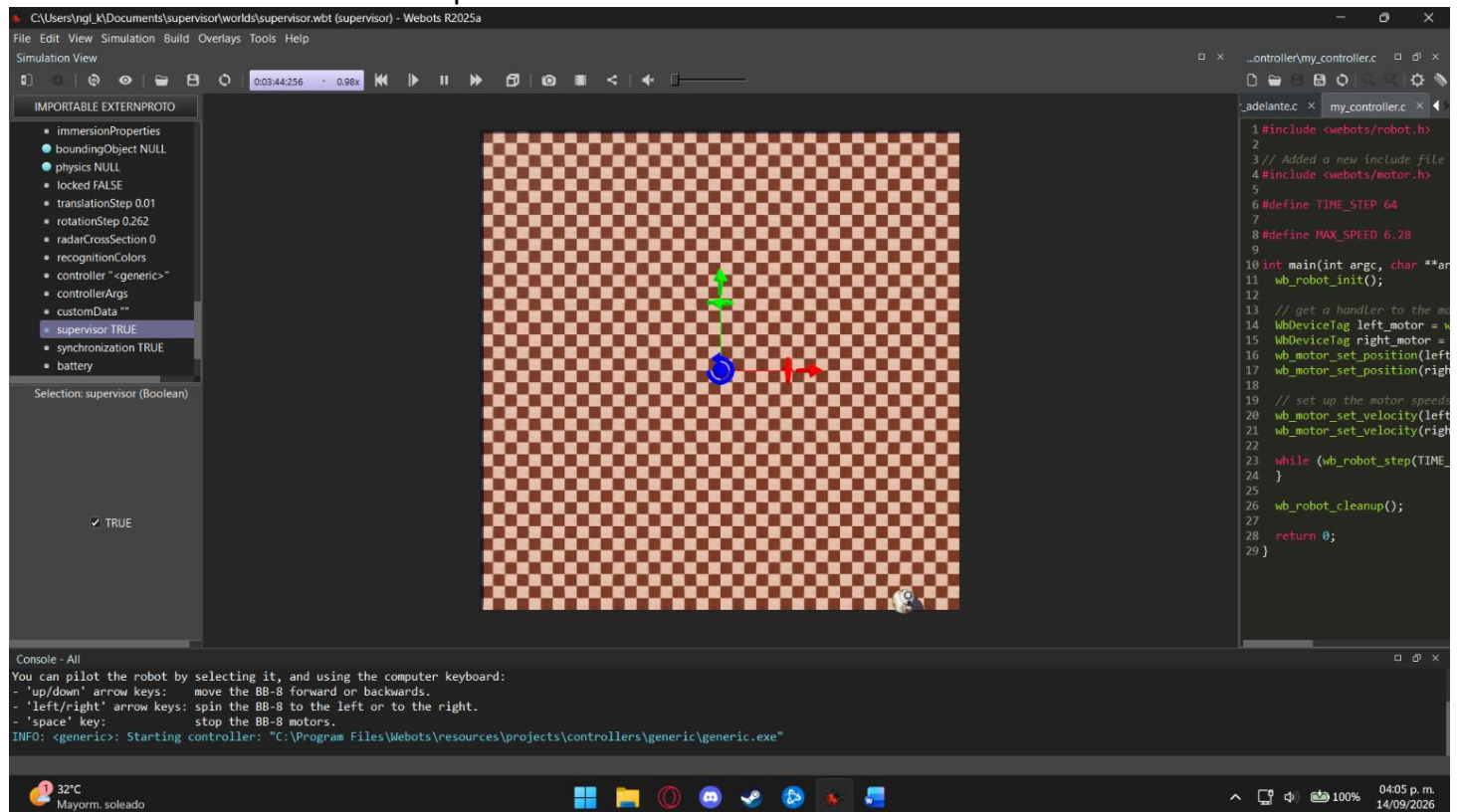
## 5.- Integración de Robot “Supervisor” al mundo



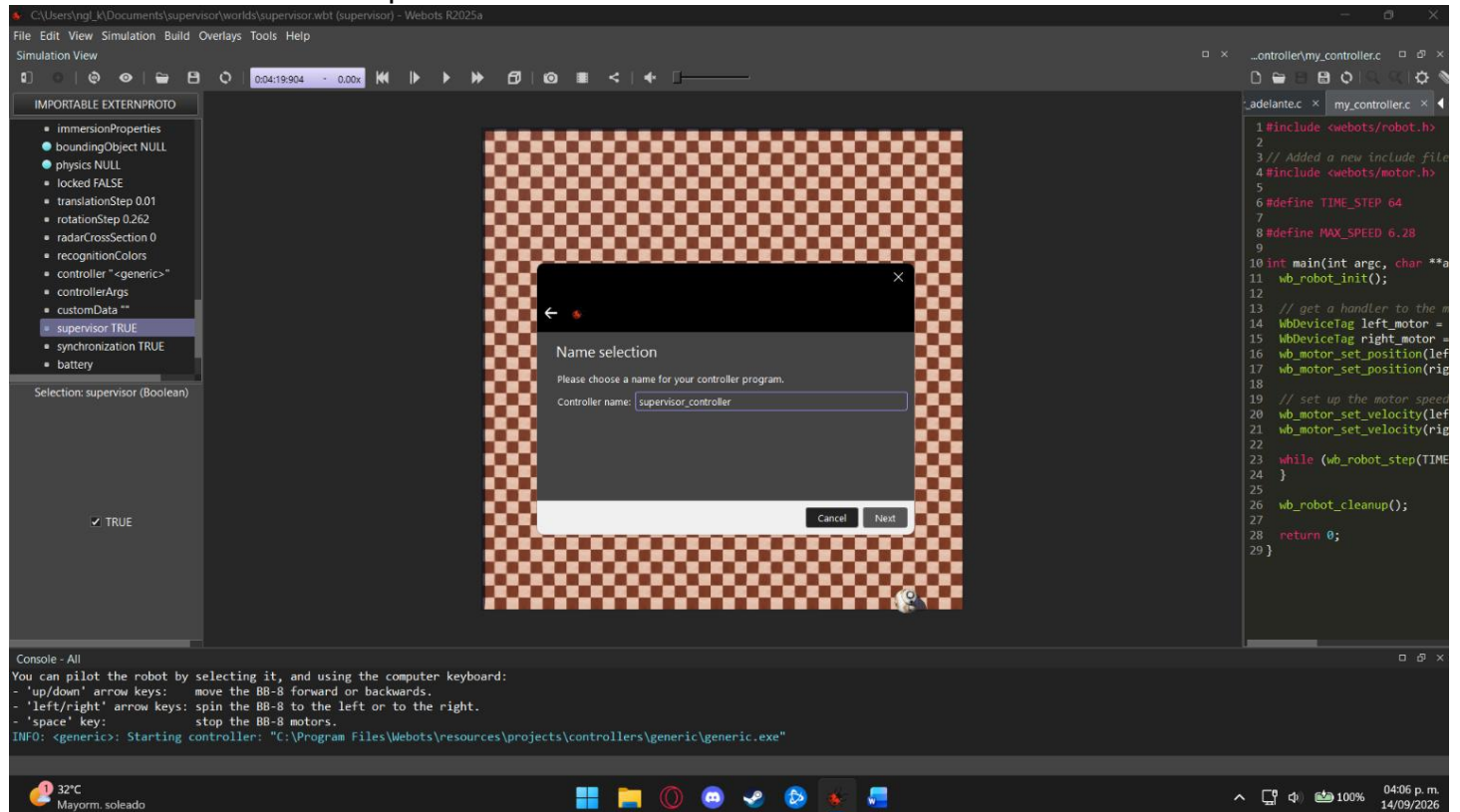
## 6.- Cambiar el nombre del Robot a “Supervisor”

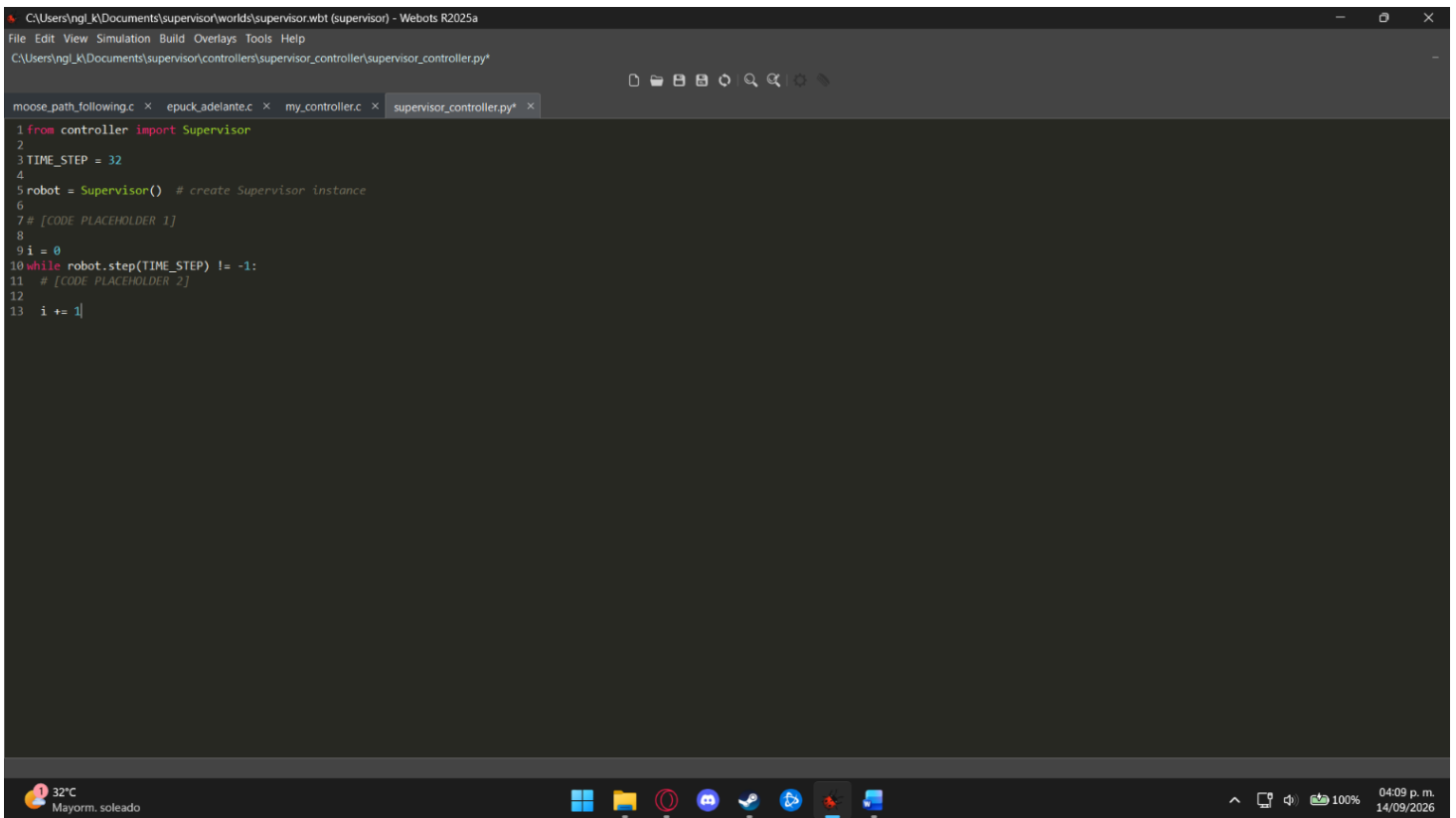


## 7.- Cambiar de False a True Supervisor



## 8.- Crear controlador del supervisor



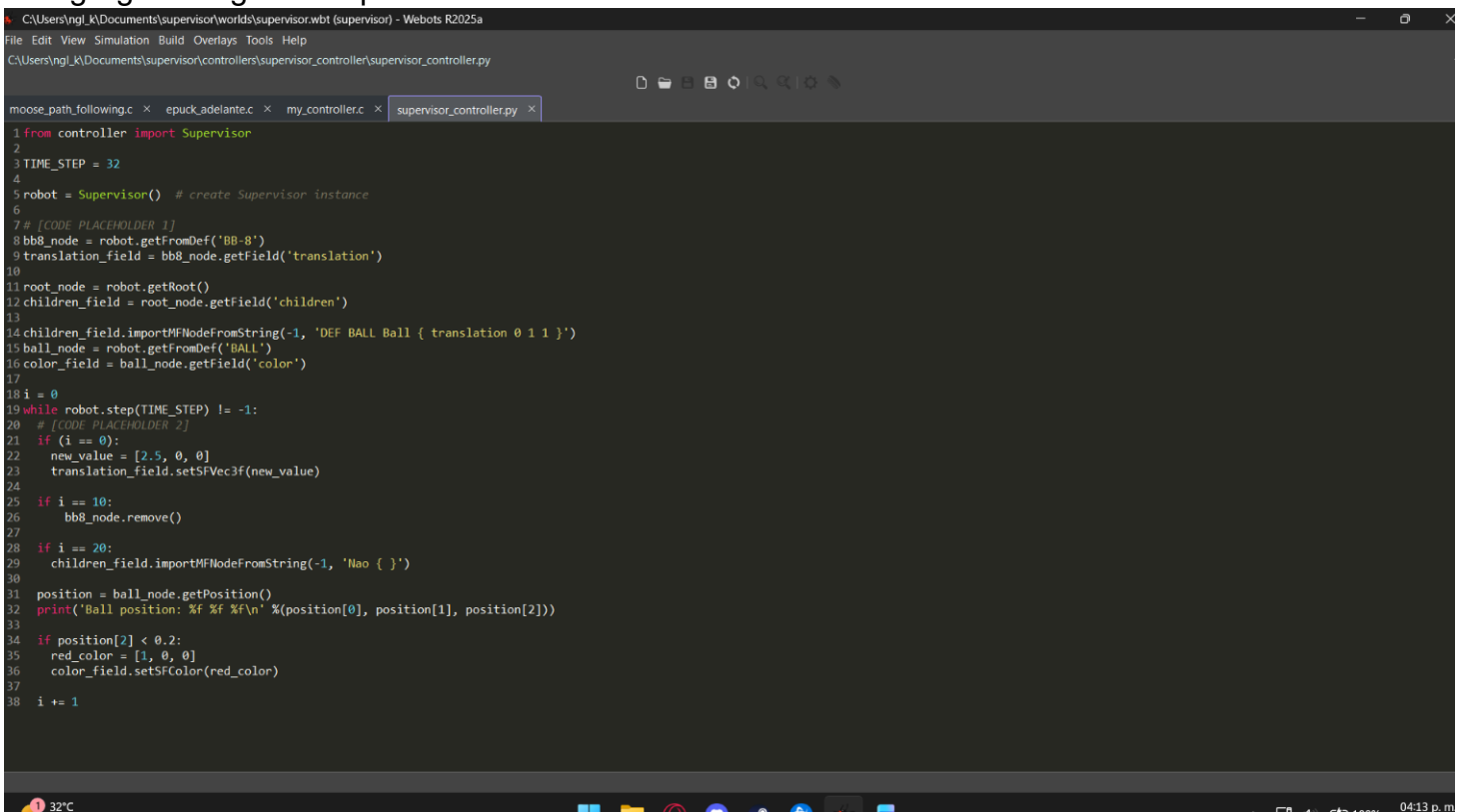


A screenshot of a code editor window titled "C:\Users\ngl\_k\Documents\supervisor\worlds\supervisor.wbt (supervisor) - Webots R2025a". The editor shows a Python file named "supervisor\_controller.py" with the following code:

```
1 from controller import Supervisor
2
3 TIME_STEP = 32
4
5 robot = Supervisor() # create Supervisor instance
6
7 # [CODE PLACEHOLDER 1]
8
9 i = 0
10 while robot.step(TIME_STEP) != -1:
11     # [CODE PLACEHOLDER 2]
12
13     i += 1
```

The taskbar at the bottom shows the system temperature as 32°C, the name "Mayorm. soleado", and the date and time as 04:09 p.m. 14/09/2026.

## 9.- Agregar código del supervisor al controlador anteriormente creado

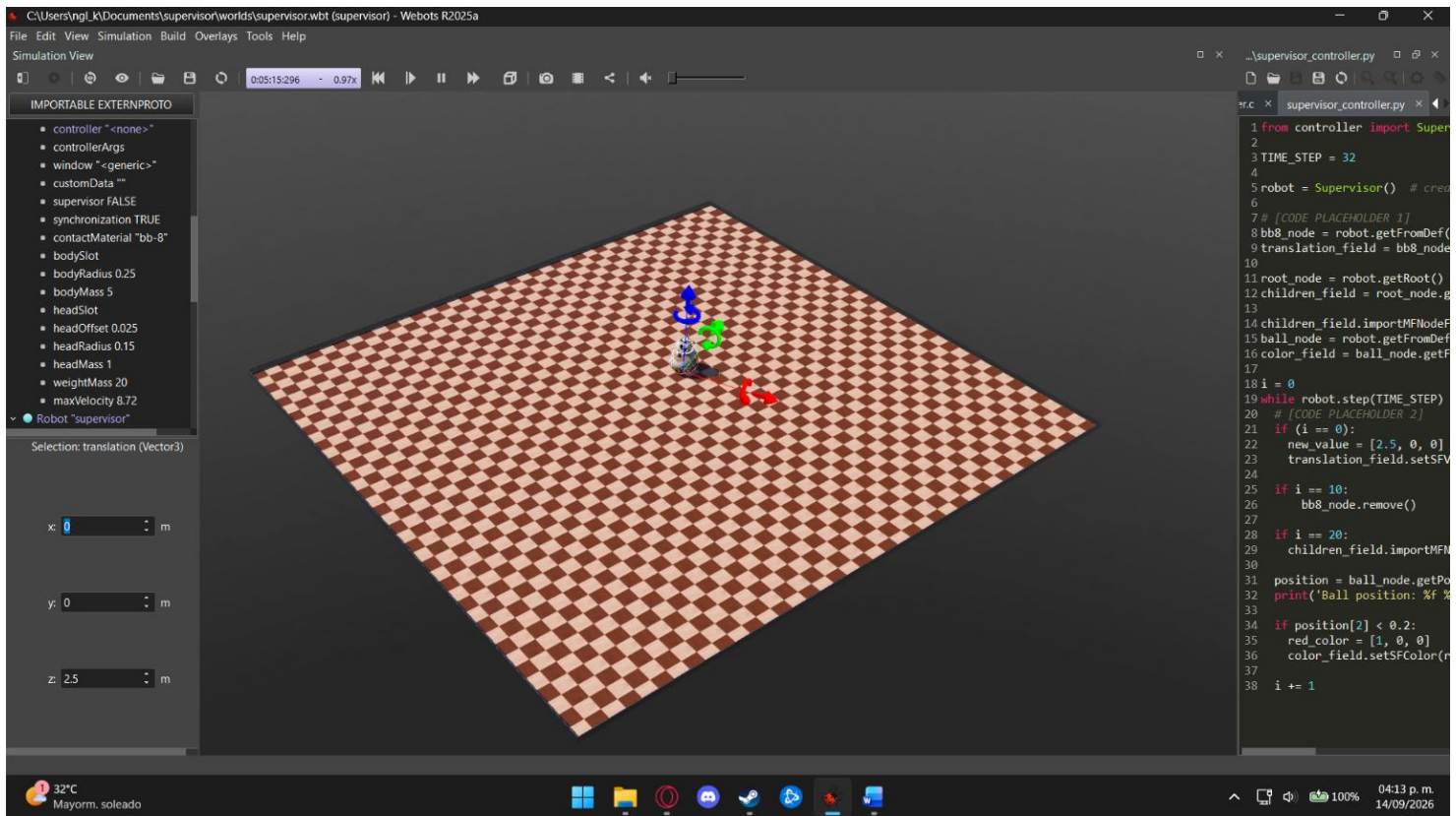


A screenshot of the same code editor window, now showing the updated "supervisor\_controller.py" file. The code has been modified to include logic for controlling a ball and a Nao robot:

```
1 from controller import Supervisor
2
3 TIME_STEP = 32
4
5 robot = Supervisor() # create Supervisor instance
6
7 # [CODE PLACEHOLDER 1]
8 bb8_node = robot.getFromDef('BB-8')
9 translation_field = bb8_node.getField('translation')
10
11 root_node = robot.getRoot()
12 children_field = root_node.getField('children')
13
14 children_field.importMFNodeFromString(-1, 'DEF BALL Ball { translation 0 1 1 }')
15 ball_node = robot.getFromDef('BALL')
16 color_field = ball_node.getField('color')
17
18 i = 0
19 while robot.step(TIME_STEP) != -1:
20     # [CODE PLACEHOLDER 2]
21     if (i == 0):
22         new_value = [2.5, 0, 0]
23         translation_field.setSFVec3f(new_value)
24
25     if i == 10:
26         bb8_node.remove()
27
28     if i == 20:
29         children_field.importMFNodeFromString(-1, 'Nao { }')
30
31     position = ball_node.getPosition()
32     print('Ball position: %f %f %f\n' %(position[0], position[1], position[2]))
33
34     if position[2] < 0.2:
35         red_color = [1, 0, 0]
36         color_field.setSFColor(red_color)
37
38     i += 1
```

The taskbar at the bottom shows the system temperature as 32°C, the name "Mayorm. soleado", and the date and time as 04:13 p.m. 14/09/2026.

## Resultado del tutorial



## Conclusión

En conclusión, a lo largo de todos los tutoriales aprendimos paso a paso cómo usar Webots: primero creamos mundos sencillos con objetos y un robot, luego modificamos el entorno, mejoramos la apariencia gráfica, programamos controladores para que los robots se movieran y evitaran obstáculos, construimos sólidos compuestos, diseñamos nuestro propio robot de cuatro ruedas, lo convertimos en un PROTO reutilizable y finalmente usamos un supervisor para controlar y modificar la simulación.